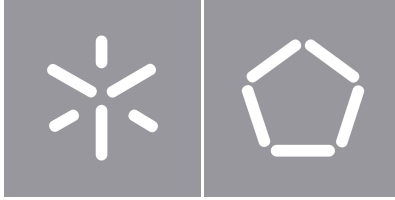


University of Minho
School of Engineering

Pedro Miguel Castilho Martins

Development of a home assistant using open-source large language models



University of Minho
School of Engineering

Pedro Miguel Castilho Martins

**Development of a home assistant
using open-source large language models**

Master's Dissertation
Informatics Engineering

Dissertation supervised by
Prof. Dr. João Miguel Fernandes

Copyright and Terms of Use for Third Party Work

This dissertation reports on academic work that can be used by third parties as long as the internationally accepted standards and good practices are respected concerning copyright and related rights.

This work can thereafter be used under the terms established in the license below.

Readers needing authorization conditions not provided for in the indicated licensing should contact the author through the RepositórioUM of the University of Minho.

License granted to users of this work:



CC BY-NC-ND

<https://creativecommons.org/licenses/by-nc-nd/4.0/>

Acknowledgements

I would like to express my sincere gratitude to all those who have contributed to the completion of this dissertation. Their support, encouragement, and guidance have been invaluable throughout this journey.

First and foremost, I am grateful to my supervisor, Professor João Miguel Fernandes, for his attentive guidance and continuous support. His careful feedback and advice on the structure and writing of this dissertation greatly contributed to its clarity and overall quality.

I am also sincerely grateful to my mentors at DTx, Nelson Ricardo Rodrigues and Manuel Alves, for their invaluable guidance, technical expertise, and for giving me the opportunity to contribute to the R2UT project.

I would also like to acknowledge DTx-Colab for providing the opportunity to carry out this research within a corporate setting, integrated into the Ready to Use Technology (R2UT) project, an initiative within the Portuguese Recovery and Resilience Plan that promotes innovative practices such as modular construction and digital transformation. DTx also provided access to the machines and computing resources required for the development of this work, including GPU infrastructure and the devices used in the smart speaker prototype. This collaboration offered not only practical insights but also the chance to connect academic research with real-world applications, which greatly enriched the value of this dissertation.

Additionally, I am grateful to DST group for the access to their installations during the periods of on-site work, which provided the necessary equipment and workspace that supported the development of the Clarisse assistant.

Finally, I am profoundly grateful to my friends and family for their encouragement and unconditional support. Their presence has been a source of strength and motivation, making this achievement possible.

Statement of Integrity

I hereby declare having conducted this academic work with integrity.

I confirm that I have not used plagiarism or any form of undue use of information or falsification of results along the process leading to its elaboration.

I further declare that I have fully acknowledged the Code of Ethical Conduct of the University of Minho.

University of Minho, Braga, october 2025

Pedro Miguel Castilho Martins

Abstract

This dissertation presents the design and implementation of *Clarisse*, a virtual home assistant developed within the Ready to Use Technology (R2UT) project, an initiative promoting modular and digitally integrated housing solutions, to enable natural interaction in smart domestic environments. The system combines Large Language Models (LLMs) with a full voice interaction pipeline for European Portuguese, covering wake word detection, speech recognition, and text-to-speech synthesis.

The assistant follows a modular, multi-agent architecture in which specialized agents handle device control, information retrieval, and reminder scheduling. After evaluating several open-source LLMs, *Llama-3.3-70B-Instruct* was selected for its balance between reasoning ability and local deployability. The voice interface, built with open-source STT and TTS components, produced natural European Portuguese speech despite the limited availability of high-quality open resources. The assistant was integrated with a smart speaker prototype and a mobile application interface for seamless interaction.

Evaluation showed response times close to the five-second target, with small overruns mainly in voice interactions and clear potential for optimization. Because the system relies on open-source LLMs and modular inference pipelines, its performance scales naturally with advances in model efficiency and hardware capacity. The results demonstrate the feasibility of locally deployed, privacy-preserving assistants, advancing more transparent and adaptable smart-home technologies.

Keywords Artificial Intelligence, Human-Computer Interaction, Intelligent Automation, Internet of Things, Large Language Models, Natural Language Processing, Open-Source Models, Prompt Engineering, Tool Calling, Virtual Assistant.

Resumo

Esta dissertação apresenta a concepção e a implementação da *Clarisse*, uma assistente virtual doméstica desenvolvida no âmbito do projeto Ready to Use Technology (R2UT), uma iniciativa que promove soluções de habitação modulares e digitalmente integradas, para permitir a interação natural em ambientes domésticos inteligentes. O sistema combina Large Language Models (LLMs) com um *pipeline* completo de interação por voz para português europeu, abrangendo a deteção de palavra de ativação, o reconhecimento de fala e a síntese de texto para voz.

A assistente segue uma arquitetura modular e multi-agente, na qual agentes especializados gerem o controlo de dispositivos, a recuperação de informação e o agendamento de lembretes. Após a avaliação de vários LLMs *open-source*, o *Llama-3.3-70B-Instruct* foi selecionado pelo seu equilíbrio entre capacidade de raciocínio e possibilidade de implementação local. A interface de voz, construída com componentes STT e TTS *open-source*, gerou fala natural em português europeu, apesar da disponibilidade limitada de recursos abertos de alta qualidade. A assistente foi integrada com um protótipo de altifalante inteligente e uma interface de aplicação móvel para uma interação fluida.

A avaliação demonstrou tempos de resposta próximos do objetivo de cinco segundos, com pequenas ultrapassagens, principalmente nas interações por voz, e um potencial claro para otimização. Como o sistema depende de LLMs *open-source* e de *pipelines* de inferência modulares, o seu desempenho escala naturalmente com os avanços na eficiência dos modelos e na capacidade do *hardware*. Os resultados demonstram a viabilidade de assistentes implementados localmente que preservam a privacidade, promovendo tecnologias domésticas inteligentes mais transparentes e adaptáveis.

Palavras-chave Assistente Virtual, Automação Inteligente, Engenharia de Prompts, Inteligência Artificial, Interação Humano-Computador, Internet das Coisas, *Large Language Models*, Modelos *open-source*, Processamento de Linguagem Natural, *Tool Calling*.

Contents

1	Introduction	1
1.1	Context	1
1.2	Motivation	2
1.3	Key Challenges	3
1.4	Main Objectives	4
1.5	Document Structure	5
2	State of the Art	6
2.1	Historical Introduction	6
2.2	Chatbots, Virtual Assistants, and RPA	7
2.3	Overview of Virtual Assistants	8
2.4	Large Language Models	8
2.5	Virtual Assistant Development Techniques	10
2.6	Related Work	12
2.7	Conclusion and Limitations	13
3	System Requirements	14
3.1	Use Cases	14
3.2	System Specifications	15
4	Implementation	17
4.1	LLMs	17
4.2	Inference	23
4.3	Architecture and Agents Design	33
4.4	Memory Management	45
4.5	Voice Interface	53
4.6	Deployment and Integration	64
5	Testing and Evaluation	70
5.1	Functional Evaluation	70

5.2	Efficiency Evaluation	74
5.3	Frontend Demonstration	77
6	Conclusions and Future Work	81
6.1	Synopsis of the Work Done	81
6.2	Future Work	82
6.3	Final Appreciation	82
A	Details of Results	88
A.1	Wake Word Training Combinations and Metrics	88
B	Listings	89
B.1	Test Memories and Retrieval Queries	89
B.2	Example: Reminder Test Case Definition (JSON)	91
C	Tooling	93
C.1	Agent Tool Specifications	93
C.2	STT Test Set Phrases	99
C.3	Load Testing Configuration	99

List of Figures

1	Progress in “open weights vs proprietary” intelligence	9
2	Use case diagram for the virtual assistant	14
3	LLM function call process	19
4	Illustration of the prefill and decode phases	23
5	Inference components	24
6	Strengths and limitations of <i>vLLM</i>	28
7	Strengths and limitations of <i>SGLang</i>	29
8	Strengths and limitations of <i>TGI</i>	30
9	Strengths and limitations of <i>TensorRT-LLM</i>	31
10	GitHub stars for selected LLM inference engines as of August 2025	32
11	Levels of autonomy	34
12	Network of agents architecture	36
13	Supervisor agent architecture	36
14	Hierarchical agent architecture	37
15	Custom agent architecture	37
16	Orchestrator task delegation workflow	41
17	Multi-agent system class diagram	42
18	System execution flow for an example request	43
19	Hot Path vs Background	49
20	Overview of tasks and datasets in MTEB	50
21	Voice interaction pipeline	54
22	Voice activity detection flowchart	58
23	Deployment diagram of the <i>Clarisse</i> virtual assistant	65
24	Physical prototype of the smart speaker built with a Raspberry Pi	66
25	Activity diagram of a request from the smart speaker	66
26	Activity diagram of a request from the mobile application	69
27	Average latency breakdown across pipeline stages	75
28	Initial interaction with the assistant	78
29	Device control through the assistant	78
30	Storing and managing user memories	79

31	Reminder creation and notification	79
32	Weather forecasts with textual and visual responses	80

List of Tables

1	Representative open-weight models available by late 2024	20
2	Current flagship open-weight models as of August 2025	21
3	Top 10 open-source embedding models under 5B parameters	51
4	Inference speed comparison between top embedding models	53
5	Performance comparison of STT models on a custom test set	61
6	Evaluation of candidate TTS solutions	63
7	Device control test case	71
8	Reminder definition test case	72
9	Domain continuity test case	72
10	Representative weather test definition	73
11	LLM benchmark performance metrics	76
12	Ranked training combinations with hyperparameters and performance metrics	88
13	Representative subset of tested phrases for STT model evaluation	99

Acronyms

AEC Acoustic Echo Cancellation

AI Artificial Intelligence

ASR Automatic Speech Recognition

CER Character Error Rate

FPH False Positives per Hour

GPU Graphics Processing Unit

HCI Human-Computer Interaction

IoT Internet of Things

LLM Large Language Model

LLMPA LLM-Based Process Automation

LTM Long-Term Memory

MAS Multi-Agent Systems

MCP Model Context Protocol

MoE Mixture-of-Experts

MTEB Massive Text Embedding Benchmark

NCCL NVIDIA Collective Communication Library

NLP Natural Language Processing

NLU Natural Language Understanding

PWA Progressive Web Application

R2UT Ready to Use Technology

RAG Retrieval-Augmented Generation

RPA Robotic Process Automation

STM Short-Term Memory

STT Speech-to-Text

TPOT Time Per Output Token

TTFA Time to First Audio

TTFT Time to First Token

TTS Text-to-Speech

UUID Universally Unique Identifier

VAD Voice Activity Detection

W3C World Wide Web Consortium

WER Word Error Rate

Chapter 1

Introduction

This chapter lays the groundwork for the dissertation by presenting the conceptual landscape surrounding virtual assistant technologies in household environments. It outlines the development of *Clarisse*, a virtual assistant driven by open-source Large Language Models (LLMs), and explores the motivation for adopting open-source Artificial Intelligence (AI) approaches to overcome the constraints of proprietary solutions, highlighting the importance of advancing accessible intelligent interaction systems in contemporary smart home environments.

1.1 Context

The Ready to Use Technology (R2UT) project, integrated into the Recovery and Resilience Plan, represents a national effort to modernize the construction sector through the adoption of modular and pre-fabricated approaches. By promoting housing solutions that are scalable, sustainable, and cost-efficient, R2UT seeks to address pressing social needs such as housing shortages, while simultaneously fostering industrial innovation and digital transformation. Central to this initiative is the ambition to create buildings and living spaces that are not only physically adaptable but also technologically integrated, ensuring that residents and facility managers can interact with their environments in more intuitive and efficient ways.

In this context, digital systems and smart technologies emerge as fundamental enablers of R2UT's vision. The increasing presence of the Internet of Things (IoT) in domestic environments provides an infrastructure of connected devices capable of monitoring and controlling diverse aspects of everyday life, from energy consumption to comfort management. However, the usability of such systems depends heavily on the availability of accessible and natural interfaces. Voice-based assistants, in particular, present a compelling solution by reducing the complexity of interaction and making technology approachable to a wide range of users, including those with limited technical skills.

While voice-based assistants present a compelling solution, their effectiveness has historically been limited by rigid command structures. However, the recent and rapid development of open-source Large Language Models (LLMs) overcomes this barrier, creating new opportunities for building truly natural interfaces. These models offer state-of-the-art conversational capabilities, allowing them to understand complex, multi-part requests and maintain context over time. Crucially, their open-source nature provides

transparency, adaptability, and independence from proprietary ecosystems. Unlike closed platforms, open-source LLMs support customization and integration, aligning perfectly with the long-term sustainability and data privacy goals of the R2UT project. This convergence of modular construction, IoT ecosystems, and open-source AI offers the foundation for developing assistants that not only control devices but also support daily routines, facilitate information access, and enhance user autonomy.

Within this framework, the development of a virtual assistant plays a central role in bridging R2UT's architectural and technological dimensions. By enabling natural, language-based interaction with living spaces, such an assistant directly contributes to creating environments that are more adaptive, accessible, and user-centered. The present dissertation explores this intersection, focusing on the design and implementation of *Clarisse* to demonstrate how advanced AI can reinforce the objectives of the R2UT project and enrich the everyday experience of its users.

1.2 Motivation

The integration of voice-activated systems offers profound benefits for enhancing convenience, accessibility, and overall user interaction with technology. Voice user interfaces provide a hands-free mechanism for operating devices, which is particularly beneficial for individuals with physical disabilities, granting them greater autonomy over their environment (Cleland et al., 2023; Aggar et al., 2022). Beyond accessibility, research has shown that voice commands offer substantial convenience for all users, streamlining the management of lighting, climate control, and entertainment systems (Masina et al., 2020; Pal et al., 2020).

Yet, while the advantages are clear, mainstream assistants such as Amazon Alexa and Google Assistant also introduce critical challenges. Privacy remains a concern, as sensitive personal data is often processed on centralized servers (Bolton et al., 2021). Their proprietary nature limits interoperability and customization, locking users into specific ecosystems (Zimmet, 2020). Moreover, these systems are restricted by predefined skills, rigid command structures, and limited contextual understanding, which constrains their ability to support natural, flexible interactions in IoT environments (Al-Safi et al., 2025).

In response, open-source alternatives like *Home Assistant*¹ have emerged, offering users the power to build highly customizable smart environments without corporate oversight. However, these platforms often demand significant technical expertise for installation, configuration, and maintenance, creating a barrier for non-technical users and limiting widespread adoption (Anik et al., 2024).

These challenges underscore the need for an alternative approach, a voice-based system that prioritizes adaptability, seamless integration and broader accessibility. The integration of LLMs provides the ideal technological foundation to build such a system. This technology enables more natural and flexible interactions, moving beyond rigid, predefined commands to handle complex, multi-step user requests.

By leveraging powerful, open-source LLMs, it becomes possible to create a virtual assistant that is

¹ <https://www.home-assistant.io/>

not only customizable and independent of corporate ecosystems but also significantly more intelligent and adaptive. This dissertation aims to develop such an assistant, demonstrating how these models can meet the demands of modern IoT environments while overcoming the critical drawbacks of existing systems.

1.3 Key Challenges

Developing a virtual assistant capable of controlling devices and executing tasks through natural language using open-source LLMs presents a multifaceted challenge that spans linguistic, technical, computational, and ethical dimensions.

One of the biggest obstacles lies in speech technologies for European Portuguese. High-quality, publicly accessible STT and TTS systems remain scarce, in contrast to the abundance of resources for languages such as English. Most available Portuguese datasets and models are trained on Brazilian Portuguese, which differs significantly in pronunciation, intonation, and vocabulary. These differences tend to be less disruptive for STT, where models can often still produce intelligible transcripts despite occasional errors. However, they become far more pronounced in TTS, where mismatched accents or prosody are immediately noticeable and can impair user experience. While commercial services from providers such as Google, Amazon, and Microsoft exist, their costs hinder adoption and limit their suitability for open-source development.

Another major challenge is ensuring that the virtual assistant can operate reliably across a wide variety of tasks. This requires not only accurate comprehension and generation of natural language responses, but also the ability to execute diverse actions in a consistent and efficient manner. Evaluating such a system therefore goes beyond measuring conversational quality and involves rigorous testing of its task performance, such as managing multi-step interactions, executing real-time operations, recovering from errors, and coping with challenges such as vague instructions, interruptions, or system-level constraints. To address these issues, it is essential to design robust fallback mechanisms and intelligent failure recovery strategies that minimize the risk of unpredictable behavior and ensure dependable performance across tasks.

Another key challenge is the selection and evaluation of suitable open-source LLMs. Models differ widely in their capacity for natural language understanding, reasoning, and compliance with system prompts, context structuring, and prompt engineering strategies. Tool-calling functionalities, which enable interaction with external APIs, are highly sensitive to prompt design, as even small variations can produce drastically different outcomes. For example, a system prompt optimized for *Llama-3.1-70B* may not yield comparable results when applied to *Mistral Large* or *DeepSeek-R1*. This means that models must be assessed with prompts carefully tailored to their specific characteristics, making fair comparative evaluation nearly impossible.

Finally, granting an LLM agency over a physical home environment introduces significant security and privacy considerations. The self-hosted architecture enhances data privacy and user control by avoiding the

transmission of personal data to external, third-party AI providers. However, this approach also transfers the responsibility for security to the system's design. The primary security challenge lies in mitigating the risks of agentic control. For instance, vulnerabilities like prompt injection could be exploited to perform unauthorized actions, demanding a system design that strictly limits each component's capabilities and access to tools.

In summary, building an open-source, LLM-driven assistant requires addressing linguistic limitations, technical integration challenges, computational constraints as well as security concerns. This dissertation tackles these issues through the development of *Clarisse*, a virtual assistant designed to demonstrate how open-source technologies can enable accessible, intelligent, and trustworthy interaction in modern IoT environments.

1.4 Main Objectives

The primary goal of this dissertation was to design, develop, and evaluate a virtual assistant, named *Clarisse*, powered by open-source LLMs. This work aims to demonstrate the practical application of these technologies in real-world scenarios, showcasing their potential to manage IoT ecosystems and extend beyond conventional chatbot interactions. To fulfill this goal, the following objectives were addressed:

- o1 Evaluate state-of-the-art open-source LLMs, selecting models suitable for integration into system agents with a focus on task execution, reasoning, and external API interactions.
- o2 Design the system architecture based on a multi-agent approach, defining agent roles, interaction protocols, and coordination strategies to ensure accurate and reliable operation.
- o3 Implement the virtual assistant by integrating the selected LLMs and tool-calling mechanisms within the designed architecture to enable conversational AI interactions in IoT environments.
- o4 Implement core functionalities beyond device control, including reminders, weather queries, and context-aware interactions, to extend the assistant's capabilities.
- o5 Evaluate Speech-to-Text (STT) and Text-to-Speech (TTS) mechanisms, identifying efficient and accurate open-source solutions for natural voice interaction, particularly in European Portuguese.
- o6 Implement a complete voice interface for the assistant by combining STT, TTS, wake word detection, and voice activity detection, and deploying it both on a smart-speaker device and within a mobile application to enable seamless spoken interaction.
- o7 Test and refine the complete system, evaluating its effectiveness across conversational interaction, task execution, and IoT integration to ensure reliable and consistent operation across contexts.

1.5 Document Structure

This section provides an overview of the structure defined for this dissertation and briefly describes the content of each subsequent chapter.

Chapter 2 presents the state of the art, reviewing the historical development of virtual assistants and analyzing how LLMs have transformed the field. It discusses chatbot development techniques, compares intent-based and LLM-based approaches, and surveys related work relevant to this study.

Chapter 3 formalizes the problem definition and specifies the system requirements. It distinguishes between functional and non-functional requirements and introduces the use cases that guided the design of the proposed solution.

Chapter 4 describes the implementation of *Clarisse*, covering the selection and evaluation of LLMs, inference strategies, the multi-agent architecture, memory management, and the development of the voice interface. This chapter also details the deployment and integration of the system.

Chapter 5 presents the testing and evaluation of the assistant. It outlines the evaluation methodology and dimensions and discusses the results obtained in functional performance, efficiency metrics, and the frontend demonstration.

Chapter 6 concludes the dissertation by summarizing the main findings, reflecting on the contributions made, and suggesting directions for future work to extend the assistant's capabilities and applicability.

Finally, the appendices provide supporting material, including detailed experimental results in Appendix A, test memories, queries, and additional code listings in Appendix B, and agent tool specifications in Appendix C, all of which complement the main chapters.

Chapter 2

State of the Art

This chapter reviews the state of the art in virtual assistants, focusing on how Large Language Models are transforming the field. The discussion covers the historical development, current landscape, and core architectures of virtual assistants, as well as the pivotal role of LLMs in advancing their capabilities. This analysis contextualizes existing work and supports the research presented in this dissertation, which focuses on the development of an LLM-based virtual assistant.

2.1 Historical Introduction

The conceptual foundations of virtual assistants date back to the 1960s with the emergence of the first systems based on Artificial Intelligence (AI). A pioneering example was *ELIZA*, created by Weizenbaum (1966). This program simulated a conversation with a psychotherapist, marking one of the first attempts at human-machine interactivity through natural language. However, the computational limitations of the time restricted its capabilities to highly structured and pre-programmed, pattern-matching responses.

During the 1970s and 1980s, AI systems evolved with the development of expert systems, such as *MYCIN*, used in medicine for diagnostics (Shortliffe et al., 1975). Although not virtual assistants in the modern sense, these technologies laid the groundwork for automated logic and reasoning.

In the 1990s, the advent of more advanced graphical interfaces and increased processing power saw the introduction of assistants like *Microsoft Clippy*. While widely criticized for its limited effectiveness and lack of adaptability, *Clippy* represented an initial step in introducing automated assistants into commercial software.

The true turning point arrived in the 21st century with the integration of advanced speech recognition and machine learning algorithms. In 2011, Apple's *Siri* revolutionized the field, followed by other major advancements, including *Amazon Alexa* (2014) and *Microsoft Cortana* (2015). Initially, these systems relied heavily on rule-based approaches and intent classification to process user commands. This allowed them to interpret a broader range of commands than their predecessors, though they still relied heavily on predefined intents and structured interactions.

Over the past decade, the evolution of virtual assistants has been driven by breakthroughs in deep neural networks, particularly the development of transformers and LLMs. This marks a fundamental

departure from assistants that primarily followed pre-programmed rules to systems capable of reasoning and generating novel responses. Modern assistants like *ChatGPT* and the latest versions of mainstream assistants now use generative AI to understand and respond to natural language in a highly contextual and personalized manner. These systems not only are capable of executing commands but also continuously learn from interactions, significantly enhancing the contextual understanding, fluency, and overall versatility of virtual assistants today.

2.2 Chatbots, Virtual Assistants, and RPA

Although often used interchangeably in non-technical contexts, chatbots, virtual assistants, and Robotic Process Automation (RPA) systems differ significantly in scope, interaction style, and objectives.

Distinguishing these paradigms is important to contextualize the contribution of this dissertation, which lies at the intersection of conversational AI and intelligent task execution.

Chatbots are primarily designed for conversational interaction, typically structured around predefined flows or intent classification. They excel in customer service scenarios, FAQ answering, and basic guidance, but often struggle with maintaining long-term context or handling tasks beyond their scripted domain (Radziwill and Benton, 2017).

Virtual assistants, in contrast, represent a more advanced paradigm. They integrate natural language processing, contextual awareness, and multi-domain task execution (Rastogi et al., 2020). Virtual assistants such as Siri or Alexa can execute commands, integrate with external APIs, and personalize interactions by leveraging contextual and historical information. Unlike basic chatbots, they are not limited to predefined trees but are expected to flexibly adapt to diverse user goals.

RPA, on the other hand, does not focus on conversation but on automating repetitive, structured tasks within enterprise systems. RPA tools replicate human actions at the user interface level, such as filling forms, copying data, or moving files between systems (Wewerka and Reichert, 2020). While conversational AI aims to support natural interaction, RPA focuses on back-office automation and efficiency, often operating without direct user engagement.

In short, chatbots specialize in conversation, virtual assistants in task-oriented interaction through natural language, and RPA in back-office automation without dialogue. The system developed in this dissertation aligns most closely with the virtual assistant paradigm, since it not only engages in dialogue but also executes tasks, integrates with external services, and adapts its responses to user goals. For this reason, the next section focuses on virtual assistants, their foundations, and their role in enabling natural interaction with technology.

2.3 Overview of Virtual Assistants

Virtual assistants are digital services designed to simulate human conversation and provide personalized responses based on user input (Li et al., 2024). Functioning as intelligent agents, they are transforming various aspects of daily life, with applications spanning healthcare, entertainment, and education (De Blanes Sebastián et al., 2022).

A primary focus of academic and industry research is their potential to enhance human productivity by reducing cognitive load. By offloading mental effort, these systems have been shown to significantly improve task performance in complex problem-solving scenarios (Brachten et al., 2020). This core capability finds critical application in diverse fields. For example, they serve as powerful accessibility tools for mobile devices, supporting users with motor and visual impairments (Siebra et al., 2018), and assist clinicians in demanding domains like vascular surgery with diagnosis, disease management, and administrative tasks (Li et al., 2024). Underpinning this continuous evolution are advances in artificial intelligence, particularly the machine learning and neural network models that drive their development (Tsyra et al., 2020).

While their applications are broad, virtual assistants have emerged as a cornerstone technology in enabling voice-controlled interactions within smart environments. Their technical architecture typically integrates Automatic Speech Recognition (ASR), Natural Language Understanding (NLU), and Text-to-Speech modules, working in concert to interpret user commands and deliver appropriate responses or actions. Core functionalities of virtual assistants include speech-to-text conversion, natural language processing, and integration with external devices or services via APIs, critical for executing the interpreted commands within interconnected smart environments.

2.4 Large Language Models

LLMs have become a pivotal component in the field of artificial intelligence, particularly in Natural Language Processing. These models, characterized by their large parameter sizes, have shown remarkable capabilities in understanding and generating human language, leading to significant advancements in various applications. LLMs can simultaneously generate and analyze text, effectively managing extensive contextual information and complex linguistic patterns (Zhao et al., 2023).

The rapid progress of LLMs stems from two interconnected breakthroughs, a foundational architectural shift and the subsequent discovery of scaling laws. The pivotal architectural innovation was the introduction of the Transformer model, as detailed by Vaswani et al. (2023). This architecture revolutionized NLP by enabling the parallel processing of text, overcoming the sequential bottlenecks of previous models like recurrent neural networks. This efficiency unlocked the potential for training models of unprecedented size. Concurrently, this led to the discovery of scaling laws, the principle that as model parameters and training data increase, performance improves predictably, and entirely new emergent capabilities such as sophisticated reasoning and complex problem solving skills appear that were not present in smaller

models (Kaplan et al., 2020). Therefore, the combination of the Transformer’s parallelizable design and the predictable benefits of scaling are the primary drivers behind the current state of the art in language models.

The immense power and resource requirements resulting from these scaling laws have led to a fundamental divergence in the field, creating a distinction between open-source and closed-source models. Closed-source systems, such as *OpenAI’s GPT-4o*, often represent the performance frontier but come with significant limitations in transparency, flexibility, and user control. These models impose constraints on customization and adaptability, effectively tethering users to proprietary ecosystems.

In contrast, the open-source community has become an increasingly important part of the LLM landscape. Efforts by organizations such as *Meta (LLaMA)*, *Mistral*, and others have broadened access to advanced models and tools, allowing a wider range of researchers and developers to build on this work. Open availability also supports greater transparency, since model architectures, training practices, and related ethical considerations can be examined more closely (Manchanda et al., 2024). While the term “open-source” traditionally implies complete transparency including training data and methodologies, many of these accessible models are more accurately described as “open-weights”, providing pre-trained weights while maintaining proprietary training procedures. For the purposes of this research, open-weights models offer sufficient accessibility and functionality.

As shown in Figure 1, which summarizes performance across a set of ten rigorous evaluations, the progress of open-weights models has accelerated significantly. Although a performance gap remains, the most advanced open-weights models approach the performance of many proprietary systems at a steady pace.

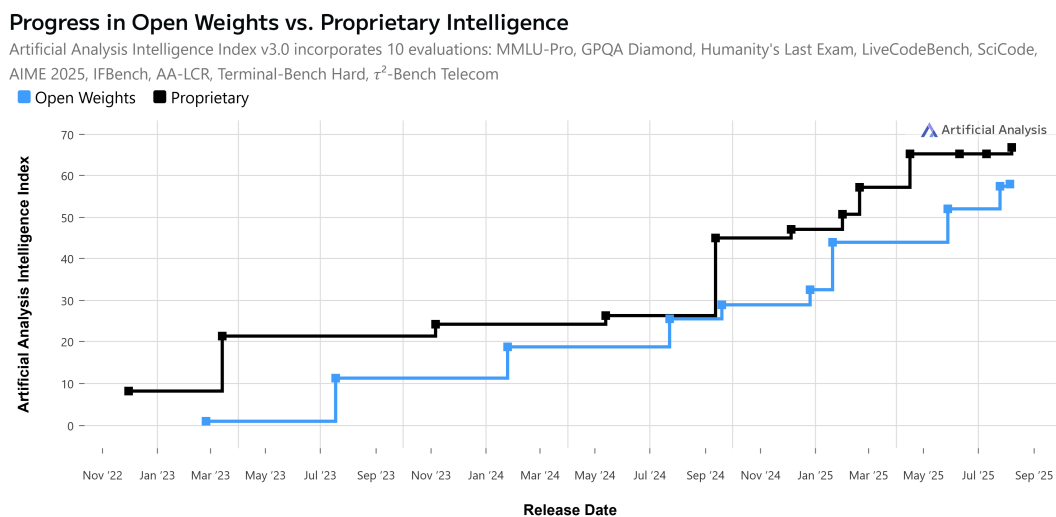


Figure 1: Progress in “open weights vs proprietary” intelligence, from *Artificial Analysis Intelligence Index*¹

The practical deployment of open-weights models has been significantly enhanced through archi-

¹ <https://artificialanalysis.ai/models/open-source#progress-in-open-weights-vs-proprietary-intelligence>

tectural innovations and efficiency techniques that improve performance and local deployment feasibility. Mixture-of-Experts architectures selectively activate subsets of parameters during inference, achieving higher performance with fewer active computations. Distillation techniques allow the creation of smaller “student” models that replicate the behavior of larger “teacher” models, making advanced capabilities accessible with reduced computational requirements (Sanh et al., 2020). Additionally, quantization methods reduce parameter precision to decrease memory usage and accelerate inference without substantial accuracy loss (Zafir et al., 2019). These optimization techniques are particularly valuable for open-weights models, as they enable researchers and developers to deploy and customize advanced language capabilities locally without relying on proprietary cloud-based services or extensive computational infrastructure.

2.5 Virtual Assistant Development Techniques

This section outlines the background and examines related studies on general virtual assistant development techniques, concentrating on two primary methods commonly employed across various domains: intent-based approaches (Section 2.5.1) and LLM-based approaches (Section 2.5.2).

2.5.1 Intent-Based Approaches

Intent-based virtual assistants focus on identifying user intents, which represent the underlying goals or purposes expressed in a conversation. By recognizing these intents, the virtual assistants deliver contextually relevant responses. In addition to detecting intents, they also identify entities, which are key pieces of information extracted from user input, such as dates, locations, or product names, that further refine the virtual assistant’s understanding of the user’s request (Alsafari et al., 2024). The response generators in intent-based assistants primarily rely on knowledge bases composed of training phrases, intents, and responses (Luo et al., 2021). These virtual assistants use sophisticated NLU algorithms and machine learning techniques to analyze user inputs and classify their intentions (Luo et al., 2021). As a result of this analysis and intent classification, the virtual assistant can be trained to respond to user requests with predefined responses.

Intent-based virtual assistant development platforms provide the infrastructure to simplify the creation of intent-based virtual assistants via forms or graphical notations. Several platforms, such as *Dialogflow*, *Lex*, *Watson*, and *Rasa*, dominate the current landscape, each offering specific functionalities, advantages, and limitations, and enabling virtual assistant development through the definition of user intents to be recognized (Perez-Soler et al., 2021). While these tools have facilitated virtual assistant development, they often come with inherent trade-offs regarding flexibility, scalability, and reliance on pre-built solutions.

Rasa², an open-source framework for intent-based virtual assistants, incorporates modules for ASR, spoken language understanding, and dialogue management. These modules transform speech to text,

² <https://rasa.com/>

parse user intents, track dialogue states, and generate appropriate responses. Additionally, platforms like *Dialogflow*³ by Google contribute to the development of these systems, supporting both commercial and academic research. *Dialogflow* lets app developers provide their users to interact with interfaces through voice and text exchanges enabling developers to focus on other integral parts of app creation rather than on delineating in-depth grammar rules (Nuruzzaman and Hussain, 2018).

However, the development and maintenance of intent-based systems demand substantial resources, as the developer must identify all potential user intentions, formulate appropriate virtual assistant responses, and establish any required functionalities or data for the system (Alsafari et al., 2024).

2.5.2 LLM-Based Approaches

LLMs offer powerful functionalities for NLP tasks, such as text understanding, comprehension, generation, and categorization. As a result, they serve as compelling solutions for applications like translation and intent analysis during text processing (D'urso et al., 2024). These characteristics help address challenges with intent-based virtual assistants, reducing the need for extensive specification of user intents, conversation flows, and outputs, with a more natural and flexible conversation experience.

Virtual assistants built with LLMs, like ChatGPT or Gemini, are typically general purpose and open-domain. To focus the output of LLMs on specific interactions, some researchers suggest developing task-oriented virtual assistants on top of LLMs using prompting, which involves prepending textual instructions and examples of the intended interaction to the LLM input to improve its output (Cuadrado et al., 2024).

With the gaining traction of these models, new tools and frameworks have emerged to simplify the development of applications that harness the power of LLMs for complex, multi-step reasoning tasks and integration with external systems.

For example, LangChain is a widely used open-source framework designed to streamline the development of conversational AI applications. It offers tools and abstractions to integrate LLMs with various computational and informational resources. Additionally, LangChain facilitates rapid application development by providing modular components and customizable pipelines, which are beneficial for creating bespoke AI applications (Topsakal and Akinci, 2023). Another example is the framework proposed in Cuadrado et al. (2024), called *Taskyto*, which enables lighter specifications and more natural conversations without requiring predefined intents. It utilizes a YAML-based format to orchestrate task execution across the integrated models, leveraging structured prompts to define the internal model's behavior. Additionally, the *Prompt Sapper* in Cheng et al. (2023) tool offers a no-code environment for building AI chains, enabling users to create prompt-based AI services through visual programming, thus lowering the barrier for non-programmers.

³ <https://dialogflow.cloud.google.com>

2.6 Related Work

This section reviews the existing literature on the use of Large Language Model in virtual assistants, with a focus on applications relevant to the development of LLM-based assistants capable of performing complex actions. The applications reviewed align with the objectives of this thesis, which focuses on implementing an LLM-based virtual assistant. They cover a wide range of fields, from digital banking to mobile app automation to healthcare communication and intent-based networking.

The integration of LLMs into virtual assistants has emerged as a significant trend in recent years, driven by the undeniable achievements of this relatively new technology of LLMs. This ability to understand natural language, unlike any other technology, allows these systems to tackle increasingly complex tasks, using only human-like input, eliminating the need for structured inputs or predefined commands. As a result, researchers and developers have already begun exploring a wide range of applications across various domains.

For example, in D'urso et al. (2024) the researchers explored the use of LLM-powered chatbots to enhance Intent-Based Networking systems by improving intent acquisition and translation. Their research highlights how conversational agents can bridge the gap between user intentions and automated network configurations. They proposed a chatbot that leverages Prompt Engineering, Retrieval-Augmented Generation (RAG), and Function Calling to improve accuracy in intent translation.

Xu et al. (2024) introduced a general-purpose device designed for LLM interactions, integrating voice, multimodal data processing (audio, video, sensor data), and privacy-focused local processing. The proposed system features a lightweight, compressed LLM deployed on the device itself, optimized through quantization, distillation, and memory-efficient techniques to minimize computational overhead while maintaining performance. However, for more complex and resource-intensive tasks, the device can dynamically offload processing to a cloud-based model ensuring scalability and performance. Furthermore, the study shows how versatile LLMs can be in order to be integrated in a resource-constrained edge device while maintaining low latency, real-time responsiveness, and intelligent decision-making capabilities.

Yang et al. (2024) introduced *Talk2Care*, an LLM-powered voice assistant designed to facilitate communication between older adults and healthcare providers by leveraging natural language processing for enhanced accessibility and information summarization. Their study highlights key benefits such as improved patient-provider communication, reduced workload for healthcare professionals through AI-generated summaries, and increased user engagement via voice-based interactions. This is a notable example of how, despite common digital literacy challenges among older adults, the use of AI-driven virtual assistants is well received and even preferred for disclosing personal health information, underscoring the potential of LLMs in healthcare domain.

Guan et al. (2024) focused on LLM-Based process automation (LLMPA), showcasing an intelligent virtual assistant capable of executing multi-step operations within mobile applications. Their work presents a system designed to interpret high-level user commands and decompose them into executable actions. It

integrates natural language understanding, interface element detection, and sequential action prediction to perform real-world tasks within mobile environments. Unlike traditional assistants that rely on predefined API calls, LLMPA dynamically interprets user intents and simulates human-like interactions, significantly enhancing flexibility in task execution. The researchers demonstrated their approach by deploying the system within the Alipay app, showcasing its ability to navigate app interfaces, input information, and complete transactions based on high-level instructions. This study provides a foundation for building assistants that not only engage in dialogue-based interactions but also perform actions dynamically, which is an important consideration in this LLM-based virtual assistant.

2.7 Conclusion and Limitations

The state of the art shows that LLMs have transformed virtual assistants from intent-based, rule-driven systems into flexible agents capable of natural interaction, reasoning, and tool execution. Open-weight models, combined with advances in inference engines and multi-agent orchestration, are making LLM-based assistants increasingly viable outside proprietary ecosystems, demonstrating potential for democratizing advanced AI while maintaining adaptability and transparency. However, several limitations remain in existing approaches:

- **Computational and deployment constraints:** Even with optimizations (quantization, distillation, mixture-of-experts), deploying high-performing LLMs locally is resource-intensive and often impractical for consumer-grade hardware.
- **Evaluation challenges:** Fair benchmarking is difficult due to the variability in prompt engineering and model behavior across architectures, hindering consistent comparison.
- **Integration complexity:** LLM-based assistants often suffer from “model lock-in”, where system design and prompt structures become tightly coupled to a specific model, limiting portability and maintainability.
- **Ethical and security concerns:** Prior work notes risks such as prompt injection, data leakage, and misuse of tool-calling capabilities, but comprehensive safeguards are still underdeveloped.
- **Accessibility gaps:** While progress has been made in healthcare and mobile domains, many assistants are still limited in multilingual support, multimodality, and inclusivity for less-resourced languages or diverse user populations.

These gaps motivate the present work, which explores how open-source LLMs, combined with modular multi-agent architectures and voice pipelines adapted for European Portuguese, can address some of these challenges and move toward more trustworthy, accessible, and efficient virtual assistants.

Chapter 3

System Requirements

This chapter formalizes the requirements for the proposed virtual assistant. Building upon the problem context discussed previously, this section translates the high-level project goals into a concrete set of specifications. It begins by outlining user-centric interactions through use cases. Subsequently, these are detailed as specific functional requirements, defining what the system must do, and non-functional requirements, which describe the quality attributes it must satisfy, such as performance, reliability, and usability. This formal elicitation provides a clear and verifiable foundation for guiding the system's design and implementation.

3.1 Use Cases

To outline the system's core functionalities and align them with the objectives of the project, a use case diagram was developed. Figure 2 illustrates the actions available to users within the system.

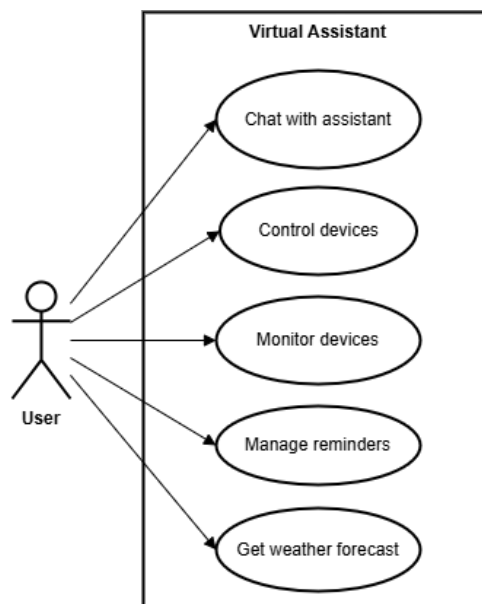


Figure 2: Use case diagram for the virtual assistant

The system enables users to manage their smart home devices and personal tasks through natural

language, either by voice or text. Core interactions focus on real-time device control, for example, issuing commands such as “*turn on the living room lights*” or making status inquiries like “*is the heater running?*”.

Beyond device management, the assistant supports personal organization by handling reminders. Users can set time-based alerts, for instance, “*remind me to go shopping tomorrow morning*”, and receive notifications through the mobile application or smart speaker. The assistant also provides access to external information, including local weather forecasts.

In addition, the system supports open-ended conversation, ensuring that requests outside specific tasks still receive natural and meaningful responses. The assistant also incorporates a memory mechanism to improve personalization over time. During conversations, it can store relevant information about the user, such as preferences, routines, or facts explicitly provided, and recall it in future interactions. Users retain control over these stored memories, with the ability to review, update, or delete them as needed. This ensures that personalization is both adaptive and transparent.

3.2 System Specifications

This section identifies and clearly defines the essential functionalities and qualities expected from the virtual assistant in a smart home context. Building on the use cases outlined previously, the following functional and non-functional requirements have been determined to ensure the assistant effectively meets user needs, performs reliably, and complies with relevant standards.

3.2.1 Functional Requirements

The virtual assistant is designed to simplify and enhance interaction between users and their smart home devices. Unless otherwise specified, the following functional requirements presuppose that interaction occurs primarily through natural language, whether spoken or written, and are defined accordingly.

FR1 The user shall be able to request which smart home devices are available to them.

FR2 The user shall be able to request the status of any available device.

FR3 The user shall be able to issue commands to control any available device.

FR4 The user shall be able to create reminders.

FR5 The user shall be able to receive reminder notifications through push notifications on the mobile application.

FR6 The user shall be able to receive reminder notifications through voice output on the smart speaker.

FR7 The user shall be able to request current weather conditions and forecasts.

FR8 The system shall use European Portuguese for all text and voice interactions.

- FR9** The system shall provide a relevant conversational response when a user request does not correspond to a predefined task.
- FR10** The system shall adapt its behavior to the active user by learning user preferences and constraints through past interactions.
- FR11** The user shall be able to review and delete personal information gathered by the automatic learning mechanism at any time through the mobile application's settings interface.

3.2.2 Non-Functional Requirements

To ensure the household assistant delivers reliable performance, scalability, and compliance with essential standards, the system must adhere to the following non-functional criteria:

- NFR1** The system shall provide a text interface accessible through a mobile application.
- NFR2** The system shall provide a voice interface accessible through a mobile application.
- NFR3** The system shall provide a voice interface accessible through a smart speaker.
- NFR4** The system shall initiate a response within 5 seconds.
- NFR5** The AI services of the system shall be deployable on a single NVIDIA H200 Graphics Processing Unit (GPU).
- NFR6** The system shall support horizontal scaling to serve an increasing number of simultaneous users.
- NFR7** The assistant must encrypt all communication with the external device system to ensure data security.
- NFR8** The system shall use only AI models and services under licenses compatible with the project's open-source goals.

3.2.3 Summary

The requirements presented in this chapter establish a complete and verifiable specification of the virtual assistant. Functional requirements define the concrete capabilities that enable users to interact naturally with their smart home and personal tasks, while non-functional requirements specify the quality attributes necessary to ensure performance, reliability, security, and maintainability. Together, these requirements provide a stable foundation for the system's design and implementation, guiding architectural decisions in the following chapter.

Chapter 4

Implementation

This chapter presents the implementation of the proposed intelligent household voice assistant, detailing the selection and evaluation of suitable Large Language Models (LLMs), the inference strategies adopted to balance performance and efficiency, and the design of a multi-agent system architecture that coordinates specialized components. It further describes the mechanisms for memory management, enabling personalization and contextual continuity, and the development of the voice interface that integrates speech recognition and synthesis for natural interaction. Together, these elements illustrate the design decisions and practical considerations that shaped the final system.

4.1 LLMs

A cornerstone of this research is the exploration and evaluation of freely accessible, open-source LLMs. The selection of an appropriate LLM is critical, as it fundamentally shapes the capabilities and limitations of the virtual assistant. This section surveys several prominent open-source LLMs, outlining their key characteristics and filtering those that appear most promising for this use case, while taking into account the memory constraints of the development environment.

Given the core requirement of controlling household IoT devices and executing various other tasks, robust tool calling capabilities are essential. This feature enables the LLM to interact with external systems and execute specific actions, such as adjusting thermostat settings or turning on lights. In addition, due to the conversational nature of the assistant, it is essential to examine its ability to adopt a consistent persona and respond appropriately to user inputs, while minimizing the risk of unintended information disclosure or susceptibility to jailbreak attempts.

Finally, this analysis also considers a multi-agent architecture. Rather than seeking a single, universally superior model, the aim is to understand the strengths and limitations of different LLMs across core assistant tasks, such as setting reminders, maintaining a natural conversational style, or controlling smart devices. While practical constraints, including memory usage, may favor relying on a single model, these insights can still indicate whether one versatile LLM is sufficient or whether a combination of specialized models would deliver a more effective solution.

4.1.1 Tool Calling

A defining requirement of the virtual assistant is the ability to interact with its environment by retrieving information, executing commands and interfacing with external services, extending beyond merely producing fluent responses. This capability is enabled by a process known as **tool calling**, wherein the LLM generates structured text that specifies a function to invoke and its corresponding arguments. While this capability has become common in state-of-the-art models, tool calling is not an inherent property of all LLMs. Rather, it is a specialized feature deliberately engineered into models through extensive fine-tuning. During this training phase, models learn to recognize when user requests require external actions and to produce outputs that conform to a predefined format.

It is important to underscore that the LLM itself has no intrinsic ability to execute actions or interact with external systems. Its role is strictly limited to processing text inputs and producing text outputs. Everything beyond this, including the interpretation of model outputs as function calls and the subsequent execution of those functions, is implemented in the surrounding software infrastructure. Inference frameworks bridge this gap by exposing tools to the model through predefined schemas that describe their names, parameters and expected structure. These frameworks often provide chat templates to format the input prompts in ways that align with the model's training and maximize its likelihood of emitting tool calls in the prescribed schema. At inference time, two primary strategies are employed to generate valid tool invocations from these schemas.

- **Guided decoding:** In this approach, the output space of the LLM is restricted according to the tool schema. Invalid tokens are masked during decoding (their probability is set to negative infinity, which makes it impossible for the model to select them), which ensures that any produced call is syntactically valid. This mechanism is particularly effective when a tool call is mandatory or when a strict output format is required.
- **Parser-based extraction:** In this approach, the LLM generates text that is unconstrained at inference time but the LLM has been trained to follow a specific format for tool invocation. A corresponding parser is then implemented to interpret this text and extract a valid tool call. This strategy enables automatic tool choice, granting the model the autonomy to decide between invoking a tool and responding in natural language. However, this approach provides no syntactic guarantees, and malformed calls or hallucinated arguments must be handled by the execution environment. But because guided decoding preemptively forces a tool call structure, parser-based extraction remains the necessary approach for automatic tool selection, where the model's primary decision is whether to invoke a tool in the first place.

This separation of responsibilities establishes the LLM as a planner, generating only text-based instructions, while the application acts as the executor that parses, executes and reintegrates results into the dialogue. Figure 3 illustrates this loop, emphasizing the separation between planning (LLM) and execution (infrastructure).

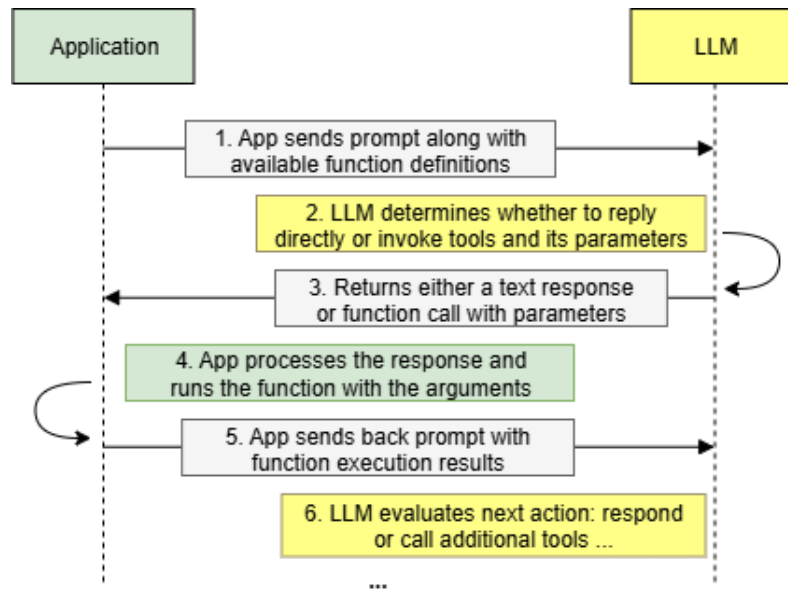


Figure 3: LLM function call process

While this interaction loop captures how tools are invoked and executed within a single application, the Model Context Protocol (MCP) represents an attempt to extend this capability into a standardized protocol across contexts¹. MCP defines a common way for tools to be described, discovered and invoked dynamically, avoiding the need to hard-code wrappers for each service. This makes MCP particularly attractive for connecting LLMs to big mainstream services such as productivity suites, communication hubs or enterprise resource systems, where the same tool may need to be reused across very different contexts. However, this generality can be counterproductive in focused assistants. Because MCP tools are intentionally generic, user prompts must often guide the model explicitly toward the correct tool and arguments. In applications where users are expected to give short, underspecified requests such as “*dim the lights a bit*” or “*remind me after lunch*”, this reliance on explicit signaling adds friction and diminishes intuitiveness.

In the present project, the tools are tailored to very specific functions such as controlling IoT devices, handling reminders, and managing local tasks, and they are unlikely to be reused outside this assistant. Designing them for MCP would therefore add complexity without delivering meaningful benefits. A more targeted approach, with tools and prompts optimized for the assistant’s scope, better supports the goal of ubiquity and natural interaction. MCP remains highly promising for general-purpose agents that need to integrate with many external services, but in this case direct function-calling APIs proved more practical and better matched to the assistant’s objectives.

¹ <https://modelcontextprotocol.io/>

4.1.2 Open-Source LLMs

Open-source LLMs offer critical benefits for this research, including transparency, customizability, and the ability to run locally without reliance on proprietary APIs. These qualities encourage reproducibility and foster a vibrant ecosystem of community-driven improvements, making them ideal for developing a flexible and future-proof virtual assistant. The selection of an appropriate model, however, is contingent on the state-of-the-art at the time of development, hardware constraints, and specific task requirements.

The LLM Landscape at the Time of Selection

The initial model selection for this project was conducted in December 2024, during the development phase of the assistant. At that time, the open-source landscape was characterized by a range of powerful, dense models that had demonstrated strong performance in reasoning and instruction-following. The primary candidates considered are summarized in Table 1.

Table 1: Representative open-weight models available by late 2024

Model	Parameters (Active)	Context	Intelligence	License
Llama-3.3-70B-Instruct	70B	128k	28	Llama Community License
Llama-3.1-405B-Instruct	405B	128k	26	Llama Community License
Qwen-2.5-72B-Instruct	72B	131k	28	Qwen License
QwQ-32B-Preview	33B	32.8k	28	Apache-2.0
DeepSeek-V2.5	236B (21B)	128k	21	DeepSeek License
Phi-4-Base	14B	16k	25	MIT

The column *Parameters (Active)* shows the total number of parameters and, for Mixture-of-Experts (MoE) models, the number of active parameters used during inference. Dense models have active parameters roughly equal to the total, while MoE models activate only a fraction. The *Intelligence* column reports the *Artificial Analysis Intelligence Index*², which aggregates performance across various benchmarks. While this index is a useful comparative signal, it should be interpreted cautiously, as such benchmarks may not fully reflect real-world assistant performance.

Analysis of the available models at the time revealed a crucial trade-off between model size, architectural complexity (dense vs. MoE), and ecosystem maturity. Models such as *Phi-4-Base* and *QwQ-32B-Preview* were quickly ruled out due to their relatively short context windows (16k and 32.8k tokens, respectively), especially when compared to alternatives offering 128k or more. Conversely, extremely large models like *Llama-3.1-405B-Instruct* and *DeepSeek-V2.5* offered only marginal improvements in reasoning benchmarks relative to their massive parameter counts.

Among the remaining candidates, the most competitive options were *Qwen-2.5-72B-Instruct* and *Llama-3.3-70B-Instruct*. While Qwen models are renowned for their extensive multilingual coverage, informal test-

² <https://artificialanalysis.ai/methodology/intelligence-benchmarking>

ing with sample prompts in European Portuguese suggested weaker consistency and nuance compared to Llama. This indicates that, in this context, Llama delivered stronger depth of support, while Qwen's strengths lay in broader language coverage.

Ultimately, the *Llama-3.3-70B-Instruct* model, presented the most balanced profile for this project's needs. It offered state-of-the-art performance on core reasoning benchmarks and robust, well-documented tool-calling functionalities. More importantly, it benefited from the largest and most active community support, which is invaluable for development and troubleshooting. This ecosystem strength, coupled with its consistent performance across languages, made it a safer choice. Given the project's deployment target of a single NVIDIA H200 Graphics Processing Unit (GPU) with 141 GB of VRAM, a 70B parameter model was deemed manageable, especially with quantization techniques.

The Evolving Landscape and Contemporary Models

The field of LLMs is characterized by exceptionally rapid progress. Since the initial model selection in December 2024, several new flagship open-weight models have been released, pushing the boundaries of performance and efficiency. For contextual comparison, an analysis of the open-source landscape as of August 2025 is presented in Table 2, highlighting their scale, context length, intelligence scores and licensing terms. This analysis highlights two major trends, namely the refinement of MoE architectures and the emergence of models specifically tuned for complex chain-of-thought reasoning tasks.

Table 2: Current flagship open-weight models as of August 2025

Model	Parameters (Active)	Context	Intelligence ^a	License
Llama-4-Scout	109B (17B)	10M	33	Llama Community License
Llama-4-Maverick	402B (17B)	1M	42	Llama Community License
Llama-3.3-70B-Instruct	70B	128k	31	Llama Community License
Qwen-3-235B-A22B-2507	235B (22B)	256k	64 / 51	Apache-2.0
Qwen-3-30B-A3B-2507	30.5B (3.3B)	262k	54 / 44	Apache-2.0
DeepSeek-V3.1	685B (37B)	128k	60 / 49	MIT
DeepSeek-R1-0528	685B (37B)	128k	59	MIT
OpenAI-gpt-oss-120B	117B (5.1B)	131k	61	Apache-2.0
GLM-4.5-Standard	355B (32B)	128k	56	MIT/Apache
GLM-4.5-Air	106B (12B)	128k	53	MIT/Apache
Gemma-3-27B-Instruct	27.4B	128k	25	Gemma Terms
Mistral-Small-3.2	24B	128k	32	Apache-2.0

^a The (R / NR) values correspond to model operation with reasoning enabled (R) and disabled (NR), respectively.

A primary distinction in contemporary models is the availability of reasoning-oriented variants. Reasoning variants are typically trained or configured to “think aloud”, producing intermediate steps before committing to a final answer, a process often referred to in the literature as chain-of-thought reasoning (Wei

et al., 2022). This approach often yields more accurate or nuanced outputs for tasks demanding multi-step problem-solving. The trade-off, however, is longer response latency and higher computational overhead, since more tokens are generated internally during inference. For the scope of this project, non-reasoning instruct models are often more suitable. The targeted use cases for the assistant include conversational interaction and IoT device control, which do not typically require complex mathematics, code generation or heavy logical reasoning. In this context the alignment benefits of instruct fine-tuning and the reduced latency of non-reasoning mode tend to outweigh the potential accuracy gains from reasoning-enabled operation.

The prominence of Mixture-of-Experts (MoE) architectures reflects a wider trend toward scaling efficiency. While these designs improve throughput, their high VRAM requirements for storing inactive experts pose a significant constraint on the project’s *NVIDIA H200* GPU, especially as its 141 GB of memory must be shared with embeddings, Speech-to-Text (STT), and Text-to-Speech (TTS) models. Depending on the quantization and offloading strategies applied, models that initially exceed the raw memory limit may still be made to run, although very large releases such as DeepSeek’s 685B parameters remain impractical.

Licensing also emerges as a decisive consideration. While many recent releases adopt permissive terms like Apache 2.0 or MIT, Meta’s *Llama* models remain bound by the more restrictive Llama Community License³. The multimodal *Llama 4* release, for instance, includes explicit licensing restrictions that currently prohibit deployment within the European Union, effectively ruling it out as a candidate regardless of performance.

4.1.3 Final Model Selection

In synthesizing these technical, practical, and temporal factors, the initial selection of *Llama-3.3-70B-Instruct* remains a robust and well-justified decision for this dissertation. To ensure efficient deployment, an 8-bit quantized version of the model⁴ was utilized, balancing high performance with the VRAM constraints of the target hardware.

During integration it became clear that an LLM in a multi-agent framework is not a fully interchangeable component. The orchestration logic, prompt structures, and tool invocation strategies were gradually adapted to the characteristic behaviors of *Llama-3.3*, resulting in a degree of model lock-in. Although standardized APIs such as the OpenAI Completions interface provide a common access layer, replacing the underlying model in practice requires re-tuning and validation to preserve stable system behavior.

Looking forward, models such as *Qwen-3-30B-A3B-2507* and *GPT-OSS-120B* stand out as promising alternatives. Both achieve strong benchmark results while relying on MoE architectures that activate only a subset of their parameters during inference. This design improves throughput and reduces latency, which is particularly valuable for interactive assistants where responsiveness is central to user experience. While adapting the system to these models would require further prompt refinement and interaction

³ <https://www.llama.com/llama4/license/>

⁴ <https://huggingface.co/RedHatAI/Llama-3.3-70B-Instruct-quantized.w8a8>

adjustments, the assistant’s modular design leaves room for future experimentation as the open-source landscape continues to evolve.

In conclusion, *Llama-3.3-70B-Instruct* provided a stable and effective foundation for implementation. Its selection represents a deliberate balance between performance, ecosystem maturity and hardware feasibility, while the observed co-adaptation underscores the practical challenges of evolving complex agentic systems alongside rapidly advancing model families.

4.2 Inference

In the context of this investigation, it is important to outline the text generation process employed by large language models, commonly referred to as **inference**. This mechanism unfolds in a two-stage sequence: an initial **prefill** phase, followed by a **decoding** phase. Figure 4 illustrates this two-stage process, depicting the simultaneous computation of KV vectors for all prompt tokens during prefill, followed by the iterative generation of output tokens during decode.

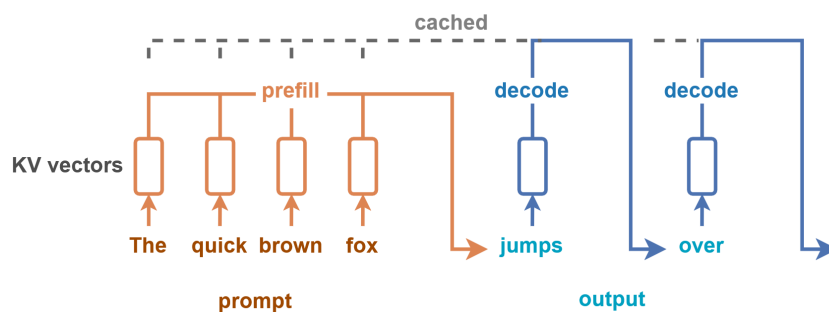


Figure 4: Illustration of the prefill and decode phases, from *Hugging Face* blog⁵

The initial prefill stage involves the parallel processing of tokens from the input prompt. In this phase, the model computes the intermediate states, specifically the key-value (KV) cache, which stores information about the input tokens, in a highly parallelized matrix-matrix operation. This stage is compute-bound, meaning its performance is primarily limited by the computational capacity of the hardware. The primary goal of the prefill stage is to prepare the model for generation by processing all input tokens simultaneously to produce the very first output token.

The decoding phase then generates text autoregressively, producing one token at a time. For each new token, the model computes its key and value vectors and appends them to the existing KV cache, which stores the states of all previously processed tokens to avoid costly recomputation. However, this sequential generation introduces a critical bottleneck. Each token generation requires loading both the model weights and the entire KV cache from high-bandwidth memory into the GPU’s on-chip caches. As the time required for these memory transfers far exceeds the time needed for the actual arithmetic operations, performance becomes constrained by memory bandwidth rather than computational throughput. The decoding phase

⁵ <https://huggingface.co/blog/tngtech/llm-performance-prefill-decode-concurrent-requests>

is thus memory-bound, where inference speed depends primarily on the rate at which data can be moved from GPU memory, not on the speed of the underlying computations.

This generation process concludes either when the model produces a designated end-of-sequence (EOS) token or when a user-stipulated condition, such as a maximum token count, is satisfied.

It is crucial to understand that these tokens, the fundamental units of text processed by LLMs, can represent either complete words or sub-word segments. The precise methodology for segmenting text into these tokens is not uniform but rather varies considerably across different model architectures. This variability has significant implications, for instance, while inference performance is often quantified using token-based metrics (e.g., tokens per second), such figures are not directly comparable across disparate model families due to these underlying tokenization differences.

4.2.1 Inference Components

In the context of deploying applications based on LLMs, model serving is typically organized into two main components: the **server** and the **engine**. At a broad level, the server focuses on routing user requests, whereas the engine is responsible for managing the models and efficiently batching incoming requests. Figure 5 illustrates the interaction between these components, showing how end-user applications send requests through the server, which are then processed by the engine's query queue scheduler and dynamic batching mechanisms before reaching the models.

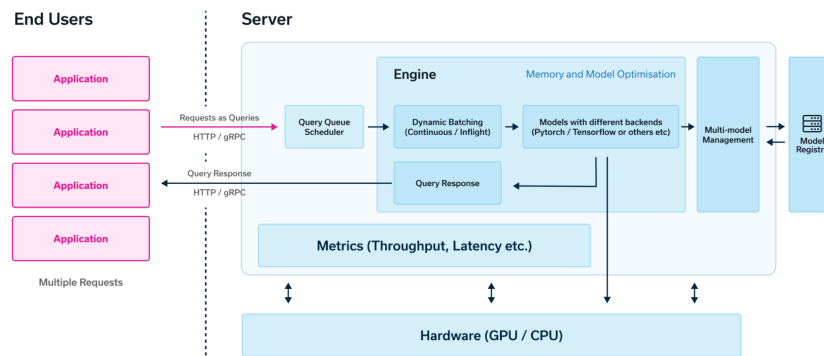


Figure 5: Inference components, from *Run:AI* blog⁶

An LLM inference **server** is a production-grade system designed to wrap the core inference engine. Its responsibilities include managing API endpoints, handling network requests, load balancing across multiple hardware accelerators and providing essential operational tools for monitoring and logging system performance. Inference servers are critical for deploying LLMs at scale, ensuring that they can serve numerous concurrent users while maintaining high availability and responsiveness. An additional consideration is the importance of serving platforms that expose an OpenAI-compatible API. This interface has emerged as a *de facto* standard across the ecosystem, enabling client applications to interact with different

⁶ <https://www.run.ai/blog/serving-large-language-models>

back-end models without requiring major code changes. Given the fast release of new LLMs, maintaining compatibility with this standard affords the flexibility to swap models quickly, ensuring that systems remain adaptable and future-proof.

The LLM inference **engine** is a specialized software system engineered to run LLM computations with maximum efficiency. Its primary objective is to generate model outputs with minimal latency and maximal throughput. To achieve this, it employs a range of computational optimizations, including quantization, operator fusion and sophisticated memory management techniques. Engines are designed to work in close coordination with hardware accelerators like GPUs, managing model loading, memory allocation and the execution of the auto-regressive generation process to serve batched user prompts effectively.

4.2.2 Inference Metrics

To rigorously evaluate the performance of a LLM serving system, four key metrics are typically considered. These metrics provide a comprehensive view of both system efficiency and the end-user experience.

- **Time To First Token (TTFT):** This metric measures the system’s initial responsiveness, defined as the time interval between the submission of a user’s prompt and the generation of the first output token. A low TTFT, which is determined by the latency of the prefill stage, is particularly important for applications requiring immediate feedback, such as interactive conversational agents.
- **Time Per Output Token (TPOT):** This metric reflects the generation speed for subsequent tokens in a response for a single user. It is equivalent to the latency of a single decode step and directly influences the perceived “flow” or “speed” of the model by the end-user.
- **Latency:** This represents the total elapsed time for the model to deliver its complete textual output. This can be derived from the preceding metrics using the formula:

$$\text{Latency} = \text{TTFT} + (\text{TPOT} \times \text{Number of Tokens Generated})$$

- **Throughput:** This captures the overall capacity of the inference server, defined as the total number of output tokens generated per second across all concurrent users and requests.

While the metrics defined above apply broadly to any LLM serving system, their relative importance depends on the application. In the case of this dissertation, where the model powers a conversational virtual assistant capable of multi-turn dialogue and tool invocation, some metrics carry more weight than others.

A low TTFT is critical for perceived responsiveness, ensuring the assistant appears attentive immediately after a request. Likewise, maintaining a fast and steady TPOT supports natural conversational flow and enables effective integration with text-to-speech systems. Latency, beyond its impact on dialogue, plays a central role in more complex agentic tasks involving tool calls or multi-step workflows, where end-to-end task completion time impacts user satisfaction. Finally, throughput remains an important metric for multi-user scalability, but in this context it is always balanced against the higher priority of preserving individual responsiveness.

4.2.3 Inference Optimizations

Making large language models work efficiently depends heavily on speeding up inference. Since LLMs generate text one token at a time, and each token relies on all the previous ones, the process can easily become a performance bottleneck in real-time applications.

To tackle this, a variety of optimization strategies have been developed to enhance efficiency, particularly for deployment in large-scale, multi-user applications. This section outlines several of these key methods.

- **Batching and scheduling:** To maximize GPU utilization and overall throughput, inference engines group multiple user requests into batches. Simple *static batching* processes a fixed batch of requests and waits for all of them to complete before starting the next, leading to poor hardware utilization as GPUs sit idle waiting for the longest decoding sequence to finish. Modern systems instead employ *continuous batching*, where new requests are dynamically added to the batch as soon as others complete. This strategy is often coupled with sophisticated scheduling, such as *chunked prefill*, which breaks down the compute-intensive prefill stage into smaller chunks. This prevents long prompts from monopolizing resources and pausing the decoding of other active requests, leading to better average latency and throughput.
- **KV cache memory management:** The memory required to store the KV cache for all concurrent requests is a major bottleneck in LLM serving. For a model with 13 billion parameters, the cache for a single user with a long context can consume over 1.7 GB of GPU memory (Kwon et al., 2023a). *PagedAttention* addresses this problem by storing the KV cache in non-contiguous memory blocks, in a manner similar to virtual memory in operating systems. This design reduces internal fragmentation and improves memory sharing, which increases the maximum achievable batch size.
- **Optimized compute kernels:** LLM inference is dominated by matrix multiplication and attention computations. The performance of these operations can be significantly accelerated by using specialized compute kernels. Fused kernels combine multiple operations (e.g., matrix multiplication, addition, and activation function) into a single GPU instruction sequence, reducing memory access overhead. Furthermore, optimized attention mechanisms like *FlashAttention* (Dao et al., 2022) reorder the attention computation to be more memory-efficient, delivering substantial speedups without compromising model accuracy.
- **Quantization:** This technique involves reducing the numerical precision of the model's weights, for example from 16-bit floating-point (FP16) to 8-bit integers (INT8) or even 4-bit formats (FP4). Lower precision reduces the model's memory footprint and can accelerate computation on compatible hardware, often with only a minor impact on predictive accuracy (Frantar et al., 2023). Inference engines may support models that are quantized post-training or perform on-the-fly quantization.
- **Speculative decoding:** To overcome the sequential bottleneck of decoding, speculative decoding uses a smaller, faster draft model to generate a sequence of candidate tokens. The larger, primary

model then verifies this sequence in a single, parallelized forward pass. If the draft is accurate, a whole chunk of tokens can be accepted at once, significantly accelerating the decoding process (Leviathan et al., 2023).

In summary, a modern inference engine integrates these optimizations to make LLM deployment feasible, achieving low-latency and cost-efficient inference on available hardware. The application of such techniques is essential for integrating powerful LLMs into real-world products and services, including this assistant based on LLMs presented in this dissertation.

4.2.4 Inference Engines

The optimization techniques discussed in the preceding section are implemented and bundled within specialized serving frameworks, often called inference engines. Deploying large language models at scale, particularly within large organizations, involves complex setups in distributed computing environments. This operational challenge, combined with the computational demands of LLMs, underscores the need for efficient and cost-effective serving solutions. These frameworks are engineered to optimize the serving process, reducing operational costs while delivering the responsive user experience required for real-world applications.

It is worth noting that some tools focus primarily on local accessibility rather than production deployment. Developer-oriented solutions such as *Ollama* or *LM Studio* make it easy to run models on consumer-grade hardware. While highly valuable for experimentation and prototyping, these tools generally lack the scalability and concurrency features required for production workloads.

To better understand the design trade-offs of large-scale serving frameworks, this analysis adopts the evaluation methodology proposed by Park et al. (2025). Each inference engine is assessed along six dimensions: scalability, general-purpose applicability, ease of deployment, ease of use, throughput-awareness, and latency-awareness. **Scalability** captures an engine’s ability to extend across multiple devices or nodes, making it suitable for large-scale or distributed workloads. **General-purpose** support reflects how broadly applicable an engine is across different model architectures and hardware platforms. **Ease-of-deploy** concerns the simplicity of installation and setup, considering whether prebuilt binaries, package managers, or container images are readily available. Complementing this, **ease-of-use** evaluates the developer experience after deployment, including the quality of documentation, the clarity of APIs, and the availability of community support. Performance is further assessed through two optimization criteria, with **throughput-awareness** measuring the ability to maximize token generation rates and **latency-awareness** considering how effectively the system minimizes response times.

With this framework in place, the remainder of this subsection turns to open-source inference engines aimed at large-scale serving, outlining the current landscape and their key characteristics.

vLLM

vLLM is an open-source LLM serving framework that originated from a research project at *UC Berkeley* and has since grown into a widely adopted community-driven project. It introduced an innovative memory management technique called *PagedAttention*, which treats GPU memory like a virtual memory system by storing attention key/value tensors in fixed-size “pages” that can be efficiently allocated and reused (Kwon et al., 2023b). This approach dramatically reduces memory fragmentation and waste. By managing the KV cache more efficiently, the system can keep more sequences in memory simultaneously and thus serve a greater number of users.

Another key feature is continuous batching, in which incoming requests are dynamically batched together on the fly to maximize GPU utilization at each decoding step. Combined with *PagedAttention*, this design delivers state-of-the-art throughput, achieving up to $23\times$ higher throughput than naive static batching approaches⁷.

The framework has been deployed in widely used applications, including *Chatbot Arena* and the *Vicuna* demo (Kwon et al., 2023a), demonstrating its practical scalability. It also supports distributed multi-GPU inference and a broad set of open-source models, such as Llama, Mistral, Qwen and DeepSeek. Designed to maximize throughput and memory efficiency on GPU hardware, *vLLM* is regarded as a breakthrough that significantly advanced the accessibility of large-scale LLM serving in 2023.

The radar chart scores presented in this section are based on the evaluation of Park et al. (2025). While their scoring framework and values are adopted for consistency, these results have been corroborated through additional research, which confirmed that the relative strengths and limitations remain valid in practice.

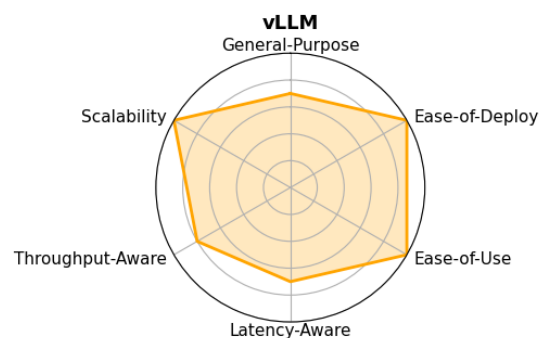


Figure 6: Strengths and limitations of *vLLM* across six evaluation dimensions, adapted from Park et al. (2025)

Figure 6 presents the evaluation of *vLLM*, highlighting strong scalability and throughput due to multi-GPU clustering and advanced scheduling. Ease-of-deploy and ease-of-use are also rated high thanks to pip/Docker packaging and OpenAI-compatible endpoints, though distributed setup requires manual steps. Latency is rated medium, as *FlashAttention* and *PagedAttention* mitigate, but do not fully eliminate, long

⁷ <https://www.anyscale.com/blog/continuous-batching-llm-inference>

input delays.

SGLang

*SGLang*⁸ is an open-source inference engine developed by LMSYS for serving both large language models and vision-language models. It combines a high-performance backend runtime with a flexible frontend programming interface, enabling structured and programmable inference at low latency.

Its key innovation, *RadixAttention*, organizes KV caches in a radix tree structure for automatic and efficient reuse. This allows fast prefix matching across multiple requests, reducing redundant computation and memory usage in scenarios such as multi-turn dialogue and few-shot prompting Zheng et al. (2024). Together with continuous batching, which dynamically groups incoming requests to maximize GPU utilization, *SGLang* achieves high throughput and responsiveness. Beyond these features, the backend supports speculative decoding, paged attention, tensor parallelism and a wide range of quantization methods. It is compatible with a broad set of models and has already been adopted in large-scale deployments. While some tooling is still maturing, *SGLang*'s combination of structured programmability and backend optimizations makes it a versatile and scalable inference engine.

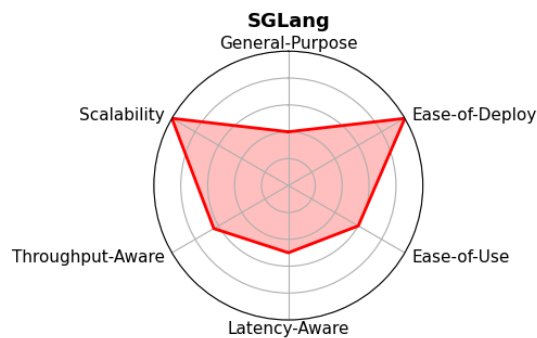


Figure 7: Strengths and limitations of *SGLang* across six evaluation dimensions, adapted from Park et al. (2025)

Figure 7 shows that *SGLang* achieves strong scalability through cache-aware load balancing and cluster execution, while throughput is rated medium despite an advanced batch scheduler. Ease-of-deploy remains high but ease-of-use is more moderate, as the Python DSL provides flexibility but comes with a learning curve. Latency is also medium, with *RadixAttention* helping reduce tail latencies.

Text Generation Inference (TGI)

Hugging Face Text Generation Inference (TGI) is an open-source toolkit for deploying and serving LLMs. Primarily written in Rust to maximize performance, TGI was developed to serve as the official inference backend for Hugging Face-hosted models, with a particular focus on scalability, reliability and efficient deployment.

⁸ <https://github.com/sgl-project/sglang>

TGI introduces several advanced features to optimize inference workloads. It implements its own version of continuous batching at the request scheduling layer, integrates highly optimized transformer kernels such as *FlashAttention*, and supports efficient key-value caching. In addition, it exposes a REST API compatible with OpenAI's API format, offers streaming token output, supports serving multiple models concurrently, and can be deployed across a range of hardware backends.

Initially representing the state-of-the-art for production text generation performance, TGI maintained its leadership position until the release of *vLLM* in mid-2023. Nevertheless, TGI continued advancing through its 3.x versions, significantly improving token throughput and support for long-sequence inference. One benchmark report indicated that TGI achieved up to $3\times$ higher token throughput and was $13\times$ faster than *vLLM* on very long input prompts⁹. However, it is important to note that this comparison was made without customizing *vLLM*'s launch configurations, potentially affecting the validity of the results.

Overall, TGI provides a balanced combination of high throughput, stability, and early access to newly released open models, a benefit stemming from Hugging Face's extensive partnerships within the machine learning ecosystem.

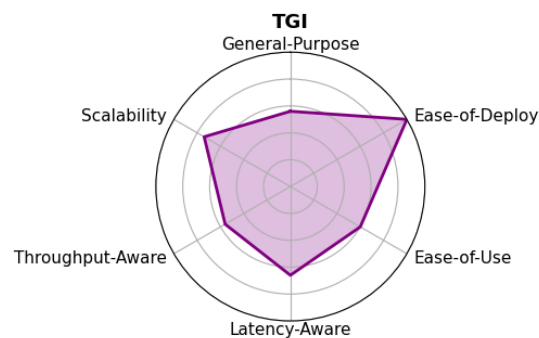


Figure 8: Strengths and limitations of *TGI* across six evaluation dimensions, adapted from Park et al. (2025)

Figure 8 presents the evaluation of Hugging Face *TGI*, which scores high on ease of deployment through its polished API and strong integration with Hugging Face tooling. Ease of use is more moderate because the documentation is brief and largely community-driven. Throughput is also strong due to in-flight batching (a dynamic scheduling approach where new requests are continuously added to the active batch as others complete, rather than waiting for an entire static batch to finish) and optimized kernels, while scalability is robust with support for multi-GPU deployments. Latency is rated medium, reflecting gains from kernel fusion but no specialized attention reuse. General-purpose applicability is moderate, as TGI can operate across *vLLM* and *TensorRT-LLM* back ends to support NVIDIA, AWS Inferentia, and Intel Gaudi hardware.

⁹ <https://huggingface.co/docs/text-generation-inference/en/conceptual/chunking>

TensorRT-LLM

TensorRT-LLM is an open-source inference framework developed by NVIDIA as part of its broader *NeMo* ecosystem for end-to-end generative AI development. It is specifically designed to optimize the deployment of large language models on NVIDIA GPUs, combining high-performance execution with a flexible Python API to support both research and production environments.

The framework integrates tightly with NVIDIA's *TensorRT* compiler, which analyzes computation graphs, selects optimized CUDA kernels, and fuses operations to minimize overhead. This enables efficient use of tensor cores and support for low-precision formats (INT8, FP8), leading to reduced latency and improved throughput.

Beyond single-GPU acceleration, *TensorRT-LLM* supports multi-GPU and multi-node deployments through integration with NVIDIA Triton Inference Server and the NCCL. Features such as in-flight batching and pipeline parallelism further improve utilization under dynamic workloads.

While these capabilities deliver substantial performance gains on NVIDIA hardware, the framework's reliance on proprietary infrastructure limits its portability. *TensorRT-LLM* is therefore best suited to organizations already operating within the NVIDIA ecosystem, where it provides a powerful end-to-end solution for large-scale LLM deployment.

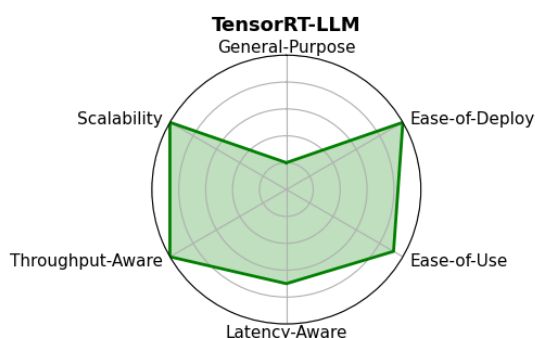


Figure 9: Strengths and limitations of *TensorRT-LLM* across six evaluation dimensions, adapted from Park et al. (2025)

Figure 9 illustrates *TensorRT-LLM*'s evaluation across the six aspects. It excels in scalability thanks to NCCL-based model sharding and multi-node support, and achieves solid throughput with in-flight batching and optimized scheduling. Ease of deployment is high, supported by auto-configuration tools, and ease of use is also relatively high thanks to well-written documentation and active community resources that simplify onboarding and troubleshooting. Latency is rated medium, with some techniques such as Flash-Infer improving generation speed. General-purpose applicability is limited, however, since the framework is tightly coupled to NVIDIA hardware.

4.2.5 Inference Evaluation Summary

The per-engine analysis confirms that there is no universal “best” inference engine, as performance varies depending on workload characteristics, model choice and deployment constraints. From the radar charts evaluation across multiple criteria, however, *vLLM* and *SGLang* stand out as the most capable general-purpose options. Independent evaluations and practitioner reports likewise show them to be closely matched out of the box, with head-to-head results often depending on the specific workload and configuration.

Beyond raw technical benchmarks, long-term viability, community support and development velocity are equally decisive for adoption, particularly in research and development contexts. Figure 10 complements the technical analysis by highlighting these community-oriented dimensions, offering an additional perspective on framework selection.

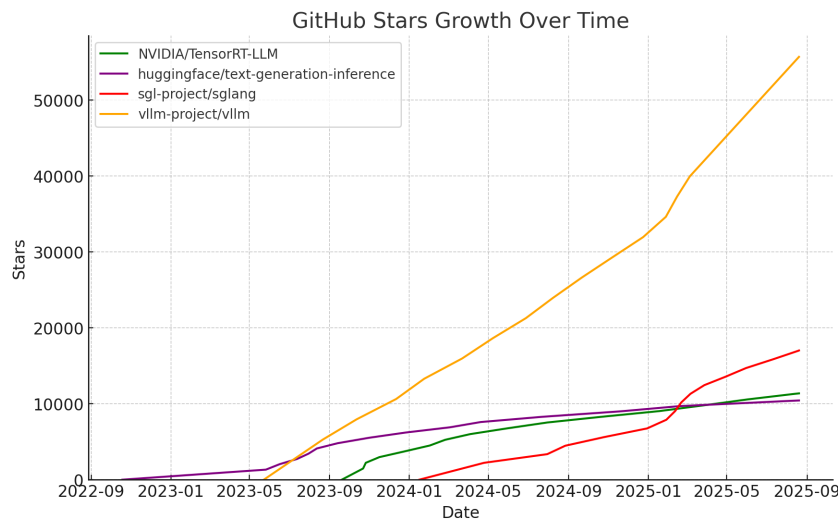


Figure 10: GitHub stars for selected LLM inference engines as of August 2025

Looking at the star trends together with development activity, *vLLM* and *SGLang* combine frequent updates with steady growth in adoption, while *TGI* and *TensorRT-LLM*, despite having accumulated many stars early on, show slower commit activity and narrower model coverage, suggesting reduced support for ongoing improvements and future feature development.

While technical benchmarks place *vLLM* and *SGLang* in close competition, the metrics related to community adoption and project momentum provide a clear differentiator in this evaluation. As shown in the Figure 10, *vLLM* has a significant lead in community engagement. This widespread adoption often translates into more rapid support for new open-source model releases, a critical advantage in a fast-moving field, as well as a larger collective knowledge base for troubleshooting and development. Many research labs and open-source contributors have standardized on *vLLM* as their primary inference backend, leading to day-one support for new models such as Mistral, Gemma, Qwen and, most recently, the open-source ChatGPT family of models.

Therefore, considering ecosystem maturity and immediate access to state-of-the-art open-source models, *vLLM* was selected as the foundational serving framework. Its proven ability to efficiently manage memory and schedule requests under complex, multi-user scenarios makes it a robust and future-oriented choice for the development and evaluation of the virtual assistant presented in this work.

4.3 Architecture and Agents Design

Building intelligent systems involves more than just deploying powerful models. It requires thoughtful design around how different components reason, act and collaborate. As AI capabilities evolve, there's increasing value in combining autonomous decision-making with structured coordination. This is especially true in systems that must handle diverse user intents, leverage multiple specialized capabilities, and deliver cohesive responses in real time.

This section introduces the architectural foundations of the system, focusing on how it balances autonomy and orchestration, enables modular intelligence through domain-specific agents and manages interaction flow to produce reliable and context-aware outcomes. By outlining the roles of individual agents, the coordination mechanisms that link them, and the overall control logic, the section shows how the system integrates both flexibility and structure in its execution.

4.3.1 Foundations of Agent Design

The terminology surrounding agents in artificial intelligence, particularly in the emerging landscape of LLM-based systems, is often applied inconsistently, leading to conceptual ambiguity. To avoid this, the following subsection establishes clear definitions of the foundational terms used in this work.

AI Agents

An AI Agent may be broadly defined as an autonomous software entity designed to perceive its environment (or input data), make context-sensitive decisions and take actions to achieve a specific goal with minimal ongoing human direction. With the advent of large language models, the concept of “agent” has broadened, becoming more flexible and accommodating a wider spectrum of autonomy. In this contemporary view, an AI agent can be understood as any system component that leverages an LLM to guide the flow of an application by selecting outputs, steps, or actions in response to input. Depending on its degree of agency, this may range from a single LLM call producing an output within a fixed workflow, to more complex systems that dynamically choose actions, tools, or goals with minimal human intervention.

But even this definition may not achieve universal consensus, since many stakeholders implicitly associate the notion of agents with advanced, human-like autonomy and general intelligence. This lack of agreement underscores the relevance of perspectives such as that articulated by Andrew Ng, who has argued that it is more productive to focus on the degree of agentic capability a system exhibits rather than

debating whether it does or does not qualify as a “true” AI agent. This framing shifts the discussion from rigid definitions to a more nuanced consideration of how agentic a system is, providing a more flexible and practical lens for evaluating contemporary architectures.

The framework illustrated in Figure 11 situates LLM-based applications along a spectrum of autonomy, ranging from systems that are almost entirely human-driven to those capable of self-directed execution. At its core, the degree of autonomy corresponds to how extensively an LLM determines the systems behavior.

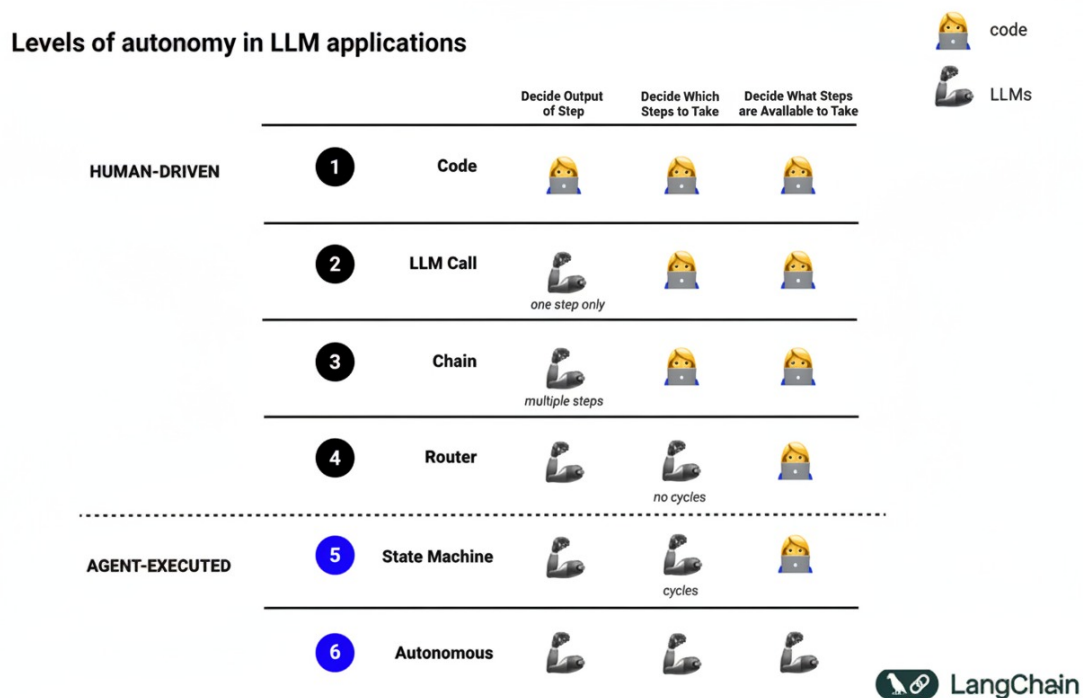


Figure 11: Levels of autonomy, adapted from *Langchain Blog*¹⁰

This spectrum is defined by how responsibility for three key decisions is distributed between the human developer and the LLM: determining the output of a given step, deciding which steps to take, and defining what steps are available for execution.

At the lower, human-driven end of the spectrum, control is largely retained by the developer. In the *LLM Call* and *Chain* categories, the LLM is primarily tasked with generating the output for one or more steps within a sequence that is hardcoded by the developer. The defining characteristic here is that the execution path is fixed. Moving up the autonomy ladder, the *Router* architecture shows increased autonomy by tasking an LLM with selecting the next path from a set of predefined workflows. However, it avoids cycles, which keeps the execution path finite and predictable. If any steps include logic to decide whether the system should continue or stop, creating something like a loop, then the system is classified as a *State Machine*.

A foundational approach to implementing such looped, decision-making processes in agents is the *ReAct* (Reason + Act) paradigm, introduced by Yao et al. (2022). *ReAct* agents combine reasoning capa-

¹⁰ <https://blog.langchain.com/what-is-an-agent/>

bilities of LLMs with action execution, enabling them to iteratively think through problems, use tools, and act based on observations to achieve the user goals. This cycle enhances the agent's ability to handle multi-step tasks adaptively, and it has become a widely adopted framework in LLM-based systems.

Further along the spectrum are highly *Autonomous* agents that build on these features and go beyond them, such as by generating new tools dynamically. These designs occupy the upper end of the autonomy spectrum, offering extreme independence but introducing substantial challenges in areas like predictability, safety, and control. In essence, greater autonomy often leads to increased unpredictability, making systems harder to debug and manage in production environments.

While individual agents can handle sophisticated tasks, scaling to complex, multifaceted problems often requires more than increasing autonomy in a single entity. It demands distributing responsibilities across multiple specialized agents that collaborate under orchestrated workflows. The next part of this section examines these Multi-Agent Systems and their architectural patterns.

Multi-Agent Systems

According to Wooldridge (2009), Multi-Agent Systems (MAS) are systems composed of multiple interacting computational elements, known as agents, which are capable of acting autonomously up to a certain limit and can interact with other agents in order to meet their design objectives.

Single-agent systems encounter several notable difficulties when scaling to more complex tasks. One significant challenge arises from **tool overload**, where an agent managing an excessive number of tools struggles to select the optimal tool for a given task (Shu et al., 2024). Another issue relates to **context complexity**. As context from diverse tools and user interactions accumulates, the growing information load can overwhelm the LLM, leading to diminished performance. Finally, complex problems often need diverse areas of expertise. Attempting to consolidate these varied specializations within a single, monolithic agent prompt forces the creation of complex conditional logic and results in prompts instructions that are difficult to scale. This approach proves less effective than distributing each logical segment of the task to a specialized agent within a multi-agent framework (Dorri et al., 2018).

Adopting a multi-agent architecture offers several advantages over singular agent approaches. The inherent modularity of a MAS architecture facilitates a more organized development lifecycle, as the system can be decomposed into manageable sub-components that are easier to build, debug, and maintain. In addition, specialization enables the system to achieve higher efficacy by assigning domain-specific agents to targeted tasks, thereby improving both the accuracy and efficiency of outcomes (Sun et al., 2025).

In the design of MAS, several architectural patterns have been established to define how agents interact and manage control flow. These patterns range from decentralized networks to structured hierarchies, each offering different trade-offs between flexibility, control and complexity (He et al., 2025).

One of the most decentralized architectural patterns is the **Network of Agents**, illustrated in Figure 12, where multiple agents interact dynamically with every other agent without a central coordinator. While offering flexibility, this architecture, sometimes associated with frameworks like *Swarm* or *Crew*

AI, has been observed to potentially lack sufficient control, sometimes resulting in unreliable behavior, high latency and increased operational costs due to frequent LLM interactions. Consequently, its use in production environments is often approached with caution.

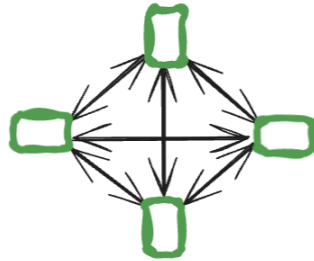


Figure 12: Network of agents architecture, adapted from *LangGraph Conceptual Docs*¹¹

A more structured approach is the **Supervisor Agent** architecture, illustrated in Figure 13. In this pattern communication flows through a central supervisor agent that coordinates multiple specialized worker agents. When a task is presented, the supervisor evaluates the request and determines which worker agent is best suited to handle it. The supervisor then delegates the task to the appropriate worker agent, which performs the task and sends the result back to the supervisor. If needed, the supervisor may direct the output to another worker agent for further processing. This central coordination ensures that the sub-agents can then concentrate on task execution without the burden of managing the overall workflow.

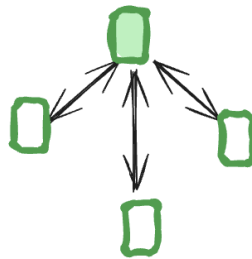


Figure 13: Supervisor agent architecture¹¹

As a natural extension of the supervisor model, the **Hierarchical Agent** architecture is designed to manage tasks of even greater complexity. This pattern introduces multiple layers of delegation, analogous to an organizational chart, where a top-level supervisor can delegate a high-level objective to a sub-supervisor. This sub-supervisor, in turn, orchestrates its own team of specialized agents to execute the sub-tasks required to achieve its assigned goal. This multi-tiered structure enables a “divide and conquer” strategy, allowing for the systematic decomposition of a large, complex problem into a series of smaller, more manageable components, each handled by a dedicated agent team. Figure 14 provides a visual representation of this hierarchical structure.

¹¹ https://langchain-ai.github.io/langgraph/concepts/multi_agent/

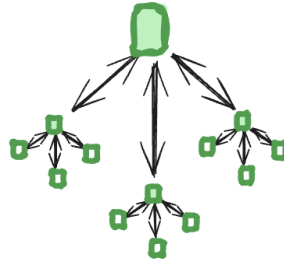


Figure 14: Hierarchical agent architecture¹¹

While these established patterns provide valuable theoretical frameworks, production-grade multi-agent systems rarely adhere strictly to a single blueprint and the most common approach involves creating a **Custom Architecture** (Figure 15).

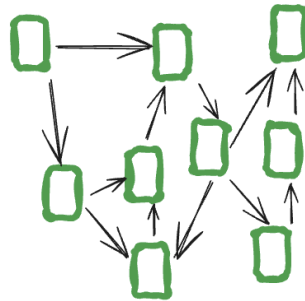


Figure 15: Custom agent architecture¹¹

Such an architecture might borrow concepts from supervisor or hierarchical models, but its core structure is specifically engineered around the unique demands and context of the particular application. This very principle of tailored design is typically viewed as crucial for obtaining the operational control and performance levels needed for success in real-world settings.

4.3.2 Proposed Multi-Agent Architecture

Building upon the foundational concepts of agent autonomy and multi-agent architectural patterns discussed previously, this subsection delineates the specific architecture adopted for the proposed system.

Functional Domain Decomposition

The initial step in designing the architecture was to analyze and group the requirements into high-level functional domains. This functional decomposition allowed for a clear separation of concerns, which was critical for building a maintainable and scalable system. The primary domains identified were:

1. **Device Interaction:** Covering all functions related to smart home hardware, including retrieving a list of devices, checking their status, and issuing control commands.

2. **Reminder Management:** Focusing on the creation and management of user reminders.
3. **Weather Information Retrieval:** Covering tasks requiring external data, specifically providing weather updates.
4. **Long-Term Memory:** Capturing and retrieving relevant user facts and preferences for personalization.
5. **General Conversation:** Providing fallback support for queries and interactions outside of the above domains, ensuring the assistant can respond to non-task-specific requests.

Based on these domains, a modular, agent-based approach was adopted as the implementation strategy. Each domain was encapsulated within a dedicated AI agent, turning high-level requirements into functional components. This mapping ensured each agent had a focused purpose and a minimal set of relevant tools, enhancing both performance and reliability. The resulting agents were:

1. **Devices Agent:** This agent manages all smart-device interactions, and within the previously defined autonomy framework in Section 4.3.1 (AI Agents), it is classified as a *State Machine*. Concretely, the agent operates through a decision cycle in which the LLM selects an action, observes the outcome, and decides whether to continue calling tools or stop. Within this structure, the agent employs a dedicated toolset for discovering available devices, inspecting their properties and updating their state, where the tools leverage the functionalities provided by the Devices API. It is equipped with a toolset comprising *get_user_devices*, *check_specific_device_settings*, and *update_device_settings* (detailed in Appendix C.1.1), which collectively enable the discovery, inspection, and manipulation of device states via the system's Devices API.
2. **Reminder Agent:** Responsible for parsing and managing time-sensitive user requests, this agent is classified as a *State Machine* within the autonomy framework. In practice, it grounds the LLM with the user's current list of reminders, automatically injected into the context before inference, so that duplicate entries are avoided and queries about existing reminders can be answered directly. By reducing redundant conversational turns and constraining tool selection, the agent provides a more efficient and reliable user experience. Its functional capabilities are exposed through three primary tools: *create_reminder*, *get_reminders*, and *delete_reminder*, with full specifications available in Appendix C.1.2.
3. **Weather Agent:** This agent provides access to weather forecasts, both daily and hourly, and is classified as a *State Machine* within the autonomy framework. It also follows a decision loop, where the LLM reasons over the user request, selects an appropriate tool, observes the result, and continues as needed. To ensure temporal accuracy, its system prompt is dynamically augmented with real-time contextual information such as the current date, time, and day of the week at inference time. This grounding enables the agent to correctly interpret relative time-based queries like "tomorrow" or "in three hours." It utilizes two tools, *get_daily_forecast* and *get_hourly_forecast*, whose technical details are provided in Appendix C.1.3.

4. **Memory Agent:** This agent enables long-term personalization by extracting, normalizing, and storing user facts in a dedicated memory store. Within the autonomy framework, it is classified as a *State Machine* and operates in the same tool-use cycle as the other agents, though its role is specialized in handling factual knowledge about the user. The Memory Agent listens for user inputs that may contain personal facts. When it identifies such information, it reformulates the statement into a consistent format and stores it in the memory base. Before persisting, it checks for duplicates or malformed entries to keep the store concise and consistent, since the current set of user memories is injected into the agent’s context window through the system prompt. The agent operates in a stateless manner, meaning it does not carry over previous dialogue turns as context and only analyzes the current user prompt. This design keeps it focused on capturing newly introduced information, avoiding irrelevant context and improving the precision of fact extraction. All memory operations are handled by a single, versatile tool, *manage_memory*, as specified in Appendix C.1.4.
5. **Conversational Agent:** This agent serves as the default handler and conversational fallback for any query that does not fall into one of the previously defined functional domains. Within the autonomy levels taxonomy, it is most closely associated with a *LLM Call*. It operates without direct access to external action tools but performs retrieval over the vector database of user memories, selecting entries most relevant to the input query. These retrieved memories are injected into its context before inference, enabling more personalized and contextually aware responses. By combining retrieval with generation, the Conversational Agent follows a Retrieval-Augmented Generation (RAG) approach. Although this component primarily provides conversational responses, its integration of RAG and long-term memory justifies treating it as an agent within the multi-agent system, even if it exhibits lower autonomy than the tool-oriented agents.

The tool-using agents in this system follow an iterative decision-action-observation loop inspired by the *ReAct* paradigm, but with one important difference: they decide on their next action directly, without verbalizing their intermediate reasoning in natural language. This choice reduces computational overhead by avoiding the additional token generation required for explicit reasoning, while retaining the benefits of structured decision-making, resulting in greater efficiency for real-time interaction.

Each agent maintains its own context and chat history, enabling it to stay focused on its specific tasks without being distracted by unrelated information from other agents. This separation reduces context complexity by ensuring agents work only with the information relevant to their role, which in turn improves their performance and reliability.

Supporting Infrastructure

The virtual assistant architecture relies on two key auxiliary infrastructure services that provide specialized functionality to the system’s agents. These services were pre-existing components in the development

environment and form the foundational layer that enables the agents to interact with external systems and manage persistent operations.

The **Devices API** serves as the unified REST interface for smart home device management. Built on the *Eclipse Ditto* framework, it provides a standardized abstraction layer over the underlying *MQTT* messaging protocols. This service is responsible for handling all device state management, command execution, and status reporting through consistent HTTP endpoints, allowing the Devices Agent to remain agnostic to the underlying protocols or hardware manufacturers while still controlling and monitoring the devices. The agent's interaction with the API is implemented through its tools, where each tool encapsulates a specific HTTP request.

The **Background Jobs Service** is implemented in *C#* and uses the *Hangfire* library for background task processing. The service exposes an API that uses cron-based scheduling expressions for creating, retrieving, and deleting jobs, and is designed to handle task persistence across different timezone configurations, including adjustments for daylight saving time. The Reminder Agent uses this API to offload the responsibility of managing the lifecycle of user reminders. This design decouples the immediate, conversational act of setting a reminder from the long-term, asynchronous act of its delivery.

For notification delivery, the service utilizes an integrated, self-hosted **ntfy**¹² **server** which is an open-source, publish-subscribe notification service that acts as a lightweight, real-time message broker. The Background Jobs Service (the publisher) sends notifications to dynamically generated, user-specific topics via standard HTTP POST requests. The client application (the subscriber), in turn, establishes a persistent and authenticated WebSocket connection to its designated topic on the *ntfy* server. This architecture allows the *ntfy* server to instantly push JSON-formatted notifications to the client as soon as they are published, enabling reliable, real-time event delivery.

The Orchestration Layer

With responsibilities distributed across multiple agents, a central coordination layer was needed to analyze each incoming user request and delegate it to the appropriate specialist.

To fill the coordination role, and drawing direct inspiration from the Supervisor Agent architecture, a component was designed to function as a central Orchestrator. This component acts as the system's primary triage node, responsible for analyzing all incoming user requests and managing the end-to-end execution flow. This process is exemplified in Figure 16, where a command such as “*Turn on the lights*” is analyzed, identified as a device control task, and subsequently delegated to the Devices Agent for execution.

¹² <https://github.com/binwiederhier/ntfy>

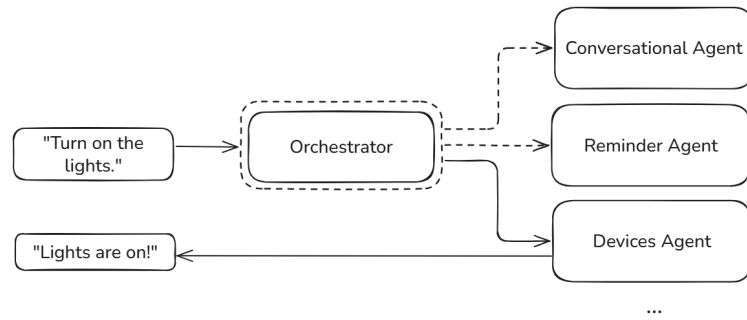


Figure 16: Orchestrator task delegation workflow

To enable this routing capability, the Orchestrator leverages a LLM to perform intent classification upon receiving a user prompt. The model is specifically prompted to analyze the query and return a structured JSON object that decomposes the user’s prompt, handling multi-intent requests by separating them into independent sub-tasks, each assigned to a relevant functional domain.

For example, a user prompt such as *“What’s the weather forecast for Lisbon tomorrow, and also remind me to wake up early.”* would be processed to produce the JSON structure shown in Listing 4.1.

```

{
  "Device": "",
  "Weather": "What's the weather forecast for Lisbon tomorrow",
  "Reminder": "remind me to wake up early",
  "Memory": "",
  "Conversational": ""
}
  
```

Listing 4.1: Example of JSON structure for multi-intent classification

This structured output is then parsed by the Orchestrator’s procedural logic, which identifies that both the Weather and Reminder domains are present in the query and subsequently manages the execution of their respective agents with the associated text fragments.

Unlike a classic supervisor pattern that requires evaluating agent results, the Orchestrator in this system only dispatches tasks. This “dispatch-only” design intentionally omits the result-processing step, which avoids a costly additional LLM call in terms of latency for post-execution analysis. Consequently, the architecture supports the parallel execution of sub-tasks, improving system responsiveness.

Beyond classification, this initial stage also performs a critical prompt enhancement through contextual resolution. Leveraging the complete conversational history, which the individual agents do not have access, the Orchestrator disambiguates pronouns and other contextual references before forwarding the request. This is crucial for maintaining conversational continuity, as specialized agents operating without a universal context history would otherwise be unable to correctly interpret follow-up prompts.

For instance, consider the following two-turn exchange:

User: *“Remind me that I have a meeting in Porto on Friday.”*

User: *“Check the weather forecast for there on that day.”*

Without intervention, the Weather Agent would receive the second, ambiguous query and fail to understand it. Instead, the Orchestrator intercepts this query and resolves it into the complete, self-contained instruction:

“Check the weather forecast for Porto on Friday.”

This routing mechanism ensures that each specialized agent receives an unambiguous and fully actionable task, establishing a clear and predefined workflow.

Full Architecture Overview

To consolidate the components and design choices described so far, this section provides a high-level overview of the multi-agent architecture. The goal is to illustrate how the different types of agents, ranging from simple to more autonomous, are combined to form the text reasoning component of the assistant.

Figure 17 presents the class diagram of the multi-agent system, highlighting how agents span different levels of agency and specialization.

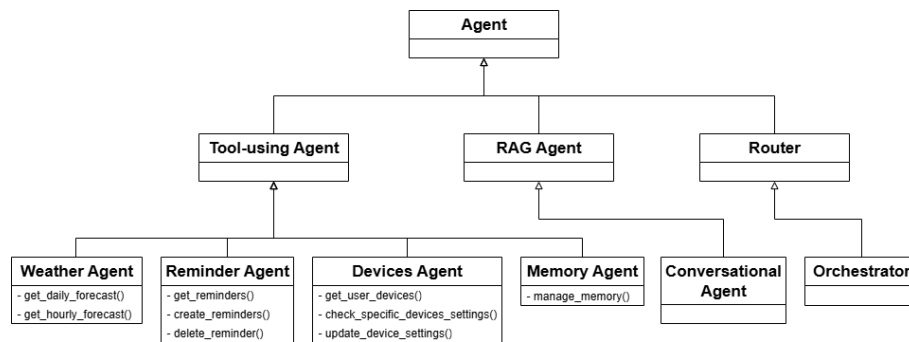


Figure 17: Multi-agent system class diagram

As shown in Figure 17, the architecture is organized into three main architectural patterns, each representing a distinct approach to agent design. The tool-using agent pattern enables domain-specific agents like Weather, Reminder, Devices, and Memory to follow iterative decision-action-observation loops, selecting and executing tools based on user requests. In parallel, the RAG agent pattern supports retrieval-augmented generation, as exemplified by the Conversational Agent which retrieves relevant user memories to provide personalized responses without direct tool execution. Finally, the Router pattern provides coordination mechanisms through the Orchestrator, which serves as the central delegation hub by analyzing requests and routing them to appropriate specialized agents. This design ensures that each functional domain operates with the appropriate level of agency and specialization, from tool-driven task execution to memory-enhanced conversation.

The architecture of the assistant is organized as an orchestrated multi-agent system, which combines a structured, high-level workflow for coordination and task distribution with specialized, autonomous AI agents responsible for executing domain-specific sub-tasks.

Figure 18 illustrates this architecture in action, showing how the system processes a complex, multi-intent query from initial user input to the final synthesized response, thereby exemplifying the principles introduced in the previous sections.

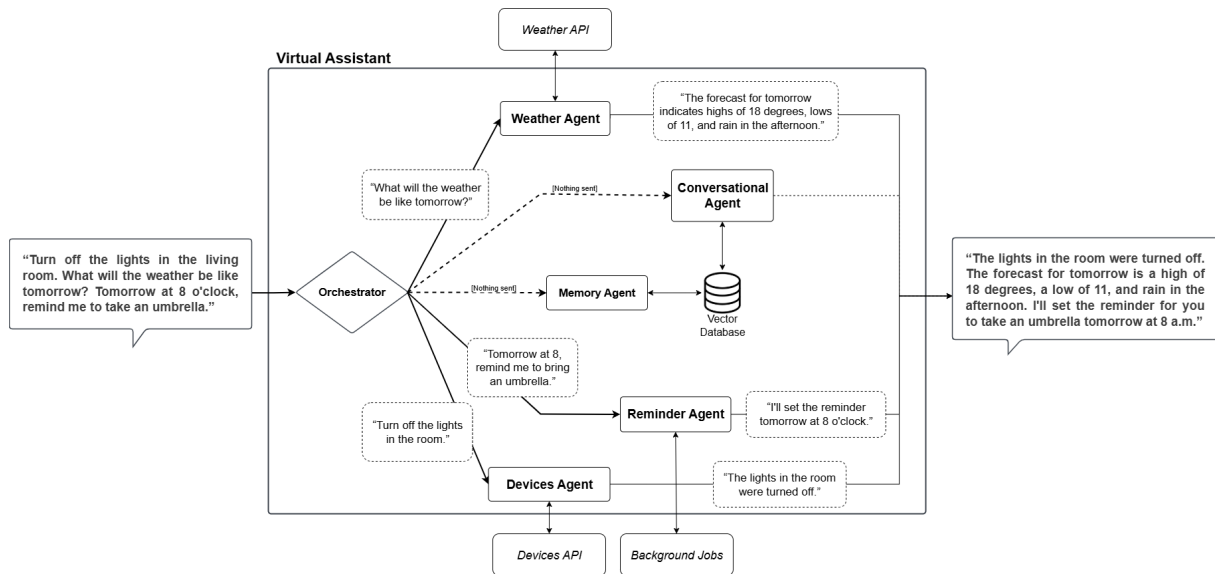


Figure 18: System execution flow for an example request

The flow illustrated in Figure 18 can be traced through the following stages:

1. **User Input:** The process begins with a user issuing a single, multi-intent prompt: *"Turn off the lights in the living room. What will the weather be like tomorrow? Tomorrow at 8, remind me to take an umbrella."*
2. **Orchestration and Decomposition:** This prompt is received by the **Orchestrator**, which analyzes the utterance and decomposes it into three distinct and independent sub-tasks: a device control command, a weather query, and a reminder creation request.
3. **Task Delegation:** The Orchestrator assigns each sub-task to the corresponding agent: the device-related task goes to the **Devices Agent**, the weather task to the **Weather Agent**, and the reminder task to the **Reminder Agent**.
4. **Autonomous Execution:** Each specialized agent operates independently to fulfill its assigned task. The Devices Agent interacts with the Devices API to turn off the lights, the Weather Agent queries the Weather API for the forecast, and the Reminder Agent schedules the requested notification via the Background Jobs service. During this stage, each tool-using agent leverages its internal decision-making loop and specialized tools to execute its instructions.
5. **Response Synthesis:** The final stage involves aggregating the results from all successfully executed tasks. The system synthesizes these individual outcomes into a single, coherent, and comprehensive response that addresses all parts of the user's initial query: *"The lights in the room were turned off. The forecast for tomorrow is a high of 18 degrees, a low of 11, and rain in the afternoon. I'll set the reminder for you to take an umbrella tomorrow at 8 a.m."*

This architecture exemplifies a hybrid approach that leverages the reliability and predictability of a structured workflow for overall control, and the intelligence and adaptability of autonomous agents for handling domain-specific complexities. A key architectural advantage of this orchestrator-based design is its approach to scalability and latency management. New capabilities can be added to the system by introducing new specialized agents as parallel “branches” without increasing the vertical depth of the processing pipeline. This design avoids the latency accumulation that would occur in a more layered architecture and ensures that the number of sequential LLM calls at the orchestration level remains bounded. However, this approach acknowledges a trade-off: while latency remains constant as new domains are added, the classification accuracy of the orchestrator may degrade if the number of possible domains becomes excessively large, increasing the potential for ambiguity. For the scope of this project, this trade-off was deemed acceptable, prioritizing a responsive and extensible system.

4.3.3 Safety and Security Considerations

Ensuring safe and appropriate outputs is a critical concern for any conversational assistant deployed in user-facing environments, requiring dedicated mechanisms to prevent the generation of harmful or inappropriate responses.

Modern LLMs, including *Llama-3.3-70B-Instruct*, undergo alignment training that incorporates safety measures to avoid generating harmful content. This training provides a foundational level of protection that is standard across contemporary language models. However, production deployments typically complement this with application-level filtering mechanisms.

In this system, a streaming output filter monitors the assistant’s generated responses in real-time. As tokens are produced during the decode phase, the output is checked against a curated list of prohibited terms and patterns. When a match is detected, generation is immediately terminated and a safe fallback response is returned to the user.

Alternative approaches, such as employing additional language models to analyze output for inappropriate content, offer more sophisticated and context-aware filtering. However, the added latency introduced by secondary model inference was deemed incompatible with the real-time interaction requirements, making the pattern-matching approach more suitable for this use case due to its lower complexity and robustness.

4.3.4 Agentic Frameworks

The development of agentic systems has been accelerated by a number of specialized frameworks, including *LangChain*, *Autogen*, and *CrewAI*. These tools offer pre-built components and abstractions designed to simplify the development lifecycle of agentic applications, particularly for tasks like LLM integration, tool definition, and state management.

However, during the initial research phase, an evaluation of these frameworks revealed several con-

cerns that influenced the project's technical direction. First, the high-level abstractions, while useful for rapid prototyping, tend to obscure the underlying prompt construction and LLM interaction logic. This opacity complicates debugging and makes it difficult to understand system behavior when issues arise. Second, these frameworks exhibited significant instability during the evaluation period, with frequent updates introducing breaking changes that required code refactoring even when existing functionality was working correctly.

Based on this assessment, the project adopted an implementation approach using foundational Python libraries rather than a high-level agentic framework. This decision prioritized transparency and control over development velocity, allowing for explicit authority over prompts, tool definitions, and orchestration logic. While this approach required developing custom tool integration modules and managing inter-component communication manually, the resulting system behavior became more predictable and debuggable, while reducing dependency on rapidly evolving external frameworks.

It should be noted that the agentic framework ecosystem continues to evolve very fast, and new or already existing frameworks may have addressed the stability and over-abstraction issues identified during the initial evaluation. However, the decision was made based on the maturity level of available options at the project's inception.

4.4 Memory Management

AI agents, similar to humans, utilize memory systems to process, store, and recall information, which is crucial for decision-making, learning, and adaptation (Zhang et al., 2024). However, a simple LLM does not inherently “remember” previous conversations or responses. Every time the model is called, it generates a response based solely on the information provided in the messages input parameter, and prior exchanges must be re-supplied to the context window to simulate continuity.

While modern LLMs can process long contexts, they often struggle to use them effectively. Performance degrades as prompts grow, with models focusing on the beginning or end and overlooking information in the middle (Liu et al., 2023).

Supporting an assistant that must recall past conversations and user preferences over time requires dedicated memory management strategies.

4.4.1 Memory Types

In the context of LLMs based agents, memory can be broadly categorized into **Short-Term Memory (STM)** and **Long-Term Memory (LTM)**, each serving distinct roles.

Short-Term Memory: STM holds information only for the duration of the ongoing interaction, typically seconds to minutes. It serves as the agent's immediate workspace, ensuring coherence and responsiveness within a single conversation. Its capacity is bound by the model's context window, meaning older inputs are ignored.

Long-Term Memory: In contrast, LTM is designed for persistent storage across sessions, retaining knowledge for the entire lifespan of the agent. It has far greater capacity than STM, supporting the accumulation of facts, preferences, and past interactions. In practice, LTM relies on external modules such as databases, vector stores, or knowledge graphs for scalable retrieval. This persistence enables richer personalization and continuity, making it essential in domains that require long-term interaction and adaptation.

The interplay between STM and LTM is crucial for an effective AI agent and both must be managed carefully to ensure optimal performance.

4.4.2 Short-Term Memory Management

Managing short-term memory for LLM applications requires balancing precision and recall against latency and cost.

One of the most commonly used strategies is **filtering messages** from the conversation history. The simplest form of this is keeping the last N messages, where only the most recent turns are retained. The main drawback of filtering is its lack of contextual awareness. By applying fixed rules, important information such as prior intent, reasoning steps, or tool interactions may be removed, which can lead to incoherent model responses due to the bad precedents set by the trimmed history. For tool-augmented agents, this can be especially problematic. Removing parts of a tool invocation or result can disrupt the model's understanding of the interaction sequence, causing it to hallucinate missing steps or omit necessary tool calls altogether.

To prevent this, a more nuanced strategy is applied in these agents, **filtering by logical groups**. A group consists of a set of related messages (e.g., a user request, the subsequent tool calls, and the corresponding tool responses). Trimming entire groups ensures that the contextual integrity of complete interaction cycles, especially those involving tools, is preserved, even when removing older history. While more complex to implement, this method yields significantly better results in maintaining conversational coherence for agents reliant on tool usage.

4.4.3 Long-Term Memory Management

The capacity for agents to learn from past interactions, adapt to user preferences, and maintain consistent, intelligent behavior fundamentally relies on the creation of **memories**. Before an agent can leverage historical knowledge or past experiences, it must first effectively capture and store relevant information from its interactions. This process of forming memories is the foundational step that fills long-term memory with the essential information needed for future retrieval.

However, the information an AI agent needs to retain is diverse, ranging from specific past events to general world knowledge or learned skills. To manage this variety, AI agents can employ different mechanisms to create distinct types of long-term memories suited to storing specific kinds of knowledge.

Consequently, designing an effective memory system requires careful consideration of interrelated factors.

Types of Memory

Episodic memory allows AI agents to recall specific past experiences, similar to human memory of individual events. This type of memory is valuable for case-based reasoning, where agents can use past cases for reasoning or preserve successful interactions as learning examples to guide future behavior. Implementation can range from logging key events and outcomes in a structured way to storing detailed “episodes” that encapsulate successful problem-solving narratives (Sumers et al., 2024).

Semantic memory refers to memory that holds structured factual knowledge such as definitions, rules, or user-specific details, which an agent can query to support reasoning and provide contextually appropriate responses. This includes details like user preferences (“*User prefers concise, bulleted summaries.*”), important user-specific dates or facts (“*User’s dog is named Max.*”), or learned constraints (“*Avoid suggesting seafood, user mentioned an allergy.*”).

In this use-case, the focus is on capturing **semantic memory**, specifically, facts and preferences shared by the user during conversation. This includes concrete details, such as names, interests, and restrictions, that help the assistant personalize future responses and maintain contextual continuity over time.

Memory Structure

The organization of stored information within memory systems can adopt diverse structural formats, tailored to the nature of the information being retained. Memories can be represented using predefined schemas that dictate their composition and relationships. These schemas provide flexibility while maintaining consistency in data representation. Two primary structural approaches have been identified:

- **Profiles:** These schemas consolidate information into a single, often entity-centric record (e.g., a user profile). They typically contain predefined fields for specific attributes like characteristics or biographical details. This structure facilitates in-place updates as new information pertaining to the entity becomes available, while also helping to filter out irrelevant or unnecessary details to maintain a focused and efficient memory. For example, a user profile might look like Listing 4.2.

```
{  
  "name": "Tom",  
  "age": 22,  
  "friends": ["Bob"]  
}
```

Listing 4.2: Example of a user profile schema with basic attributes

- **Collections:** This approach captures discrete pieces of information as individual memory units, often containing fields for situational context and the specific content being remembered. This format offers greater generalization and accommodates a wider variety of information types. For example, a collection might consist of categorized memory strings such as:

- User prefers to be called Sam.
- Sam has experience in JavaScript.
- Sam values clarity and collaboration.

In this case, the choice of a **collection-based** memory structure offers the flexibility needed to accommodate the wide range of user-provided information that emerges naturally during conversation. Rather than being constrained by predefined profile fields, it allows the assistant to retain nuanced and varied details, making the memory system more adaptive and expressive.

Memory Format

In addition to the structure of memory, it's also important to consider the format in which memory is represented. A primary distinction in long-term memory formats for AI agents lies between using natural language and embeddings (Wang et al., 2024). Storing memories directly as **natural language** text offers advantages in readability. However, its main drawback is typically retrieval inefficiency. As memory accumulates, it often becomes too large to fit entirely within an LLM's limited context window or too computationally expensive to process in full. Consequently, agents usually need to selectively retrieve only the most relevant past information, and searching through large volumes of raw text to identify these specific memories can be slow and computationally demanding.

Conversely, **embeddings** encode memories into numerical vector representations. The primary advantage of this format is its retrieval efficiency, allowing for fast similarity searches that can surface relevant information even within large-scale memory stores. As a result, embeddings have become a foundational component of both RAG architectures and long-term memory systems, where relevance-based retrieval is essential.

Memory Operations

For long-term memory to be effective, an agent must perform operations to manage the stored information. These operations allow the agent to acquire, store and retrieve knowledge over time based on its interactions and internal processes. The primary operations include memory reading and memory writing.

Memory reading, or retrieval, focuses on extracting relevant information from the memory store to inform the agent's current reasoning. Effective retrieval often depends on identifying memories that are most pertinent to the current context or query. Key criteria commonly used to score and select memories from the memory set include recency, importance and relevance (Wang et al., 2024), but in this use-case, the focus is on **relevance**, calculated through semantic similarity search over vector embeddings. This approach allows the agent to retrieve only memories that are contextually aligned with the current conversation, enhancing its ability to provide personalized and coherent responses.

Memory writing involves taking raw observations or interactions and projecting them into contents that are actually stored in memory, making them more informative and concise (Zhang et al., 2024).

Memories can be formed in different ways. One approach is **active formation**, also known as the *hot path*, which occurs during an interaction and allows for the immediate capture of critical information. While this ensures that newly saved memories are accessible right away in future sessions or prompts, it can introduce latency. Alternatively, **background formation** takes place between interactions or during idle periods. Its main advantage lies in eliminating latency during user requests and decoupling memory tasks from the core agent logic. However, this approach requires careful consideration of *when* to trigger updates, such as after a period of inactivity or on a predefined schedule, and how *frequently* these updates should occur to maintain a reasonably current memory without incurring excessive background processing costs. An illustration of the differences between these approaches is shown in Figure 19.

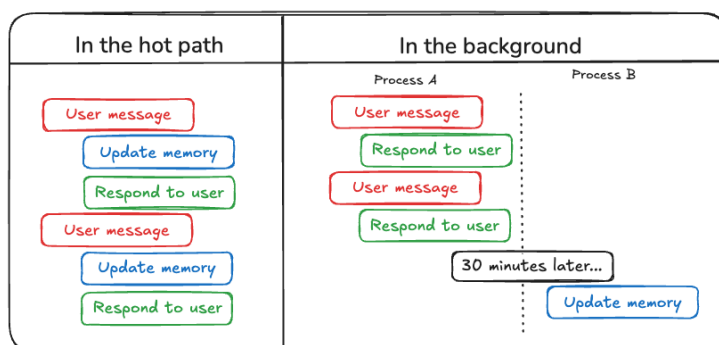


Figure 19: Hot Path vs Background, from *LangGraph Conceptual Docs*¹³

To address the question of when semantic memories should be created, I opted for an **active formation** strategy, also known as “hot-path”. This approach captures relevant information in real time during user interactions, ensuring that new memories are immediately available and reflect the most current context. It also simplifies implementation without major latency issues, thanks to efficient embedding-based retrieval with lightweight models.

4.4.4 Embedding Models

Embedding models are central to the assistant’s memory system, as they determine how conversational inputs are represented and retrieved. During response generation, embeddings of the current input are compared against stored memories to retrieve the most semantically aligned entries, which are then injected into the prompt to improve contextual understanding. The quality of this retrieval depends directly on the choice of embedding model.

To guide selection, the Massive Text Embedding Benchmark (MTEB), introduced by Enevoldsen et al. (2025), provides a standardized evaluation for embedding models across a diverse range of NLP tasks, the structure of which is shown in Figure 20.

¹³ https://langchain-ai.github.io/langmem/hot_path_quickstart/

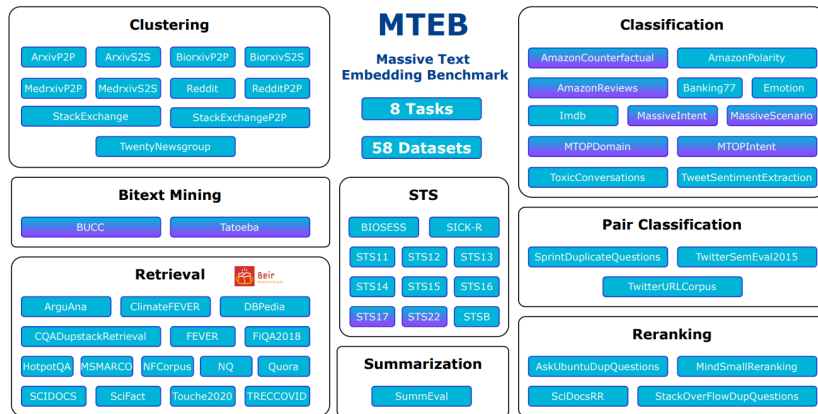


Figure 20: Overview of tasks and datasets in MTEB, from *HuggingFace Blog*¹⁴

Although it spans eight categories, the task of **retrieval** is the most relevant here. Retrieval tasks in MTEB simulate scenarios in which a query must return the most relevant documents from a corpus based on semantic similarity. This closely matches the assistant’s memory mechanism, where the current user input serves as a query to fetch relevant past dialogue fragments from a vector store. Embedding models that perform well on MTEB’s retrieval benchmarks are therefore well-suited to ensuring that the chatbot can access contextually appropriate memory elements.

In some retrieval systems, a reranking step is added, where initial candidates are re-ordered using a heavier cross-encoder model to improve precision. However, given the modest precision needs and strict latency requirements of this assistant, reranking was deemed unnecessary. Direct vector retrieval provides a better balance between relevance and responsiveness.

Another important factor to consider is **language coverage**. The embedding model must provide strong support for Portuguese, while ideally also covering other languages for broader applicability. Multilingual models address this need by learning shared representations across multiple languages, allowing them to transfer knowledge from high-resource languages to those with more limited linguistic data.

Regarding **model size**, for use cases where conversational memory aids context but does not drive critical decision-making, smaller embedding models can offer a favorable trade-off. While larger models may yield slightly better semantic precision, the performance gains often come at the cost of increased memory, latency, and deployment complexity. In this case, where real-time interaction is prioritized and the precision requirements are relatively modest, a compact model is preferable.

The alignment of the model’s training data with the intended application domain is also key. There are domain specialized models that are more prepared to handle more technical vocabulary terms, such as those found in legal, medical or scientific texts. However, this application targets a general-purpose assistant capable of handling everyday tasks and queries rather than any specialized domain. Therefore, a general-purpose model suits this case well.

Licensing and usage rights should also be carefully considered. For this application, it’s important to

¹⁴ <https://huggingface.co/blog/mteb>

choose a model with a permissive license that allows for flexible use and deployment without restrictive terms.

Taking these restrictions and considerations into account, it is useful to review which models perform best on the MTEB retrieval benchmarks. Table 3 presents the top 10 open-source embedding models with fewer than 5B parameters, ranked by retrieval performance.

Table 3: Top 10 open-source embedding models under 5B parameters for retrieval tasks, from MTEB Leaderboard¹⁵

Rank	Model	Retrieval (%)	Zero-shot (%)	Memory (MB)	Params	Embed. Dimensions	Max Tokens
1	Qwen3-Embedding-4B	69.60	94	15341	4B	2560	32768
2	jina-embeddings-v4	66.44	94	7500	3B	2048	8194
3	Qwen3-Embedding-0.6B	64.65	94	2272	595M	1024	32768
4	inf-retriever-v1-1.5b	62.96	77	2944	1B	1536	32768
5	gte-Qwen2-1.5B-instruct	60.78	–	6776	1B	8960	32768
6	snowflake-arctic-embed-l-v2.0	58.36	94	2166	568M	1024	8192
7	multilingual-e5-large-instruct	57.12	94	1068	560M	1024	514
8	gte-multilingual-base	56.50	94	582	305M	768	8192
9	jina-embeddings-v3	55.76	100	1092	572M	1024	8194
10	Solon-embeddings-large-0.1	55.69	–	2136	559M	1024	514

The *Zero-shot* score reflects how well each model performs on benchmarks without having been trained on any data from those specific tasks, where higher percentages indicate better generalization and performance.

Among these results, the *Qwen3-Embedding-4B* model stands out as the top performer, achieving a retrieval score of 69.60% with a memory footprint of 15,341 MB and 4 billion parameters. But it's also notable the fact that *Qwen3-Embedding-0.6B*, with only 595 million parameters, achieves a very close retrieval score of 64.65% while being significantly smaller in size. This significant reduction in size and memory consumption, with only a modest drop in performance, makes it a much more practical choice for this use case, where efficiency and low resource usage are prioritized over precision.

Jina-embeddings models offer a good balance between performance and size, but their license imposes non-commercial restrictions, making it unsuitable for this use case despite being open-source. The *multilingual-e5-large-instruct* model stands out for its robust multilingual capabilities and is widely regarded by the community as a reliable option for general-purpose embedding tasks. There is only one downside to it that is the max tokens it supports, which is only 514 tokens, which limits the reach of the model to process only short texts, which may not be ideal where generally a larger number is desirable. Then there are the ones without information on the zero-shot performance, such as the *gte-Qwen2-1.5B-instruct* and *Solon-embeddings-large-0.1*, which do not guarantee the same level of performance as the others, not being a trustworthy options.

¹⁵ <https://huggingface.co/spaces/mteb/leaderboard>, accessed on 5 July 2025.

Based on these options and the considerations discussed earlier, a subset of models was selected for a second phase of testing. These tests were designed to reflect the intended use case more closely, highlighting practical differences between models and revealing any deployment issues that could hinder their adoption. The selected models were:

- *infly/inf-retriever-v1-1.5b*
- *Qwen/Qwen3-Embedding-0.6B*
- *intfloat/multilingual-e5-large-instruct*
- *Snowflake/snowflake-arctic-embed-l-v2.0*
- *NovaSearch/stella_en_1.5B_v5*

For the evaluation, the tasks were relatively simple but still representative of the intended use case for which I aim to employ embedding models. To define the environment in which the tests would be conducted, I manually created 26 synthetic memories, a number chosen to reflect the typical scale of user memories, formatted identically to how user memories would be stored in the vector store during the actual flow of the application (short statements about preferences, habits, or contextual details).

Using this dataset, I designed 20 retrieval tests in which natural language queries were issued to the system and each test evaluated whether the retriever returned the most relevant memories. Some tests expected exact matches, while others used sets of semantically related results with relaxed thresholds (e.g., a minimum number of expected items out of the top k results). Full test definitions are provided in Appendix B.1.

To summarize the evaluation results, the performance of each retriever can be ranked as follows:

- *multilingual-e5-large-instruct*: **19/20 (95%)**
- *Qwen3-Embedding-0.6B*: **19/20 (95%)**
- *stella-en-1.5B-v5*: **17/20 (85%)**
- *inf-retriever-v1-1.5b*: **15/20 (75%)**
- *snowflake-arctic-embed-l-v2.0*: **3/20 (15%)**

From these results, it is clear that the *multilingual-e5-large-instruct* and *Qwen3-Embedding-0.6B* models performed the best, achieving the highest pass rates across the 20 test scenarios. Given their strong outcomes in these targeted evaluations and their consistently high rankings across public embedding model leaderboards, the remaining models were excluded from further consideration.

Between the two leading candidates, the most distinguishing factors are inference speed, memory requirements and maximum token limits. The *Multilingual E5* model is smaller in size (1068 MB versus 2272 MB for the *Qwen*) but supports only 514 tokens, compared with 32,768 tokens for the *Qwen*. To assess inference speed, both models were evaluated under identical conditions by generating embeddings for 500 input items. The results, presented in Table 4, show that despite its larger size, the *Qwen* model achieves inference times that are very similar to those of the *Multilingual E5*.

Table 4: Inference speed comparison between top embedding models

Metric	Multilingual E5 (s)	Qwen3-0.6B (s)
Total Wall-Clock Time ^a	1.49	1.59
Average Latency	0.87	0.83
Min Latency	0.37	0.36
Max Latency	1.43	1.59
Std. Deviation	0.21	0.28

^a Wall-clock time reflects the real elapsed time for batch or parallel processing, not a sum of per-item latencies.

Considering the results and the practical requirements of the system, the **Qwen3-Embedding-0.6B** model was selected as the embedding model for the virtual assistant’s memory system, instead of the *multilingual-e5-large-instruct* model. While both models achieved strong retrieval accuracy and showed similar inference speeds, the deciding factor was the *Qwen* model’s significantly larger maximum token capacity (32,768 vs. 514). This extended context window allows the system to process longer inputs and more complex interactions, providing greater flexibility for capturing and retrieving conversational context effectively.

4.5 Voice Interface

Having established the virtual assistant’s core, detailing its natural language capabilities and the intricate architecture powering its reasoning and task execution, the next step involves extending this robust foundation into a more practical and intuitive experience for everyday use. In the context of a home assistant, adding a voice interface represents a natural extension. Instead of solely relying on screens or keyboards, the assistant should also be able to respond to voice commands in a hands-free, natural way, allowing users to interact with it as they would with any other smart speaker. This section explores how that transition was approached, and what layers had to be added to bridge the gap between a powerful text-based chatbot and a voice assistant.

4.5.1 System Architecture Overview

This subsection presents a high-level architectural overview of the voice-enabled virtual assistant, detailing how a voice interaction layer was integrated with the existing text-based reasoning core. To support natural, hands-free interaction in a household setting, the assistant must integrate several audio and language components into a seamless, responsive pipeline.

At the core of this voice pipeline is a sequence of modular stages, each responsible for a specific function. Together, these stages transform live audio into intelligent behavior and vice versa. The system is organized into the following key components, as shown in Figure 21:

1. Wake Word Detection
2. Voice Activity Detection (VAD)
3. Speech-to-Text (STT)
4. *Clarisse* Multi-Agent System
5. Text-to-Speech Synthesis (TTS)

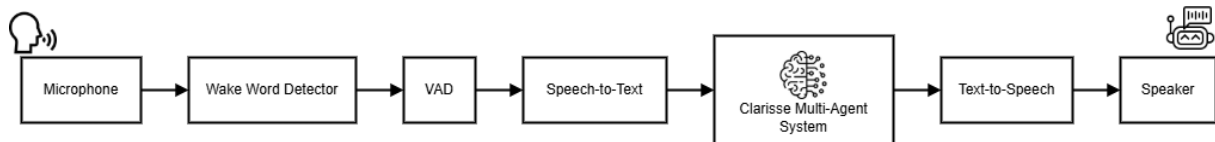


Figure 21: Voice interaction pipeline

4.5.2 Wake Word Detection

The first challenge in enabling truly hands-free interaction with the assistant is determining how and when it should start listening. In traditional interfaces, user input is explicitly initiated through typing or direct interaction with a user interface. However, for a voice-first interface the assistant must passively await user intent without requiring manual activation. This is where wake words play a crucial role.

A wake word is a short, easily remembered phrase that signals to the system that the user is ready to issue a command. It acts as a verbal on-switch, using phrases like “*Hey Assistant*” or “*Computer*” that, once detected, prompt the system to begin processing audio input in real time. This mechanism allows the assistant to remain idle most of the time, conserving power and avoiding unnecessary processing, while still being immediately available when needed.

This architectural division between a wake word engine and a full speech-to-text model is critical for achieving both responsiveness and resource efficiency. Wake word detectors are specifically designed for continuous, low-power operation, allowing them to run persistently without significant CPU or memory usage. In contrast, full speech-to-text models are computationally intensive and would quickly drain system resources if left running at all times. From a privacy perspective, wake word detection also ensures that audio is only processed and transcribed after the user explicitly addresses the assistant, reducing the risk of capturing or storing unintended background conversations.

Selection Criteria

The selection of a wake word detection engine was guided by the project’s core commitment to open-source, self-hosted solutions. The following criteria were established:

- **Licensing:** The engine had to be available under a permissive open-source license for unrestricted use.

- **Customization:** It needed to support the training of custom wake words locally, without reliance on external cloud services.
- **Performance:** It had to be efficient enough for real-time, on-device detection on low-power hardware.
- **Integration:** Comprehensive documentation was required to ensure a straightforward integration process.

Candidate Analysis and Selection

Several alternatives were evaluated before selecting the final solution. The commercial engine *Picovoice Porcupine* demonstrated impressive accuracy and efficiency. However, creating custom wake words requires using *Picovoice*'s paid cloud service. This dependency on proprietary infrastructure conflicted with the project's goal of a fully open and self-hosted solution. *Mycroft Precise*, once a strong contender in the open-source space, has seen its development stagnate since the discontinuation of *Mycroft AI*. While it remains functional, its outdated tooling, higher false-accept rates, and weaker community support make it less suitable for the project. *Howl* offered an interesting approach, but its limited development activity and smaller community reduced its long-term viability.

Following this comparative analysis, **openWakeWord**¹⁶ emerged as the optimal choice for custom wake word detection. Several factors motivated this selection. First, its lightweight and modular architecture facilitates straightforward integration and extensibility. Second, the framework provides pre-trained models alongside an automated training pipeline that supports both real and synthetically generated datasets through TTS, significantly simplifying the customization workflow. Third, being optimized for edge devices such as the Raspberry Pi, *openWakeWord* delivers efficient, low-latency performance without cloud dependencies. Additionally, its active adoption by prominent open-source communities, including *Home Assistant* and *OpenVoiceOS*, demonstrates its reliability and continued relevance in production environments. Finally, by leveraging industry-standard frameworks such as ONNX and TensorFlow Lite, *openWakeWord* ensures both portability and optimized performance across diverse hardware platforms.

Implementation Details

The *openWakeWord* library includes pre-trained models for common wake words (e.g., “*Alexa*”, “*Jarvis*”), but since the assistant developed in this work is named *Clarisse*, a custom model was required. The framework provides high-level workflows for creating new wake word models through Jupyter notebooks, which abstract most of the architectural complexities while still exposing advanced options for fine-tuning.

The training process for a *openWakeWord* model can be divided into two stages:

1. **Data Generation:** Positive samples of the wake word are created using open-source TTS models, with added noise and acoustic variations to simulate real-world conditions. Negative samples

¹⁶ <https://github.com/dscripka/openWakeWord>

(speech, music, ambient noise) are added to reduce false activations.

2. **Model Training:** The framework relies on an embedding-based keyword spotting system. Incoming audio is converted to mel-spectrograms, passed through a frozen convolutional backbone to extract embeddings, and finally classified by a lightweight neural head. Training adjusts only the classifier, making the process efficient while preserving generalization from large-scale pre-training. In the end, each training run produces metrics such as accuracy, recall, and False Positives per Hour (FPH), providing an initial indication of model quality.

A limitation encountered in the provided notebooks was the lack of native support for European Portuguese. These notebooks utilize a `piper-sample-generator` submodule to generate training samples, but this submodule supported only English, French, German, and Dutch voices. This restriction resulted in phonetically inaccurate pronunciations of “*Clarisse*”, leading to poor dataset quality. To address this, two strategies were explored:

- **Approximation with English phonetics:** Generating English phrases that mimic Portuguese pronunciation, for example, “*Hey Cledeess*” produced by an English TTS model sounds close to “*Hey Clarisse*” in Portuguese.
- **Adaptation for Portuguese models:** Extending the generator module to support European Portuguese voices directly, a more technically demanding task, while also incorporating Brazilian Portuguese and Spanish TTS models to compensate for the scarcity of available European Portuguese voices. This combination, together with minor textual tweaks (e.g., “*Hey Clãriss*”), produced training samples with greater phonetic fidelity to native European pronunciation.

Both strategies yielded workable datasets, but the adapted Portuguese-based approach was ultimately used, as it offered greater flexibility for potential future wake words with more complex phonetic structures.

With the sample generation process established, the next step involved fine-tuning the model’s classifier. The training pipeline exposes several hyperparameters that strongly influence detection quality:

- `number_of_examples`: controls the size of the positive dataset and therefore the variety of pronunciations and acoustic conditions.
- `number_of_training_steps`: defines the optimization depth. Higher values reduce underfitting but risk overfitting with small datasets.
- `false_activation_penalty`: adjusts the weight of negative samples in the loss function, reducing false positives at the potential cost of missed detections.

Choosing appropriate values for these parameters is an inherently iterative process, as each adjustment involves balancing competing objectives. However, the tuning strategy emphasized minimizing false positives, as unintended activations were deemed more detrimental to user experience than slightly lower sensitivity. Guided in part by insights shared within the *openWakeWord* community, I followed an iterative process of setting the hyperparameters, training a new model, and then evaluating its performance.

To validate results, model performance was assessed using standard classification metrics:

$$\begin{aligned}
\textbf{Accuracy} &= \frac{TP + TN}{TP + TN + FP + FN} \\
\textbf{Recall} &= \frac{TP}{TP + FN} \\
\textbf{FPH} &= \frac{FP}{H}
\end{aligned} \tag{4.1}$$

where TP , TN , FP , and FN denote true positives, true negatives, false positives, and false negatives, and H represents the total hours of negative audio evaluated.

Final Model and Results

After several rounds of tuning and evaluation, multiple models were trained with different parameter configurations and assessed through validation metrics. The detailed configurations and corresponding evaluation results for all trained models are presented in Appendix A.1.

Based on the analysis of these results, complemented by additional practical testing, one configuration was selected as the final model.

The chosen model was trained with 40,000 positive samples, 30,000 training steps, and a false activation penalty of 3,000. It achieved an accuracy of 76%, a recall of 52%, and an average of 0.0 false positives per hour. The complete absence of false activations, while favorable, came at the cost of a moderate recall, indicating that the model was conservative in triggering only when highly confident.

Although the validation metrics reported a recall of only 52%, this is partly explained by the challenging evaluation dataset and the conservative tuning aimed at minimizing false positives. The open-source nature of the framework, combined with the limited variability of Portuguese voices in the datasets, also constrains the robustness of the validation. Nonetheless, when interpreted comparatively with the other models, these results remain meaningful.

Real-world testing, using both my own voice and that of other speakers, confirmed the effectiveness suggested by the evaluation metrics. In everyday conditions, the system consistently recognized the wake word without noticeable issues. Despite the initial challenge of lacking a Portuguese-based sample generation solution, the adopted approach resulted in a model that performs well in practice and is suitable for integration.

4.5.3 Voice Activity Detection

Voice Activity Detection (VAD) is an audio-processing technique that identifies the presence or absence of human speech within a continuous signal. By detecting the boundaries of speech segments and distinguishing them from silence, background noise, or other sounds, VAD enables the system to determine when the user is actively speaking and when they have finished.

After the wake word has been detected and recording begins, the system needs a mechanism to determine exactly when the user's spoken prompt is complete and ready to be processed. For this purpose, VAD operates immediately after the wake word detector in the voice processing pipeline, being responsible for identifying the precise moment the user stops speaking.

For the implementation of voice activity detection in this project, the chosen solution was *Silero VAD*¹⁷, a lightweight neural model designed for real-time speech activity detection. Distributed as a Python package (*silero-vad*), it was straightforward to install and integrate into the processing pipeline with minimal configuration, which made it highly practical. Alternative open-source solutions, such as *WebRTC VAD*¹⁸, were also evaluated. However, *Silero VAD* demonstrated superior performance and greater flexibility, particularly through its adjustable silence thresholds, which allowed better adaptation to the requirements of this use case.

To optimize the user experience, the recording process follows a carefully defined sequence that ensures the entire user utterance is captured accurately, detailed in Figure 22.

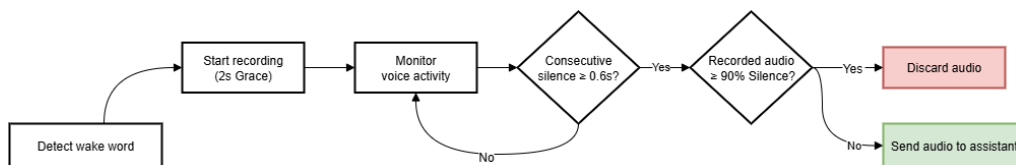


Figure 22: Voice activity detection flowchart

At the start of the recording, there is a grace period of around two seconds during which silence is ignored. This allows users to take a brief pause before starting to speak without risking premature termination of the recording. During this interval, all audio is captured regardless of content.

After this initial period, the system begins actively monitoring for silence. Recording continues as long as speech is detected, but if silence persists consecutively for 0.6 seconds, the system assumes the user has finished speaking. At this point, the recording stops and the captured audio undergoes a final validation. If more than 90% of the recording consists of silence, the system discards the input audio to prevent empty or irrelevant requests. This filtering mechanism serves as a safeguard against false wake word activations, minimizing unnecessary server requests and preventing unintended responses that could disrupt the user experience.

The specific values used for the grace period (2 seconds), silence detection window (0.6 seconds), and silence threshold (90%) were determined empirically through iterative testing, ensuring that the system provided a natural interaction experience.

By accurately determining when the user has finished speaking and filtering out false activations, VAD plays a critical role in this pipeline, ensuring that only complete and relevant audio reaches the assistant engine, thereby improving efficiency and reliability.

¹⁷ <https://github.com/snakers4/silero-vad>

¹⁸ <https://github.com/wiseman/py-webrtcvad>

4.5.4 Speech-to-Text

Speech-to-Text (STT), also referred to as Automatic Speech Recognition (ASR), is the process of converting spoken language into written text. Within the voice-assistant pipeline, STT operates after wake word detection and voice activity detection, transforming the user's speech into text that downstream natural language processing components can interpret. The STT component is therefore crucial, as its accuracy and speed directly influence whether the assistant correctly understands the user's intent and responds in a timely manner.

Selection Criteria

When selecting a suitable STT model for integration into the voice pipeline, multiple performance and usability factors must be considered. The following criteria guided model evaluation and ultimately informed the implementation choice:

- **Latency:** The model's ability to process audio efficiently is a critical factor. In a voice pipeline, each component acts as a link in a chain, with the overall performance constrained by the slowest link. Any delays introduced by the STT component degrade system responsiveness and lead to a suboptimal user experience. Minimizing latency often requires specialized hardware. For instance, OpenAI's Whisper Large model can achieve sub-second transcription latency on high-end GPUs, whereas the same model on a CPU may be tens of times slower¹⁹. This performance gap makes real-time transcription on consumer-grade, on-device hardware nearly impossible.
- **Transcription quality in the target dialect:** Accuracy must be assessed not only through standard metrics such as Word Error Rate (WER) and Character Error Rate (CER), but also with regard to performance in the target dialect. WER measures the proportion of incorrectly substituted, deleted, or inserted words relative to a reference transcription, while CER applies the same principle at the character level. Multilingual ASR systems generally underperform compared to English-centric models due to disparities in training data availability (Yadav and Sitaram, 2022). This limitation is especially pronounced for under-resourced dialects, such as European Portuguese.

Candidate Analysis and Selection

With the key requirements of low latency and accurate European Portuguese transcription in mind, this section evaluates several open-source models and compares their performance to support an evidence-based final selection.

The first candidate, **SpeechBrain**, is an open-source speech processing toolkit built on PyTorch, released under the permissive Apache License. The toolkit includes a collection of pre-trained models on Hugging Face, which can simplify deployment and speed up prototyping for speech and text processing

¹⁹ <https://blog.lopp.net/openai-whisper-transcription-testing>

applications. For European Portuguese, the model *asr-wav2vec2-commonvoice-14-pt*²⁰, fine-tuned on the Common Voice Portuguese dataset, is widely accessible. However, its reported WER of 22.51% on the *Common Voice 14.0 Portuguese* test set, together with empirical evaluations, indicates that the model is insufficient for production-level use.

Next, the **Vosk** library was assessed for its on-device capabilities. Testing a small Portuguese model (*vosk-model-small-pt-0.3*)²¹ on a Raspberry Pi showed that while it was the best-performing model for that hardware class, its transcription error rate remained too high for the assistant’s downstream pipeline.

Ultimately, OpenAI’s **Whisper** emerged as the most suitable foundational technology. Whisper is a state-of-the-art encoder-decoder transformer trained on a diverse 680,000-hour corpus of audio-text pairs, covering multiple domains and languages (Radford et al., 2022). Whisper is released under a permissive MIT license and supports fully local deployment, aligning with both privacy requirements and unrestricted adaptability.

Relevant features include prompted decoding, where an initial text prompt can guide transcription towards domain-specific vocabulary or spellings, and a range of model sizes are available (e.g., tiny, small, medium, large), balancing quality against inference latency and hardware footprint.

The open-source release of Whisper has also given rise to an active optimization ecosystem. Variants such as *faster-whisper* focus on minimizing the latency of a single transcription, prioritizing raw speed for individual tasks, which makes its architecture less suited for the demands of a high-concurrency, multi-user environment. In contrast, recent developments like the integration with inference acceleration frameworks such as *vLLM* are architecturally designed to maximize throughput²².

The selection process considered the availability of fine-tuned Whisper models for Portuguese. The model *whisper-large-v3-pt-cv19-fleurs*²³ was identified as a suitable option. It was fine-tuned on Portuguese speech data, with a reported WER of 7.56% on held-out data from its training distribution, it represents a strong baseline for high-accuracy transcription.

To validate the model selection for this project, a use-case-specific evaluation was conducted. The test set comprised 50 audio clips of typical user commands in European Portuguese, captured from 5 different speakers with equal representation, averaging 4 seconds in duration. The samples covered smart home controls, weather queries, reminders and conversational phrases (see Appendix C.2 for details). Evaluations were performed on an NVIDIA H200 GPU, with results summarized in Table 5 based on WER, CER and average latency per request.

²⁰ <https://huggingface.co/speechbrain/asr-wav2vec2-commonvoice-14-pt>

²¹ <https://alphacephei.com/vosk/models>

²² <https://huggingface.co/blog/fast-whisper-endpoints>

²³ <https://huggingface.co/fsicoli/whisper-large-v3-pt-cv19-fleurs>

Table 5: Performance comparison of STT models on a custom test set

Model	WER (%)	CER (%)	Average Time (s)
whisper-large-v3-pt-cv19-fleurs	12.90	5.17	1.173
whisper-large-v3	17.20	6.96	1.177
whisper-medium	22.58	12.92	1.111

The data in Table 5 highlights two key observations. First, the fine-tuned model demonstrates superior accuracy, achieving a Word Error Rate of 12.90%, which is a significant improvement over both the standard large-v3 (17.20%) and medium (22.58%) models. Second, it was observed that the expected trade-off between model size and performance is not evident in this benchmark with the medium model's latency advantage being minimal. This is likely attributable to the short duration of the audio samples where the fixed overhead of the inference pipeline constitutes a significant portion of the total execution time, thereby masking the raw computational differences between the model sizes. This finding reinforces that for the short, command-based interactions typical of this application, sacrificing substantial accuracy for a negligible performance gain would be an unfavorable trade-off. Therefore, given its empirical superiority in accuracy and competitive latency on representative tasks, *whisper-large-v3-pt-cv19-fleurs* was selected as the STT component for the voice pipeline.

Implementation Details

Even with software optimizations, real-time execution of high-accuracy STT models such as Whisper Large on resource-constrained platforms like Raspberry Pi-class CPUs proved unfeasible. Testing confirmed that both transcription speed and memory requirements exceeded acceptable limits for the intended application.

To maintain accuracy while ensuring low latency, inference for the selected model was offloaded to a dedicated GPU-powered server running *vLLM*. This deployment approach enables efficient handling of multiple requests, delivers consistent high-quality transcriptions without overburdening the client device and ensures that the speech-to-text component operates reliably within the broader voice-assistant pipeline.

4.5.5 Text-to-Speech

Text-to-Speech (TTS) converts written text into spoken language and is an essential element in Human-Computer Interaction (HCI). Within a virtual assistant, TTS enables the system to deliver responses audibly, allowing for more natural communication.

Contemporary TTS models leverage powerful neural networks to produce highly realistic and expressive speech. Recent advances enable the generation of conversational audio with human-like intonation, emphasis, and emotional prosody, reaching a level of realism that was previously unattainable.

Despite these advances, progress has largely focused on a relatively small set of languages. European Portuguese, in particular, remains underrepresented in high-quality, readily available text-to-speech solutions. In contrast to speech-to-text systems, where Brazilian Portuguese models can often be applied to European Portuguese contexts with limited impact on recognition quality, speech synthesis presents a far greater challenge. In text-to-speech, the phonetic, prosodic and lexical differences between the two variants are immediately noticeable, making Brazilian-oriented models inadequate for European Portuguese.

Selection Criteria

Within the scope of this project, the TTS component must satisfy the following critical requirements:

- **Native European Portuguese support:** The TTS system must provide high-quality synthesis for the European Portuguese variant.
- **Open-source availability:** The system must be distributed under permissive open-source licenses, in line with the project's reliance on freely available technologies.
- **Efficiency and low resource usage:** Ideally, the system should operate with minimal latency and computational overhead, enabling real-time performance on resource-constrained devices without heavy reliance on GPU acceleration.
- **Custom voice creation:** A personalized voice allows the assistant to develop a distinct “persona”, enhancing user engagement and reinforcing identity. The TTS system should therefore support the creation of a recognizable custom voice, for example through voice cloning, rather than relying solely on generic, pre-trained voice models.

Candidate Analysis and Selection

A comprehensive and critical evaluation of existing TTS technologies was conducted to identify the most appropriate solution for integration into the virtual assistant. While this process was guided by the strategic requirements previously outlined, the primary selection criterion was robust support for European Portuguese. This strict requirement served as the initial filtering layer, leading to the immediate exclusion of many solutions that, while highly-regarded in English-only benchmarks, were not viable for this project.

*Nari Labs' Dia-1.6B*²⁴ is an open-source TTS model released in April 2025. With 1.6 billion parameters, it is designed to generate natural-sounding speech from text. The model allows expressive control by conditioning output on a reference audio track to guide emotion and tone, and it can also produce nonverbal vocalizations such as laughter and coughing. However, *Dia-1.6B* has important limitations for this project. It currently supports only English, with no Portuguese capability, and it requires substantial computational resources, on the order of 10 GB of GPU memory. As a result, despite its expressive features, the model was excluded primarily due to its lack of language support.

²⁴ <https://huggingface.co/nari-labs/Dia-1.6B>

Kokoro TTS is an open-source model with 82 million parameters. Its relatively small size makes it suitable for deployment on resource-constrained hardware, with support for real-time inference on CPU and compatibility with ONNX pipelines. Initially developed for English, the model has since been extended to multiple languages, including Brazilian Portuguese. However, it does not provide built-in voice-cloning functionality. The lack of native European Portuguese support, together with the absence of cloning capabilities, meant that *Kokoro TTS* was not appropriate for this project’s objectives.

Parler-TTS is an open-source text-to-speech framework released by *Stability AI*. It supports expressive control through textual style prompts rather than audio-based voice cloning. For this project, however, a key limitation is that the available Portuguese voice corresponds to the Brazilian variant. As a result, *Parler-TTS* was not considered suitable for the objectives of this thesis.

*Piper*²⁵ is an open-source, MIT-licensed neural text-to-speech system explicitly designed for efficient deployment on edge devices. It features a small footprint, enabling it to deliver voice output with low latency even on resource-constrained hardware like the Raspberry Pi. The framework provides several pre-trained multilingual voices, among them a community-contributed European Portuguese model named *pt_PT-tugão-medium*²⁶. Another strength of *Piper* is its support for voice adaptation, allowing a base model to be fine-tuned with a relatively small number of recordings from a target speaker to create a personalized voice.

Table 6 summarizes the shortlisted TTS solutions. The comparison focuses on European Portuguese support, speaker adaptation capabilities, and computational cost, expressed as relative resource demands for real-time inference.

Table 6: Evaluation of candidate TTS solutions

Solution	pt-PT	Custom Voice	Computational Cost	License
Nari Dia	No	Yes	High	Apache-2.0
Kokoro TTS	No (pt-BR)	No	Low	Apache-2.0
Parler-TTS	No (pt-BR)	No	Moderate	Apache-2.0
Piper TTS	Yes	Yes	Low	MIT

Based on the evaluation presented above and summarized in Table 6, the selection process identified a single viable candidate. While several systems showed strengths in specific areas, none fulfilled the full set of requirements except *Piper TTS*. Consequently, *Piper TTS* was selected as the framework for the voice of the virtual assistant.

²⁵ <https://github.com/rhasspy/piper>

²⁶ https://huggingface.co/rhasspy/piper-voices/tree/main/pt/pt_PT/tug%C3%A3o/medium

Implementation Details

A significant aspect of implementing the virtual assistant was creating its unique voice. To personalize the assistant, *Piper* was fine-tuned to create a custom voice. Unlike zero-shot cloning methods, *Piper* adapts a pre-trained model through additional training on new speaker data. While this method requires more initial setup than zero-shot techniques, the utilities and guides provided by the *Piper* framework still make the process straightforward to implement.

For this project, a female volunteer speaker of European Portuguese recorded 50 short utterances, paired with transcriptions in a CSV file. Using this dataset, fine-tuning was carried out on the community-provided European Portuguese model *Tugão*, which served as the base checkpoint. During this process, the pre-trained weights were adapted to the new speaker data until the loss function stabilized, indicating the model had successfully converged on the target speaker’s vocal characteristics. The resulting checkpoint was exported in the standard ONNX format, producing a portable voice package with the ONNX model file and its corresponding JSON configuration file, which together enable the virtual assistant to synthesize speech using the adapted voice model.

Once integrated into the assistant, this adapted model must operate in real time. Since the assistant’s responses are generated by an LLM in a streaming fashion, the TTS system must handle partial outputs efficiently. Feeding text token by token would yield robotic speech, as TTS models require sentence-level context to produce natural intonation. To address this, the system buffers the stream until a full sentence is detected (via terminal punctuation), then sends it to *Piper*. This allows the assistant to start speaking earlier, giving the interaction a smoother feel without requiring the full assistant response to be available in advance.

Finally, a preprocessing layer was added to normalize text before synthesis. This step expands abbreviations, numbers, and symbols (e.g., “25°C” to “twenty-five degrees Celsius”), preventing errors that would otherwise result from *Piper*’s literal interpretation of input text.

In summary, the successful implementation of *Piper TTS* resulted in a robust, personalized text-to-speech system. This component meets the project’s core requirements for open-source licensing, low memory footprint and voice customization, achieving a final solution that is both functional and reliable.

4.6 Deployment and Integration

The virtual assistant follows a distributed system design, with components deployed across multiple devices to balance performance, responsiveness, and resource constraints. The system is logically divided into three main components: the **Clarisse multi-agent system**, orchestrated through a **FastAPI** server and powered by GPU-deployed reasoning components such as large language models and embedding models; the **smart speaker client**, which provides a hands-free voice interface; and the **mobile application client**, which offers a graphical user interface for interaction.

4.6.1 Clarisse Multi-Agent System Deployment

At the core of the architecture is the *Clarisse* multi-agent system, deployed on an application server running FastAPI. This service acts as the primary orchestration layer between client devices and the GPU inference components, exposing a set of REST endpoints that provide high-level functionalities of the virtual assistant while coordinating the intermediate processing and requests required to support their execution. This design decouples the lightweight client applications from the computationally heavy back-end services, ensuring a clean and reusable API boundary.

Figure 23 presents the deployment view, showing how the *Clarisse* system is exposed as a service through FastAPI, mediating all interactions between heterogeneous clients and the GPU-powered inference components.

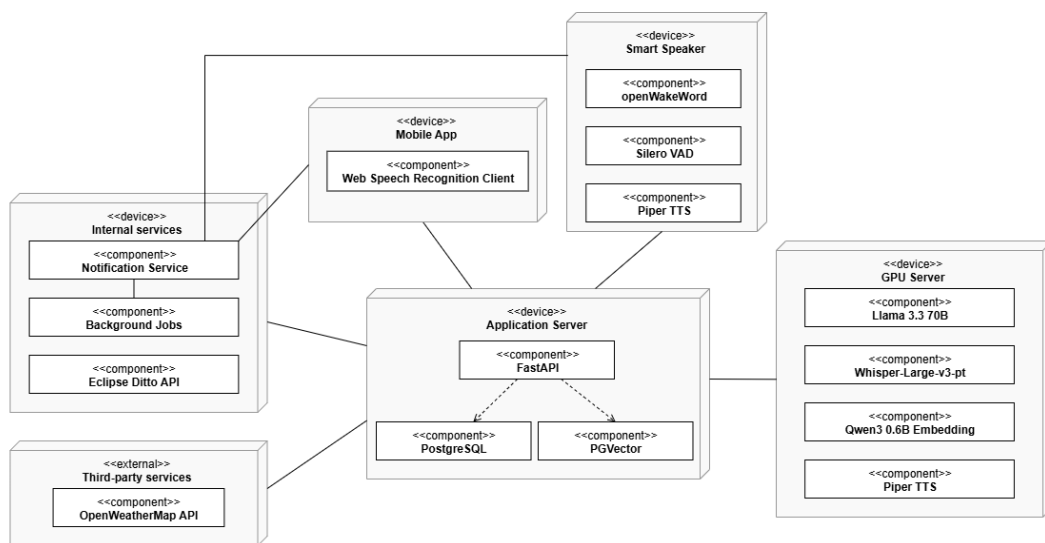


Figure 23: Deployment diagram of the *Clarisse* virtual assistant

When a request arrives from a client, whether it originates from the smart speaker or the mobile application, the FastAPI server orchestrates the full processing workflow required to generate a response. Its responsibilities include handling audio processing through GPU-powered inference services, processing textual prompts through the *Clarisse* multi-agent system, submitting requests to the LLM as determined by the agents, and executing tools when required. Through these tools, the system integrates with internal services (Background Jobs, Notification Service, Eclipse Ditto API) as well as external providers (*OpenWeatherMap*). By centralizing these responsibilities, the architecture keeps client applications lightweight while ensuring a single, consistent point for access control and system management.

4.6.2 Smart Speaker Deployment

To extend the system's capabilities to a full voice-controlled experience, a dedicated smart speaker prototype was developed. This device, built on a Raspberry Pi and equipped with a microphone and

speaker, provides the physical platform for enabling natural voice interaction with the assistant.

Figure 24 shows the physical prototype of the device, built on a Raspberry Pi and equipped with an external microphone and speaker.



Figure 24: Physical prototype of the smart speaker built with a Raspberry Pi

The underlying software architecture that supports this device follows a distributed client-server model, balancing the need for continuous, low-power listening on the speaker with the computationally intensive processing carried out on the server.

Voice Interaction Pipeline

The complete voice interaction pipeline, illustrated in Figure 25, outlines the system's voice components and their execution across the smart speaker and the GPU server, detailing the sequence of activities from user speech input to the final synthesized assistant response.

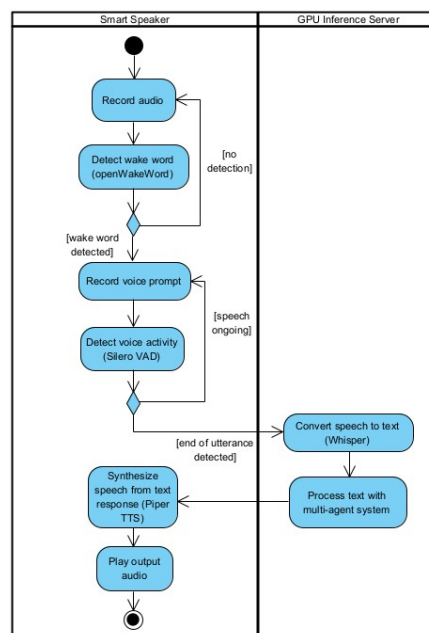


Figure 25: Activity diagram of a request from the smart speaker

The operational flow is orchestrated by a persistent Python application on the Raspberry Pi, designed for a headless, always-on environment and integrated with a microphone and a speaker. This client application handles the on-device audio pipeline while delegating computationally intensive stages of the interaction pipeline to the server-side infrastructure.

The process begins with the on-device microphone, which continuously streams audio to a local wake word detection engine, **openWakeWord**. This lightweight model is optimized for embedded systems and listens for the predefined trigger phrase, more specifically “*Hey Clarisse*”. Upon successful detection, the system activates the **Silero VAD**. The VAD’s role is to identify the natural end of the user’s spoken command, ensuring that only the relevant utterance is captured.

Once the VAD detects the user has finished speaking and confirms the segment contains human speech, the captured audio is sent to the GPU-powered inference server for transcription by a Portuguese fine-tuned **Whisper Large v3** model. As justified in Subsection 4.5.4 (Speech-to-Text), this task is offloaded to the server, as the model’s significant computational and memory requirements exceed the Raspberry Pi’s capabilities and would otherwise compromise client-side responsiveness. The resulting text is then passed to the assistant’s core text-processing engine for reasoning and response generation.

After the server has processed the request and begins generating a response, the text is streamed back to the Raspberry Pi as it is produced. The decision to perform speech synthesis on the client device is based on two synergistic factors. Firstly, transmitting a lightweight text string reduces network latency compared to streaming pre-rendered audio from the server. Secondly, the **Piper TTS** engine is highly efficient, allowing it to generate speech quickly even on the resource-constrained hardware of the Raspberry Pi. This combined approach ensures the final step of creating audio is fast, as the efficient local processing works together with the low-latency network transfer. Finally, the locally generated audio is played through the speaker, concluding the interaction by delivering an audible response.

Barge-In Support and Acoustic Echo Cancellation

Beyond the primary request-response pipeline, a critical feature for natural human-computer interaction is the ability for a user to interrupt the assistant mid-response. This “barge-in” capability is enabled by bidirectional audio streaming, where the system can simultaneously receive and transmit audio. However, implementing this feature introduced a significant challenge, called acoustic echo. The audio output from the speaker can be picked up by the microphone, creating a feedback loop where the assistant’s own voice is interpreted as a new command. This can lead to the undesirable behavior of the assistant silencing itself prematurely.

The most effective solutions to acoustic echo are typically hardware-based, employing microphone arrays with dedicated digital signal processors to perform advanced techniques like beamforming, which isolates the user’s voice by directionally focusing the microphones’ sensitivity. Given that this project was constrained to a standard microphone and speaker configuration, a software-level approach was necessary.

To address this challenge directly, a software-based Acoustic Echo Cancellation (AEC) solution was implemented at the operating system's audio subsystem level. The core of this solution is an echo cancellation algorithm, which functions by creating a virtual audio loopback. It continuously monitors the audio signal being sent to the speaker and computationally subtracts this signal from the microphone's input stream in real-time. The goal of this process is to isolate the user's voice by filtering the assistant's own speech from the microphone's input. While effective to a degree, this purely software-based method is not a perfect substitute for dedicated hardware, as it may occasionally fail to eliminate echo completely.

Given the residual echo from the software-only AEC, simple voice activity detection is insufficient for reliably distinguishing the user's speech from the assistant's own playback. Therefore, a more robust hybrid approach was adopted. This approach combines the software-based AEC with a specific trigger for silencing the assistant. Instead of any detected speech cancelling the playback, only the detection of the same wake word that initiated the interaction will cause the assistant to stop speaking.

This hybrid strategy provides two key advantages. The AEC enhances the reliability of the wake word engine by reducing echo, while the wake word trigger itself ensures that interruptions are intentional and not triggered accidentally by residual echo from the assistant's own voice. This ensures that interruptions are always deliberate and under the user's control, rather than accidental side-effects of echo.

4.6.3 Mobile Application Design and Architecture

To provide a portable, screen-based method of interaction with the virtual assistant, a mobile application was developed as a Progressive Web Application (PWA). A PWA is a web application that runs in a browser but offers an experience similar to a native app, including offline capabilities and installation on the home screen. This implementation employs an architectural model for voice interaction tailored specifically for mobile devices, leveraging their native, built-in capabilities. The design is optimized for the intermittent and user-initiated nature of mobile use cases, which distinguishes its operational principles from the continuous listening paradigm of an in-home smart speaker.

The activity diagram in Figure 26 shows how a voice request is processed in the mobile application, highlighting the explicitly user-initiated interaction and the reliance on browser-based speech recognition.

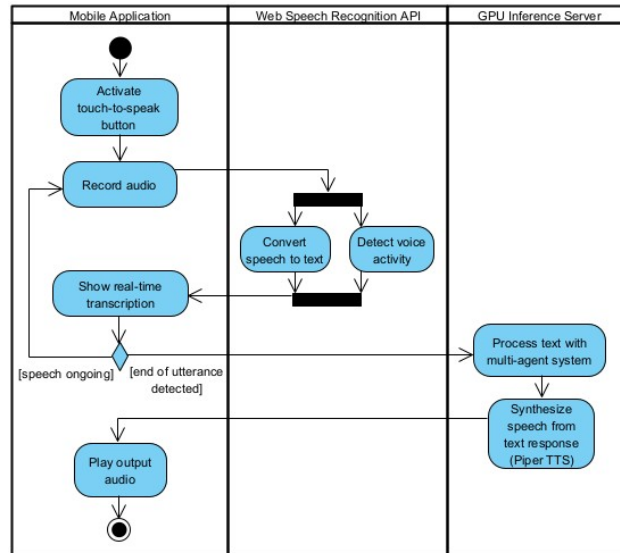


Figure 26: Activity diagram of a request from the mobile application

Unlike the “always-on” smart speaker, user interaction in the mobile application is explicitly initiated via a “touch-to-speak” mechanism within the user interface. This engages the browser’s **Web Speech Recognition API**, a high-level World Wide Web Consortium (W3C) interface that abstracts the entire STT pipeline. While the precise implementation is browser-specific, the captured audio is typically processed by a cloud-based provider managed by the platform provider (e.g., Google or Apple) which returns a transcribed text prompt. This design offers several benefits:

- **Reduced infrastructure load:** The computationally intensive task of transcription is entirely offloaded to the platform provider, incurring no processing load on the system’s servers.
- **Platform-provided accuracy and latency:** The system leverages the provider’s existing ASR models, which are configured for the Portuguese locale and designed to return real-time results.
- **Simplified development and real-time feedback:** The API abstracts the complexity of real-time audio streaming and processing. This simplifies the development process and facilitates an interface that provides the user with a live transcription of their speech, allowing for immediate verification of their input.

Once the transcription is available, it is forwarded to the **Clarisse multi-agent system**. When the assistant’s core logic generates a textual response, it is returned in parallel: as a textual reply for the frontend chat and as input to the **Piper TTS** engine. *Piper* synthesizes an audio response, which is returned to the app for playback alongside the textual output.

The decision to handle TTS synthesis on the server was driven by a significant technical constraint, namely the difficulty of compiling the native *Piper TTS* engine for in-browser execution. Offloading this step to the backend was deemed a necessary trade-off to enable voice output on the mobile platform. While it introduces network latency for audio streaming, the architecture simplifies the client-side application and is a pragmatic solution for a PWA, especially given the benefits of using platform-native STT for input.

Chapter 5

Testing and Evaluation

This chapter presents the evaluation methodology applied to the assistant, covering both functional and efficiency dimensions. The functional evaluation addresses whether the system behaves correctly across its main capabilities. In parallel, the efficiency evaluation examines how quickly the assistant produces and delivers its responses, with a particular focus on latency distribution across the interaction pipeline stages. Together, these perspectives provide a balanced assessment, where functional tests validate correctness and robustness of behavior, while efficiency tests assess responsiveness and user experience.

5.1 Functional Evaluation

Evaluating the performance of a virtual assistant powered by Large Language Models poses unique challenges. Unlike traditional software systems, which can be validated through deterministic unit tests, LLM-based assistants operate in the open-ended domain of natural language. This creates a vast and unpredictable space of possible interactions, making it difficult to define exhaustive test cases or strict correctness criteria.

A further complication is that small changes in prompt or context design can significantly alter system behavior. *Prompt engineering* shapes the model's reasoning and output through carefully crafted instructions and examples, while *context engineering* structures conversational state, memory, and auxiliary information passed to the model (Mei et al., 2025). These practices are empirical and iterative, often producing unexpected results.

To address this variability, the project adopts a test-driven development strategy, where tests are defined as an integral part of the development process and every modification to the system is validated against them to ensure correctness and stability. This approach provides regression protection, meaning that changes to prompts, workflows, or tool interfaces are checked to prevent improvements in one area from unintentionally degrading behavior in another.

The evaluation framework is implemented in *PyTest* and serves as the central validation mechanism. Each test suite targets a specific agent or capability (Devices, Reminders, Memory, Weather, Orchestrator, Speech-to-Text, Language Quality) and verifies system outputs under controlled conditions. Mocked tools preserve service interfaces while avoiding side effects, ensuring that tests measure the LLMs's reasoning

and planning rather than the reliability of external systems. The framework also emphasizes reproducibility by fixing dynamic factors such as time and reference inputs, while allowing flexibility by accepting legitimate linguistic or semantic variation. For a sample of how test cases are defined in JSON format, see Appendix B.2, which includes a representative excerpt from the test suite definition for reminders.

Building on this foundation, the following subsections detail how the evaluation framework is applied across different capabilities of the assistant, including device control, reminder scheduling, domain classification, weather queries, memory handling, language quality, and speech recognition.

5.1.1 Device Control and Argument Extraction

These tests verify that the Devices Agent selects the correct tools in the expected sequence (e.g., *get_user_devices*, *check_specific_device_settings*, *update_device_settings*) and extracts semantically correct, normalized arguments such as room names, device types and property values. Tool calls generated by the agent are compared against expected sets for a variety of scenarios, including multi-device routines and error conditions. This confirms both the accuracy of tool selection and the correctness of the arguments provided to those tools.

A representative scenario, detailed in Table 7, involves turning off the living room light. The system must identify the correct device, verify its current state, and issue the appropriate update.

Table 7: Device control test case

Environment	Living Room with devices {Light, Thermostat, Curtain}
Initial State	Living Room Light: Is_turned_on = true, Brightness = 100%
User Input	Turn off the living room light.
Expected Tool Calls	get_user_devices → check_specific_device_settings → update_device_settings
Expected State	Living Room Light: Is_turned_on = false

5.1.2 Reminder Scheduling and Temporal Reasoning

Reminder creation is validated in terms of CRON correctness for single-occurrence and recurring reminders, as well as the correct resolution of *next_execution* and optional *valid_until* values from natural language instructions (e.g., “tomorrow”, “on Friday”, “in 15 minutes”). The evaluation accepts equivalent CRON encodings provided they are semantically identical, focusing on whether the resulting schedule is interpreted correctly rather than on one fixed syntactic representation. In addition, the evaluation covers multi-turn dialogue sequences that operate on the same reminder. Typical scenarios include creating a reminder, listing it, deleting it, recreating it, and subsequently issuing a correction. These sequences test

whether the system maintains a consistent internal state across turns and whether references are correctly resolved to the intended reminder.

A representative case involves a recurring reminder that combines both time-of-day and recurrence constraints. For example, the instruction “*Remind me to exercise at 6:00 on Mondays, Wednesdays and Fridays*” must be converted into a reminder specification with the correct *date_time* and *frequency* arguments. The evaluation then checks whether these arguments, once processed by the scheduling tool, yield a CRON expression and execution times that faithfully capture the schedule intended by the user, as outlined in Table 8.

Table 8: Reminder definition test case

User Input	Remind me to exercise at 6 a.m. on Mondays, Wednesdays and Fridays.
Evaluation Clock	2025-02-20 12:00:00
Expected Schedule	CRON: 0 6 * * 1,3,5
Next Execution	2025-02-21 06:00:00

5.1.3 Domain Classification and Context Continuity

Tests confirm that each user message is correctly assigned to its domain agent and that multi-turn continuity is preserved across domain boundaries. This ensures that the orchestration layer’s classification and contextual resolution mechanisms function reliably.

Continuity across domains is validated through scenarios that require sequential involvement of multiple agents. For instance, when a user first requests to open the living room windows and then asks to be reminded to close them ten minutes later, the system must preserve referential continuity across utterances. Evaluation criteria include correct routing to both the Device and Reminder agents, and maintenance of context linking “the windows” across turns. A specific test case for this scenario is presented in Table 9.

Table 9: Domain continuity test case

Conversation Flow	1. Open the living room windows. 2. Remind me to close them in 10 minutes.
Expected Agents	1. Device, 2. Reminder
Continuity Requirement	“Windows” is correctly understood as the same entity in both turns

5.1.4 Weather Query Interpretation

Tests verify that the Weather Agent extracts the correct city from the user’s utterance and selects the appropriate forecast granularity (weekly overview vs. single day) based on the temporal expression (e.g., “*this week*” vs. “*on Friday*”). Using simulated tools that preserve real interfaces, assertions compare the captured call (tool name and parameters) against the expected city and granularity.

In the weather domain, evaluation focuses on resolving spatial and temporal expressions from user queries. A scenario involving a request for the weather in Porto, as defined in Table 10, requires the assistant to identify the city, disambiguate the country, and select the correct forecast granularity. Success is measured by accurate location extraction and selection of the appropriate temporal resolution.

Table 10: Representative weather test definition

User Input	Tell me what the weather will be like in Porto.
Expected City	Porto, Portugal
Expected Forecast	Daily (as opposed to weekly)

5.1.5 Memory Fact Extraction and Retrieval

The Memory Agent is evaluated on its ability to curate and deduplicate salient user facts from natural language input while ignoring casual conversation. After each interaction, the stored memories are compared with an expected set, checking if the number of actual entries matches the expected and that their content also matches. String verification employs semantic similarity matching using a threshold-based approach rather than exact string comparison, allowing for natural language variations while ensuring core information preservation. Scenarios also cover repeated facts, which must collapse into a single entry, as well as corrections and deletions. In addition, domain detection is validated by checking that memory-like utterances are consistently routed to the Memory domain.

A representative case is the statement “*I prefer to work in the morning.*” which should be persisted as a user preference. The evaluation checks that this information is stored once, in a standardized form that expresses the fact in the third person, captures only the core preference, is concise, and preserves the original semantic meaning.

5.1.6 European Portuguese Language Quality

Since the system is designed for European Portuguese, specific tests were implemented to ensure that responses consistently use this variety rather than defaulting to Brazilian Portuguese. To enable automatic evaluation, the approach of *LLM-as-a-Judge*, introduced by Zheng et al. (2023), was adopted. In this method, a language model is prompted with explicit evaluation criteria and instructed to output structured labels that indicate whether the tested text satisfies those criteria. All tests across other dimensions

of the system produce user-visible text (agent replies and persisted memories), which are reused and automatically checked for linguistic quality. The evaluation verifies not only that the responses are written in European Portuguese, but also that the grammar is correct and diacritics are used appropriately.

5.1.7 Speech-to-Text Recognition Accuracy

To evaluate spoken inputs under realistic conditions, the test suite used audio samples captured directly from the smart speaker prototype’s microphone, ensuring that the evaluation reflects the acoustic characteristics and environmental noise of the actual deployment hardware. The suite computes both Word Error Rate (WER) and Character Error Rate (CER) by comparing model outputs with the corresponding reference transcripts of predefined audio samples.

This evaluation was applied during the selection process of the STT model, as detailed in Section 4.5.4, where testing across 50 Portuguese voice commands recorded from five different speakers yielded an overall WER of 12.90% and CER of 5.17%. The results revealed specific challenges in transcribing hyphenated pronouns and distinguishing similar-sounding words, but these types of errors preserved sufficient semantic content for the downstream LLM-based reasoning components to interpret user intent correctly, confirming the model’s suitability for the assistant’s requirements.

5.2 Efficiency Evaluation

While functional evaluation demonstrates that the assistant produces correct and linguistically adequate outputs, it does not address how efficiently the system delivers those outputs to the user. From a user experience perspective, responsiveness is just as critical as correctness, since even a flawless answer loses value if it arrives too slowly. This section therefore evaluates the system along efficiency-oriented dimensions, focusing on responsiveness metrics such as Time to First Token (TTFT), Time to First Audio (TTFA), and the computational cost of large language model inference. Performance metrics in this section were obtained on a dedicated inference server equipped with an NVIDIA H200 140 GB GPU, an Intel Xeon Platinum 8468V CPU, and 258 GB of RAM, running *Ubuntu 22.04 LTS* with *vLLM v0.10.2* and *CUDA 12.4*.

5.2.1 Methodology and Key Metrics

Measuring request latency in *Clarisse* is challenging because the time required to handle a query can vary significantly even within the same domain. The reasoning loop inside an agent may take more or fewer steps depending on the complexity of the task, and some requests trigger additional tool calls while others do not. This variability stems from the fact that inputs are expressed in natural language, which allows infinite forms and levels of complexity, and is further amplified by the non-deterministic nature of Large Language Models. Because of this variability, latency results should be interpreted with caution, as they depend on the specific input and task. What is more informative is to analyze how time is distributed

across the pipeline and to focus on the moment the user first perceives a response, namely the Time to First Token (TTFT) and, more importantly for voice interactions, the Time to First Audio (TTFA). While TTFT reflects when the system begins generating text, TTFA corresponds to the onset of audible speech synthesis.

5.2.2 Pipeline Latency Analysis

To better understand where time is spent within the interaction pipeline, I averaged the results of 20 representative requests drawn from the speech-to-text functional evaluation tests and visualized their execution using a Gantt chart (Figure 27). The chart highlights the sequential contribution of each stage: speech-to-text (STT), agent reasoning via the large language model (LLM), and text-to-speech (TTS). Two perceptual markers are emphasized: the Time to First Token (TTFT), which indicates when the system begins generating text, and the Time to First Audio (TTFA), which marks the onset of audible speech synthesis and therefore the moment the user first hears the response.

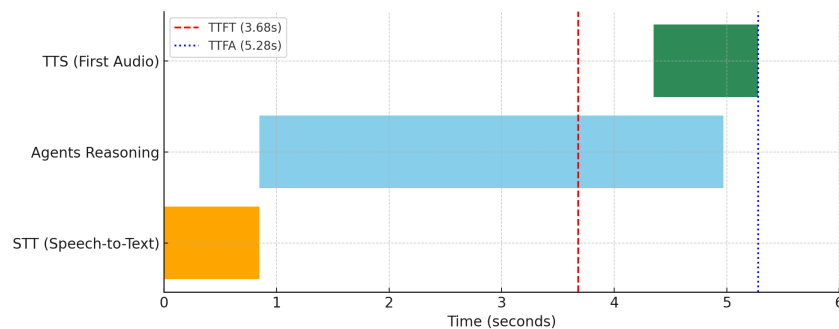


Figure 27: Average latency breakdown across pipeline stages

An important aspect of the pipeline is that speech synthesis does not always need to wait for the full completion of text generation. Instead, synthesis can begin as soon as the reasoning component produces a complete sentence that can be preemptively spoken. This is the reason why the TTS processing starts after the TTFT marker and before the reasoning stage fully concludes generating the text response. This overlap is crucial for minimizing TTFA, as it allows the system to deliver audible feedback more quickly.

Although the number of sampled requests is limited and the results cannot be generalized to every possible system interaction, given the variability previously discussed, the chart still provides valuable insight into which stages dominate processing time. It shows that the agent's reasoning stage is by far the most time-consuming part of the pipeline. Within this stage, the contribution of tool calls is negligible, averaging around 0.3 seconds per request-response exchange, meaning that the bulk of the latency stems from LLM inference itself.

5.2.3 Load and Concurrency Benchmarking

The pipeline latency analysis in section 5.2.2 identified the agent reasoning stage, which is dominated by LLM inference, as the primary contributor to overall response time. To gain a more granular understanding of this critical component's behavior, a targeted benchmark was conducted to evaluate the performance of the selected LLM (*Llama-3.3-70B-Instruct* in an 8-bit quantized format) under a simulated, dynamic load. The evaluation utilized the `vllm bench serve` utility to simulate a realistic usage scenario. The benchmark was configured to send 1,000 prompts with a fixed input length of 1,024 tokens and a required output of 128 tokens, mirroring a moderately complex interaction. To simulate increasing user traffic, a linear ramp-up strategy was employed, gradually increasing the request rate from 2 to 8 requests per second. The full configuration used for the benchmark is provided in Appendix C.3.

The following key metrics were measured:

- **Time to First Token (TTFT):** The latency from when a request is sent until the first output token is generated. This metric is crucial for the perceived initial responsiveness of the assistant.
- **Time per Output Token (TPOT):** The average time taken to generate each subsequent token after the first. This indicates the “speed” at which the assistant formulates its response and directly impacts the fluidity of streaming text-to-speech.

These metrics were captured at the 50th (median), 90th, 95th, and 99th percentiles to understand both the typical and worst-case performance under load. The benchmark yielded the following key performance indicators, as shown in Table 11.

Metric	Mean	Median	P90	P95	P99
TTFT (ms)	250.96	194.14	467.83	482.92	753.38
TPOT (ms)	41.37	42.59	44.96	45.78	55.15

Throughput Metric	Value
Request Throughput	2.94 req/s
Output Token Throughput	337.60 tok/s

Table 11: LLM benchmark performance metrics

These results provide important insights into the LLM's performance characteristics, particularly regarding responsiveness and generation consistency under load. The median TTFT of 194 ms demonstrates strong typical responsiveness, but the 99th percentile climbs to 753 ms under peak load, indicating that the prefill stage is highly sensitive to concurrency and traffic surges. In contrast, TPOT remains highly stable, with a median of 42.6 ms and a worst-case of just 55.1 ms. This consistency ensures smooth and natural text-to-speech streaming once generation begins, mitigating risks of choppy output.

In terms of overall capacity, the system demonstrated its robustness by sustaining an average request throughput of 2.94 req/s and an output of over 337 tokens/s throughout the ramp-up test. This level of performance is sufficient for the project's target environment, showing that the quantized 70B model, when served with *vLLM*, can effectively handle a moderate number of concurrent users.

5.2.4 Results Discussion

According to Amdahl's Law, the greatest potential for performance improvement lies in optimizing the component that dominates execution time. As established in the pipeline analysis, the reasoning stage of the LLM is the critical bottleneck, meaning that advances in model efficiency are the most impactful lever for system-level optimization. To explore this further, the load benchmark establishes a practical baseline showing the system sustains a moderate concurrent load, and the highly stable TPOT under pressure indicates that the generation process scales gracefully.

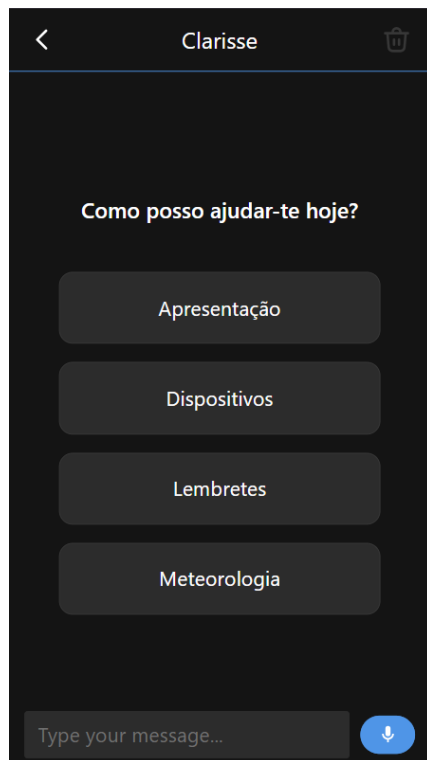
Performance improves with additional computational resources, so deployment on more capable hardware provides immediate latency reductions. As core components and infrastructure advance, these improvements translate directly into better end-to-end performance and responsiveness in production settings. The modular architecture further supports sustained evolution by enabling independent upgrades of the LLM, STT, and TTS as more efficient models become available.

With respect to user-perceived responsiveness, voice interactions did not consistently meet the five-second initiation target, with TTFA slightly exceeding the threshold in some cases. The overrun was very short, and the text interface provided earlier TTFT, improving perceived responsiveness despite the added voice path latency. Together, these findings support the suitability of the current architecture for streaming interactions while clarifying that end-to-end latency is mainly driven by the LLM's reasoning stage.

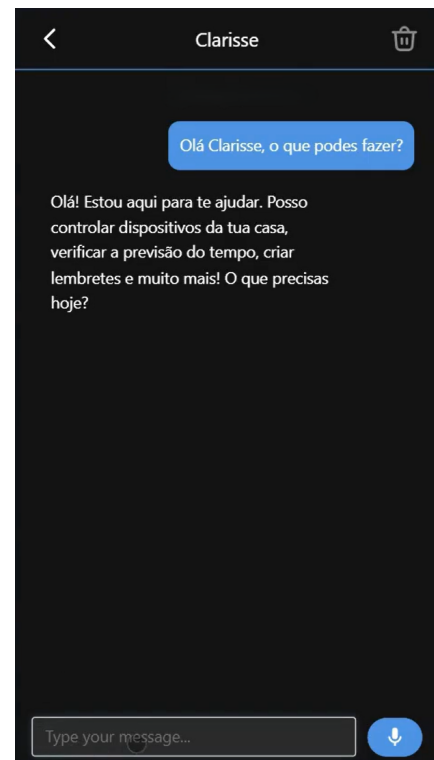
5.3 Frontend Demonstration

While the functional and efficiency evaluations presented above establish the system's correctness and responsiveness, they do not fully capture how these capabilities appear to end users. Since the voice interface is not easily represented in a static document, the mobile application frontend provides the clearest way to illustrate the assistant in action. This section therefore presents representative screenshots that complement the evaluation by showing how tested functionalities manifest in practice.

Figure 28 shows the interface during the very first interaction with the assistant. The interface displays prompt suggestions for common domains, and the user initiates the conversation by asking about the assistant's capabilities. In response, *Clarisse* lists its main functions, such as controlling smart devices, checking the weather, and creating reminders.



(a) Initial screen with prompt suggestions



(b) Assistant's self-reported capabilities

Figure 28: Initial interaction with the assistant

As verified in the device control tests, the assistant can discover available devices and execute commands on them. Figure 29 illustrates a two-turn sequence where the user first queries the available devices and then issues a command. The assistant maintains context between the turns and operates the correct device.

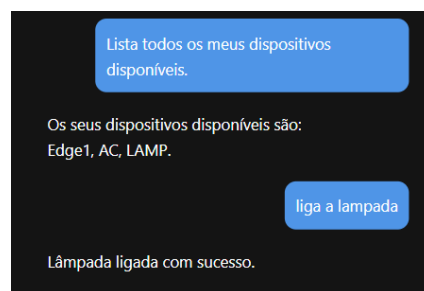
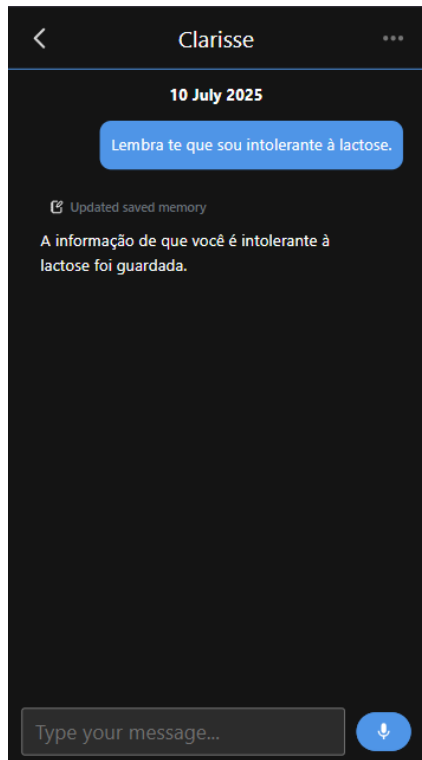
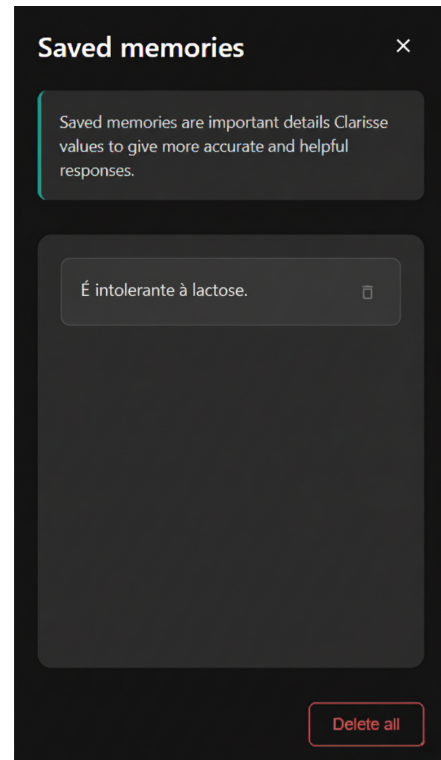


Figure 29: Device control through the assistant

To demonstrate the system's ability to manage memories, Figure 30 shows a conversation where the user establishes a preference through natural dialogue. The assistant acknowledges that this information has been stored for future use, ensuring transparency about its operation. To further support user awareness and control, the memory page provides an overview of all saved facts and allows the user to delete any entry if desired.



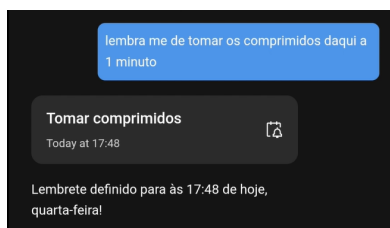
(a) User specifies lactose-free preference in chat



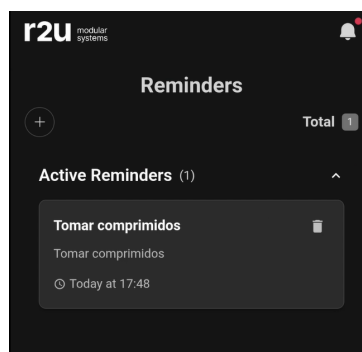
(b) Memory page showing stored preferences

Figure 30: Storing and managing user memories

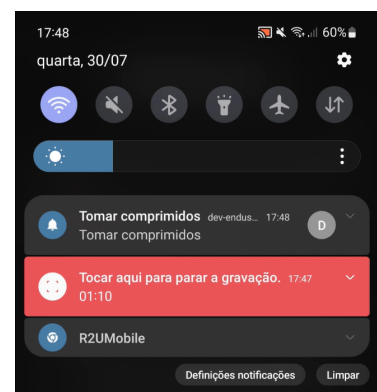
The reminder functionality was tested extensively for both relative and absolute time expressions. Figure 31 demonstrates the full workflow in the application, from creating the reminder in conversation to its display in the reminders page and final notification on the device. As with other domains, the conversational response is enriched with a widget generated from structured metadata, allowing the frontend to present the reminder in a clear and visually accessible way alongside the textual confirmation.



(a) Chat interface



(b) Reminders



(c) Phone notifications

Figure 31: Reminder creation and notification

Finally, Figure 32 shows the assistant handling weather queries where the response is not only textual but also accompanied by a dynamically rendered widget. While the Weather Agent processes the request,

it queries the API for the data and then the system uses the result to send the forecast values, location, and time information in a predefined schema. The frontend then parses this schema to render the information as a visual card, giving the user both a clear textual explanation and a visually structured forecast.

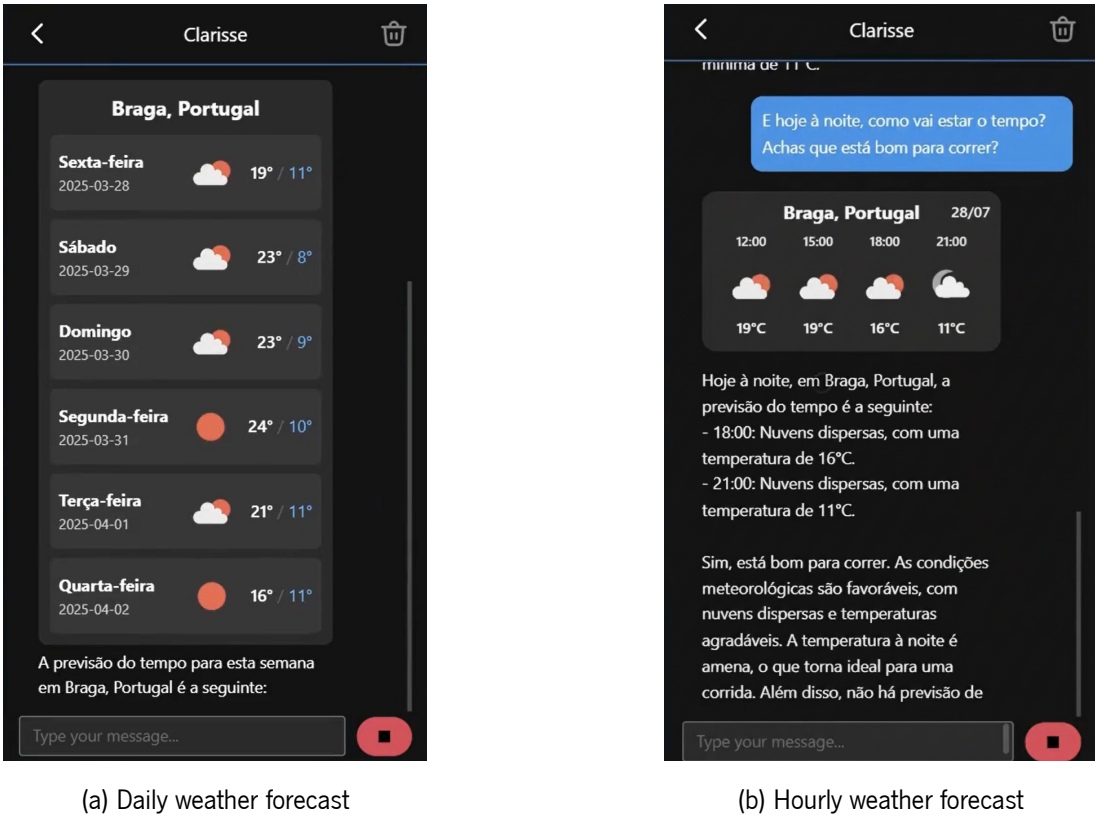


Figure 32: Weather forecasts with textual and visual responses

In summary, the evaluation confirmed that the assistant behaves reliably across its functional domains and provides responses with latency levels that, while occasionally exceeding strict thresholds in voice interactions, remain within an acceptable range for usability. The visual demonstrations complement these results by showing the assistant in action across its core capabilities. Taken together, the functional and efficiency evaluations provide a balanced view of the system’s strengths and limitations, and they establish a solid foundation for further optimization and iteration.

Chapter 6

Conclusions and Future Work

This chapter summarizes the main results of the dissertation, highlighting how the proposed objectives were accomplished and what contributions were made. It also discusses directions for future work that could further extend and refine the system, and closes with a final appreciation of the overall impact and significance of the research.

6.1 Synopsis of the Work Done

This dissertation set out to explore how recent advances in open-source large language models and speech technologies could be harnessed to develop an adaptable and accessible virtual assistant, with a particular focus on controlling IoT environments through natural voice interaction in European Portuguese. The motivation behind this work was to investigate how open-source approaches can support transparent, customizable, and language-inclusive systems, demonstrating their potential as a viable and compelling path for the development of intelligent assistants.

To achieve the dissertation’s goal, the work addressed the objectives defined in Section 1.4. Objective o1 was fulfilled by surveying open models and selecting *Llama-3.3-70B-Instruct* as the core model because it provided strong instruction following and tool use while still being deployable on available hardware. Objective o2 was achieved by creating a modular multi-agent architecture in which an orchestrator coordinated specialized agents for devices, reminders, weather, and memory, each with its own tool schemas and strategies that supported accurate and reliable operation. Objectives o3 and o4 were achieved by first performing a functional decomposition of the problem into agents that covered the core capabilities, namely device control, reminders, weather queries, memory, and conversational context, with an orchestrator coordinating tool schemas and interactions. These capabilities were enabled by serving the chosen model with *vLLM* and by developing a Python *FastAPI* service that received requests and applied the multi-agent logic to generate responses. Objective o5 was addressed by selecting a fine-tuned *Whisper* variant for Portuguese speech recognition and by adopting *Piper* for speech synthesis, which not only produced good results but also made it possible to train models on custom voices, all in European Portuguese. Objective o6 was fulfilled through the creation of a lightweight Python program running on a Raspberry Pi with a speaker and a microphone, which served as a voice-only interface and included a custom wake word

model trained with the *openWakeWord* framework on the phrase “*Hey Clarisse*”, together with *Silero-VAD* for voice activity detection. Finally, Objective o7 was met by systematically testing the complete system with a *PyTest* suite that covered device control, reminders, weather interpretation, memory usage, and European Portuguese interactions, which helped manage variability from context and prompt engineering during agent definition and ensured the overall reliability of the assistant.

Overall, the results confirmed that the system was capable of handling device control operations, managing reminders, retrieving weather information, and maintaining long-term memory, thereby validating the proposed approach and showing that open-source solutions can rival proprietary assistants in functionality and adaptability.

6.2 Future Work

Several aspects of the system could be improved to make it more complete and suitable for everyday use.

First, the mobile client could be enhanced through deeper integration with the operating system. Allowing the assistant to be registered as the default system assistant, as is possible with *Siri* or *Google Assistant*, would enable native background listening and hands-free activation. This would extend the current touch-to-speak model into a more seamless interaction.

Second, although designed for European Portuguese, the architecture could support other languages. Adding language selection, adapting prompts, and integrating suitable speech-to-text and text-to-speech models, either multilingual or language-specific, would make the system more versatile and accessible to a broader audience.

Third, the rapid evolution of large language models and speech technologies creates opportunities for improvement. The modular design allows new components to be integrated, but replacing an LLM is rarely a direct upgrade because prompt design and context handling often require re-tuning. Future work could refine these elements to take advantage of newer models, such as those using mixture-of-experts architectures, which promise faster responses while maintaining output quality.

Finally, multimodal capabilities represent a valuable extension. Allowing the assistant to process and reason over visual input, such as images or camera streams, would enable richer and more context-sensitive interactions, from object recognition to visual question answering.

6.3 Final Appreciation

This dissertation has demonstrated the feasibility of developing a transparent, extensible, and language-inclusive assistant system based on multi-agent orchestration, validated through the successful integration of open-source large language models and speech technologies into a complete European Portuguese voice pipeline. The contributions extend both academic understanding and practical applications by showing

how open approaches can provide high-quality interaction capabilities in less-resourced languages. By reducing reliance on proprietary platforms and emphasizing modularity, this work establishes a flexible foundation for future integrations and broader deployment in real-world contexts.

References

- Aggar, C., Sorwar, G., Seton, C., Penman, O., and Ward, A. (2022). Smart home technology to support older people's quality of life: A longitudinal pilot study. *International Journal of Older People Nursing*, 18(1). DOI [10.1111/opn.12489](https://doi.org/10.1111/opn.12489).
- Al-Safi, H., Ibrahim, H., and Steenson, P. (2025). Vega: Llm-driven intelligent chatbot platform for internet of things control and development. *Sensors*, 25(12). DOI [10.3390/s25123809](https://doi.org/10.3390/s25123809).
- Alsafari, B., Atwell, E., Walker, A., and Callaghan, M. (2024). Towards effective teaching assistants: From intent-based chatbots to LLM-powered teaching assistants. *Natural Language Processing Journal*, 8:100101. DOI [10.1016/j.nlp.2024.100101](https://doi.org/10.1016/j.nlp.2024.100101).
- Anik, S. M. H., Gao, X., Zhong, H., Wang, X., and Meng, N. (2024). Automation configuration in smart home systems: Challenges and opportunities. DOI [10.48550/arXiv.2408.04755](https://doi.org/10.48550/arXiv.2408.04755).
- Bolton, T., Dargahi, T., Belguith, S., Al-Rakhami, M. S., and Sodhro, A. H. (2021). On the Security and Privacy Challenges of Virtual Assistants. *Sensors*, 21(7):2312. DOI [10.3390/s21072312](https://doi.org/10.3390/s21072312).
- Brachten, F., Brünker, F., Frick, N. R. J., Ross, B., and Stieglitz, S. (2020). On the ability of virtual agents to decrease cognitive load: an experimental study. *Information Systems and e-Business Management*, 18(2):187–207. DOI [10.1007/s10257-020-00471-7](https://doi.org/10.1007/s10257-020-00471-7).
- Cheng, Y., Chen, J., Huang, Q., Xing, Z., Xu, X., and Lu, Q. (2023). Prompt Sapper: a LLM-Empowered production tool for building AI chains. *ACM Transactions on Software Engineering and Methodology*, 33(5):1–24. DOI [10.1145/3638247](https://doi.org/10.1145/3638247).
- Cleland, J., Hutchinson, C., Williams, P. A. H., Manuel, K., and Laver, K. (2023). A scoping review to explore the health, social and economic outcomes of home automation for people with disability. *Disability and Rehabilitation Assistive Technology*, 19(4):1446–1453. DOI [10.1080/17483107.2023.2196308](https://doi.org/10.1080/17483107.2023.2196308).
- Cuadrado, J. S., Pérez-Soler, S., Guerra, E., and De Lara, J. (2024). Automating the Development of Task-oriented LLM-based Chatbots. *ACM Conversational User Interfaces 2024*, pages 1–10. DOI [10.1145/3640794.3665538](https://doi.org/10.1145/3640794.3665538).
- Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and R , C. (2022). Flashattention: Fast and memory-efficient exact attention with io-awareness. DOI [10.48550/arXiv.2205.14135](https://doi.org/10.48550/arXiv.2205.14135).
- De Blanes Sebasti n, M. G., Guede, J. R. S., and Antonovica, A. (2022). Application and extension of the UTAUT2 model for determining behavioral intention factors in use of the artificial intelligence virtual assistants. *Frontiers in Psychology*, 13. DOI [10.3389/fpsyg.2022.993935](https://doi.org/10.3389/fpsyg.2022.993935).
- Dorri, A., Kanhere, S. S., and Jurdak, R. (2018). Multi-Agent Systems: a survey. *IEEE Access*, 6:28573–

28593. DOI 10.1109/access.2018.2831228.

- D'urso, S., Fontana, M., Martini, B., and Sciarrone, F. (2024). Enhancing Intent Acquisition and Translation with Large Language Models and Intelligent Chatbots: A DHCP Use Case. *2024 IEEE 10th International Conference on Network Softwarization*, pages 19–24. DOI 10.1109/netsoft60951.2024.10588897.
- Enevoldsen, K., Chung, I., Kerboua, I., et al. (2025). Mmteb: Massive multilingual text embedding benchmark. DOI 10.48550/arXiv.2502.13595.
- Frantar, E., Ashkboos, S., Hoefler, T., and Alistarh, D. (2023). Gptq: Accurate post-training quantization for generative pre-trained transformers. DOI 10.48550/arXiv.2210.17323.
- Guan, Y., Wang, D., Chu, Z., Wang, S., Ni, F., Song, R., and Zhuang, C. (2024). Intelligent Agents with LLM-based Process Automation. In *30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 5018–5027, Barcelona Spain. ACM. DOI 10.1145/3637528.3671646.
- He, J., Treude, C., and Lo, D. (2025). LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision and the Road Ahead. DOI 10.48550/arXiv.2404.04834.
- Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. (2020). Scaling laws for neural language models.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C., Gonzalez, J., Zhang, H., and Stoica, I. (2023a). vLLM: Easy, Fast, and Cheap LLM Serving with PagedAttention.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., and Stoica, I. (2023b). Efficient Memory Management for Large Language Model Serving with PagedAttention. *ACM SIGOPS 29th Symposium on Operating Systems Principles*.
- Leviathan, Y., Kalman, M., and Matias, Y. (2023). Fast inference from transformers via speculative decoding. DOI 10.48550/arXiv.2211.17192.
- Li, B., Beaton, D., Lee, D. S., Aljabri, B., Al-Omran, L., Wijesundera, D. N., Hussain, M. A., Rotstein, O. D., De Mestral, C., Mamdani, M., and Al-Omran, M. (2024). Comprehensive review of virtual assistants in vascular surgery. *Seminars in Vascular Surgery*, 37(3):342–349. DOI 10.1053/j.semvasc.2024.07.001.
- Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P. (2023). Lost in the Middle: How language models use long contexts. DOI 10.48550/arXiv.2307.03172.
- Luo, B., Lau, R. Y. K., Li, C., and Si, Y. (2021). A critical review of state-of-the-art chatbot designs and applications. *Wiley Interdisciplinary Reviews Data Mining and Knowledge Discovery*, 12(1). DOI 10.1002/widm.1434.
- Manchanda, J., Boettcher, L., Westphalen, M., and Jasser, J. (2024). The open source advantage in large language models (llms). DOI 10.48550/arXiv.2412.12004.
- Masina, F., Orso, V., Pluchino, P., Dainese, G., Volpato, S., Nelini, C., Mapelli, D., Spagnolli, A., and Gamberini, L. (2020). Investigating the Accessibility of voice assistants with Impaired users: Mixed Methods study. *Journal of Medical Internet Research*, 22(9):e18431. DOI 10.2196/18431.
- Mei, L., Yao, J., Ge, Y., Wang, Y., Bi, B., Cai, Y., Liu, J., Li, M., Li, Z.-Z., Zhang, D., Zhou, C., Mao, J., Xia, T., Guo, J., and Liu, S. (2025). A survey of context engineering for large language models. DOI 10.48550/arXiv.2507.13334.

- Nuruzzaman, M. and Hussain, O. K. (2018). A Survey on Chatbot Implementation in Customer Service Industry through Deep Neural Networks. *IEEE 15th International Conference on e-Business Engineering (ICEBE)*, pages 54–61. DOI [10.1109/icebe.2018.00019](https://doi.org/10.1109/icebe.2018.00019).
- Pal, D., Arpnikanondt, C., Funilkul, S., and Razzaque, M. A. (2020). Analyzing the adoption and diffusion of voice-enabled smart-home systems: empirical evidence from Thailand. *Universal Access in the Information Society*, 20(4):797–815. DOI [10.1007/s10209-020-00754-3](https://doi.org/10.1007/s10209-020-00754-3).
- Park, S., Jeon, S., Lee, C., Jeon, S., Kim, B.-S., and Lee, J. (2025). A Survey on Inference Engines for Large Language Models: Perspectives on Optimization and Efficiency. DOI [10.48550/arXiv.2505.01658](https://doi.org/10.48550/arXiv.2505.01658).
- Perez-Soler, S., Juarez-Puerta, S., Guerra, E., and De Lara, J. (2021). Choosing a Chatbot development tool. *IEEE Software*, 38(4):94–103. DOI [10.1109/ms.2020.3030198](https://doi.org/10.1109/ms.2020.3030198).
- Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., and Sutskever, I. (2022). Robust speech recognition via Large-Scale Weak Supervision. DOI [10.48550/arXiv.2212.04356](https://doi.org/10.48550/arXiv.2212.04356).
- Radziwill, N. M. and Benton, M. C. (2017). Evaluating quality of chatbots and intelligent conversational agents. DOI [10.48550/arXiv.1704.04579](https://doi.org/10.48550/arXiv.1704.04579).
- Rastogi, A., Zang, X., Sunkara, S., Gupta, R., and Khaitan, P. (2020). Towards scalable multi-domain conversational agents: The schema-guided dialogue dataset. In *AAAI conference on artificial intelligence*, volume 34, pages 8689–8696. DOI [10.1609/aaai.v34i05.6310](https://doi.org/10.1609/aaai.v34i05.6310).
- Sanh, V., Debut, L., Chaumond, J., and Wolf, T. (2020). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. DOI [10.48550/arXiv.1910.01108](https://doi.org/10.48550/arXiv.1910.01108).
- Shortliffe, E. H., Davis, R., Axline, S. G., Buchanan, B. G., Green, C., and Cohen, S. N. (1975). Computer-based consultations in clinical therapeutics: Explanation and rule acquisition capabilities of the MYCIN system. *Computers and Biomedical Research*, 8(4):303–320. DOI [10.1016/0010-4809\(75\)90009-9](https://doi.org/10.1016/0010-4809(75)90009-9).
- Shu, R., Das, N., Yuan, M., Sunkara, M., and Zhang, Y. (2024). Towards Effective GenAI Multi-Agent Collaboration: Design and Evaluation for Enterprise Applications. DOI [10.48550/arXiv.2412.05449](https://doi.org/10.48550/arXiv.2412.05449).
- Siebra, C., Correia, W., Penha, M., Macedo, J., Quintino, J., Anjos, M., Florentin, F., Da Silva, F. Q. B., and Santos, A. L. M. (2018). Virtual assistants for mobile interaction. *Australasian Computer-Human Interaction Conference*, pages 568–571. DOI [10.1145/3292147.3292232](https://doi.org/10.1145/3292147.3292232).
- Sumers, T. R., Yao, S., Narasimhan, K., and Griffiths, T. L. (2024). Cognitive architectures for language agents. DOI [10.48550/arXiv.2309.02427](https://doi.org/10.48550/arXiv.2309.02427).
- Sun, C., Huang, S., and Pompili, D. (2025). LLM-Based Multi-Agent Decision-Making: Challenges and Future Directions. *IEEE Robotics and Automation Letters*, 10(6):5681–5688. DOI [10.1109/LRA.2025.3562371](https://doi.org/10.1109/LRA.2025.3562371).
- Topsakal, O. and Akinci, T. C. (2023). Creating large language model applications Utilizing LangChain: A primer on developing LLM apps fast. *International Conference on Applied Engineering and Natural Sciences*, 1(1):1050–1056. DOI [10.59287/icaens.1127](https://doi.org/10.59287/icaens.1127).
- Tsyra, O., Punchenko, N., and Frazee-Frazenko, O. (2020). FEATURES OF CONSTRUCTION AND BASIC DIRECTIONS OF DEVELOPMENT OF VIRTUAL DIGITAL ASSISTANTS. *Cybersecurity Education Science Technique*, 1(9):140–148. DOI [10.28925/2663-4023.2020.9.140148](https://doi.org/10.28925/2663-4023.2020.9.140148).

- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2023). Attention is all you need. DOI 10.48550/arXiv.1706.03762.
- Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z., and Wen, J.-R. (2024). A Survey on Large Language Model based Autonomous Agents. *Frontiers of Computer Science*, 18(6). DOI 10.1007/s11704-024-40231-1.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., and Zhou, D. (2022). Chain-of-Thought prompting elicits reasoning in large language models. DOI 10.48550/arXiv.2201.11903.
- Weizenbaum, J. (1966). ELIZA—a computer program for the study of natural language communication between man and machine. *Communications of the ACM*, 9(1):36–45. DOI 10.1145/365153.365168.
- Wewerka, J. and Reichert, M. (2020). Robotic process automation—a systematic literature review and assessment framework. DOI 10.48550/arXiv.2012.11951.
- Wooldridge, M. (2009). *An introduction to MultiAgent systems*. John Wiley & Sons, 2nd edition.
- Xu, J., Wang, Q., Cao, Y., Zeng, B., and Liu, S. (2024). A General-Purpose Device for Interaction with LLMs. DOI 10.48550/arXiv.2408.10230.
- Yadav, H. and Sitaram, S. (2022). A Survey of Multilingual Models for Automatic Speech Recognition. DOI 10.48550/arXiv.2202.12576.
- Yang, Z., Xu, X., Yao, B., Rogers, E., Zhang, S., Intille, S., Shara, N., Gao, G. G., and Wang, D. (2024). Talk2Care: An LLM-based Voice Assistant for Communication between Healthcare Providers and Older Adults. *ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, 8(2):1–35. DOI 10.1145/3659625.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. (2022). REACT: Synergizing reasoning and acting in language models. DOI 10.48550/arXiv.2210.03629.
- Zafir, O., Boudoukh, G., Izsak, P., and Wasserblat, M. (2019). Q8BERT: Quantized 8Bit BERT. In *2019 Fifth Workshop on Energy Efficient Machine Learning and Cognitive Computing - NeurIPS Edition (EMC2-NIPS)*. DOI 10.1109/EMC2-NIPS53020.2019.00016.
- Zhang, Z., Bo, X., Ma, C., Li, R., Chen, X., Dai, Q., Zhu, J., Dong, Z., and Wen, J.-R. (2024). A Survey on the Memory Mechanism of Large Language Model based Agents. DOI 10.48550/arXiv.2404.13501.
- Zhao, W. X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., Min, Y., Zhang, B., Zhang, J., Dong, Z., Du, Y., Yang, C., Chen, Y., Chen, Z., Jiang, J., Ren, R., Li, Y., Tang, X., Liu, Z., Liu, P., Nie, J.-Y., and Wen, J.-R. (2023). A survey of large language models. *arXiv (Cornell University)*. DOI 10.48550/arxiv.2303.18223.
- Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., and Stoica, I. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. DOI 10.48550/arXiv.2306.05685.
- Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C., and Sheng, Y. (2024). SGLang: Efficient Execution of Structured Language Model Programs. DOI 10.48550/arXiv.2312.07104.
- Zimmet, J. (2020). Optimal discovery? Siri, Alexa, and other virtual personal assistants in libraries. *Public Access Services Quarterly*, 16(1):41–47. DOI 10.1080/15228959.2019.1693318.

Appendix A

Details of Results

A.1 Wake Word Training Combinations and Metrics

Table 12: Ranked training combinations with hyperparameters and performance metrics

Rank	N_samples (k)	Steps (k)	Accuracy	Recall	False Positives /h	Score
1	40	30	0.759	0.518	0.000	0.639
2	40	35	0.774	0.547	0.177	0.625
3	40	20	0.743	0.486	0.000	0.615
4	40	25	0.742	0.483	0.000	0.613
5	40	15	0.737	0.474	0.000	0.606
6	35	15	0.718	0.435	0.000	0.577
7	30	15	0.715	0.430	0.000	0.573
8	30	50	0.712	0.425	0.000	0.569
9	30	40	0.711	0.423	0.000	0.567
10	35	25	0.711	0.424	0.000	0.568
11	35	50	0.746	0.493	0.265	0.566
12	35	35	0.737	0.474	0.266	0.552
13	45	35	0.714	0.428	0.177	0.536
14	45	40	0.702	0.405	0.088	0.536
15	45	30	0.690	0.379	0.000	0.535
16	30	35	0.681	0.362	0.000	0.522
17	35	40	0.735	0.470	0.442	0.514
18	50	35	0.698	0.395	0.177	0.511
19	50	25	0.673	0.345	0.000	0.509
20	40	40	0.662	0.324	0.000	0.493
21	50	40	0.662	0.324	0.088	0.475
22	40	50	0.620	0.240	0.000	0.430
23	50	30	0.753	0.507	1.062	0.418
24	45	50	0.746	0.492	1.062	0.407

The *Score* in Table 12 is a composite metric balancing detection performance against false activations:

$$\text{Score} = \frac{\text{Accuracy} + \text{Recall}}{2} - 0.2 \times \text{FPH}.$$

Appendix B

Listings

B.1 Test Memories and Retrieval Queries

This appendix lists the **memories** and **queries** used in the retrieval tests for evaluating embedding models. The memories and queries are in Portuguese, reflecting the target language of the application.

B.1.1 Test Memories (26 total)

- É alérgico a amendoim.
- É vegetariano.
- Gosta de cozinhar pratos italianos.
- Gosta de cozinhar pratos asiáticos.
- Prefere café a chá.
- Tem um gato chamado Max.
- É um amante de animais.
- Gosta de viajar.
- Gosta de fazer caminhadas na natureza.
- Pratica corrida regularmente.
- Gosta de praticar ioga.
- Gosta de fazer voluntariado.
- Gosta de tocar guitarra.
- Gosta de tocar piano.
- É fã de música clássica.
- Tem interesse em arte contemporânea.
- Estuda ciência da computação.
- Tem interesse em fotografia.
- Tem interesse em jardinagem.
- Tem interesse em história antiga.
- É apaixonado por astronomia.

- É um entusiasta de tecnologia.
- É fã de ficção científica.
- Gosta de assistir a documentários.
- Gosta de ler livros de mistério.
- É um fã de jogos de tabuleiro.

B.1.2 Test Queries (20 total)

- **Query:** "Olá, preciso de ajuda"
Expected: Any 3 results
Purpose: Validate basic pipeline functionality.
- **Query:** "Sugere uma música relaxante enquanto trabalho."
Expected: {Gosta de tocar guitarra.; É fã de música clássica.; Gosta de tocar piano.}
- **Query:** "Dá-me ideias de receitas para o jantar."
Expected: {É vegetariano.; Gosta de cozinhar pratos italianos.; Gosta de cozinhar pratos asiáticos.; É alérgico a amendoim.}
- **Query:** "Sugere um restaurante para jantar esta noite."
Expected: {É vegetariano.; É alérgico a amendoim.; Gosta de cozinhar pratos italianos.; Gosta de cozinhar pratos asiáticos.}
- **Query:** "Que tempo está previsto para amanhã de manhã?"
Expected: {Pratica corrida regularmente.; Gosta de fazer caminhadas na natureza.}
- **Query:** "Tenho alergias alimentares?"
Expected: {É alérgico a amendoim.; É vegetariano.}
- **Query:** "Mostra-me notícias ou artigos interessantes para mim."
Expected: {Gosta de assistir a documentários.; É um entusiasta de tecnologia.; É fã de ficção científica.; É um amante de animais.; Gosta de viajar.; É apaixonado por astronomia.; Tem interesse em história antiga.; Tem interesse em arte contemporânea.; Tem interesse em fotografia.; Gosta de ler livros de mistério.}
- **Query:** "Cria um lembrete para amanhã de manhã para eu não me esquecer de levar a marmita e dar comida ao gato."
Expected: {Tem um gato chamado Max.; É vegetariano.; É alérgico a amendoim.}
- **Query:** "Antes de começar a trabalhar, o que costumo preferir beber?"
Expected: {Prefere café a chá.}
- **Query:** "Sugere-me um destino e atividades que eu provavelmente iria gostar."
Expected: {Gosta de viajar.; Gosta de fazer caminhadas na natureza.}
- **Query:** "Recomenda uma rotina rápida para relaxar ao fim do dia."
Expected: {Gosta de praticar ioga.}

- **Query:** "Há iniciativas de voluntariado interessantes na minha área?"
Expected: {Gosta de fazer voluntariado.}
- **Query:** "Lembra-me de comprar comida adequada para o meu animal."
Expected: {Tem um gato chamado Max.; É um amante de animais.}
- **Query:** "Sugere atividades para uma noite com amigos em casa."
Expected: {É um fã de jogos de tabuleiro.}
- **Query:** "Que livro é provável eu gostar de ler a seguir?"
Expected: {Gosta de ler livros de mistério.; É fã de ficção científica.}
- **Query:** "Recomenda um documentário que eu possa gostar este fim de semana."
Expected: {Gosta de assistir a documentários.; Tem interesse em história antiga.; É um entusiasta de tecnologia.; É apaixonado por astronomia.}
- **Query:** "Dá-me dicas para tirar melhores fotos ao ar livre."
Expected: {Tem interesse em fotografia.; Gosta de fazer caminhadas na natureza.}
- **Query:** "Que conteúdos me podem ajudar a melhorar a minha jardinagem?"
Expected: {Tem interesse em jardinagem.}
- **Query:** "Há exposições que eu possa achar interessantes?"
Expected: {Tem interesse em arte contemporânea.}
- **Query:** "Que atividade relaxante posso fazer depois do trabalho?"
Expected: {Gosta de tocar guitarra.; Gosta de tocar piano.}

B.2 Example: Reminder Test Case Definition (JSON)

The reminder test suite is defined as a simple JSON file, where each object specifies a test case with the user's utterance, the expected periodic task(s), and the simulated current time. This makes it easy to add new tests or modify existing ones. Listing B.1 shows an example of a reminder test cases definition.

```
// File: tests/reminders/input_tests/test_reminders.json
[
  {
    "test_id": "1.01-reuniao_equipa",
    "conversation_flow": [
      "Lembra-me de participar na reunião de equipa, 15 de fevereiro de 2025 às 10h."
    ],
    "expected_periodic_task": [
      {
        "cron_expression": "0 10 15 2 *",
        "valid_until": "2025-02-15T10:00:30",
        "next_execution": "2025-02-15T10:00:00"
      }
    ],
    "current_time": "2025-02-13T09:00:00"
  }
]
```

```

},
{
  "test_id": "2.06-medicacao_manha_tarde",
  "conversation_flow": [
    "Lembra-me de tomar o medicamento todos os dias às 9h e às 21h"
  ],
  "expected_periodic_task": [
    {
      "cron_expression": "0 9 * * *",
      "next_execution": "2025-03-10T09:00:00"
    },
    {
      "cron_expression": "0 21 * * *",
      "next_execution": "2025-03-10T21:00:00"
    }
  ],
  "current_time": "2025-03-10T08:00:00"
}
// ... more cases ...
]

```

Listing B.1: Example test cases for reminder functionality validation

Each test case includes:

- `test_id`: A unique identifier for the test.
- `conversation_flow`: The user's utterance(s) in natural language.
- `expected_periodic_task`: The expected created reminders and their properties (e.g., cron expression, next execution).
- `current_time`: The simulated current time for the test.

This format allows new scenarios to be added by simply appending new JSON objects, making the test suite extensible and transparent.

Appendix C

Tooling

C.1 Agent Tool Specifications

This appendix provides detailed specifications for the tools available to each specialized agent in the system. Each entry includes the tool's signature, arguments, output schema, example outputs, and relevant design rationale.

C.1.1 Devices Agent Tools

- **get_user_devices** — Retrieves the inventory of devices grouped by location so the agent knows what exists and where. Behind the scenes the raw responses from the Devices API are pre-processed into simplified structures. Instead of exposing nested asset trees, which are hierarchical collections of locations such as building, floor and room, the tool computes the full location path from this structure and returns compact lists of devices per location that the agent can more effectively reason with. This transformation step is part of the context engineering applied to enhance the agent's performance and reliability.

Arguments: *None*

Output schema: a dictionary where each key is a full location path and the corresponding value is the list of device names found in that location.

Example output:

```
{
  "Piso 1, Sala": ["Luz", "Termostato", "Televisão", "Cortina"],
  "Piso 2, Quarto Principal": ["Luz"]
}
```

- **check_specific_device_settings(location, device)** — Returns the properties of a specific device, including current values and data types. This tool must always be called before `update_device_settings`, since it provides the necessary metadata about configurable settings. As a result, it also serves as a device selector where any subsequent `update_device_settings` calls apply only to the device specified here. This ordering constraint proved to be the most effective approach in practice, as without it the agent frequently attempted to apply updates

using properties from unrelated devices, which harmed the system's performance. To prevent this, the system dynamically restricts the tool set exposed to the agent and reinforces the intended sequence through prompt engineering.

Arguments:

- `location` — string identifying the device's location (e.g., "Piso 1, Sala").
- `device` — string with the device name (e.g., "Luz").

Output schema: a dictionary where each key is the name of a property and the corresponding value is a pair with the current value and its data type.

Example output:

```
{
  "On/Off": [true, "Boolean"],
  "Brightness": [100, "Integer"]
}
```

- **`update_device_settings(setting, value)`** — Applies a change to a previously selected device (as determined by the last call to `check_specific_device_settings`). To support the agent's reasoning, the tool output is structured to provide informative feedback: it returns a clear success message when the update is applied, or an explicit error explanation when the operation fails.

Arguments:

- `setting` — name of the configurable property (e.g., "Is_turned_on").
- `value` — new value as a string (e.g., "False", "50").

Output schema: a dictionary containing a status code and a descriptive message that either confirms success or explains the reason for failure.

Example output:

```
{
  "status_code": 200,
  "message": "The setting [\"Is_turned_on\"] was successfully changed."
}
```

Example output (error):

```
{
  "status_code": 404,
  "message": "The setting [\"On\"] could not be found."
}
```

C.1.2 Reminder Agent Tools

- **create_reminder(reminder_name, description, date_time, frequency)**
Creates a new reminder with a specified name, description, scheduled time, and frequency. Internally, the tool validates the inputs, normalizes the frequency, derives a CRON expression, and registers the reminder in the background-jobs service. In the initial design, the tool expected the agent to supply a ready-made CRON expression, which the LLM had to generate directly from the user's utterance. This approach proved unreliable, so CRON expression derivation was moved into the tool and a higher-level frequency argument was introduced. The tool now computes the CRON expression from date_time and frequency, removing the need for the LLM to emit raw CRON syntax.

Arguments:

- reminder_name – short title for the reminder.
- description – additional text describing the reminder.
- date_time – ISO 8601 timestamp (e.g., "2025-05-15T10:00:00").
- frequency – can be a literal (one of "ONCE", "DAILY", "WEEKDAYS", "MONTHLY", "YEARLY"), or a list with numbers representing days of the week (0 = Sunday, 1 = Monday, etc.).

Output schema: a simple dictionary with the created reminder id or validation errors.

Example output:

```
{
  "ReminderId": "8b5f1f7c-1c8a-41e2-8f49-f52f4f27c3fb"
}
```

Example output (error):

```
{
  "errors": "Date_time must not be in the past."
}
```

- **get_reminders()** – Retrieves the list of active reminders. Internally, reminders are stored as periodic tasks defined by CRON expressions, but the tool translates these into more digestible, human-readable descriptions (e.g., 0 9 * * * → “every day at 09:00”) so the agent has an easier job when the user refers to a reminder by its frequency.

Arguments: *None*

Output schema: a list of reminders, each containing an identifier, name, description, CRON description and next execution time.

Example output:

```
[
  {
    "id": "rem-123",
    "reminder_name": "Meeting",
    "description": "Team meeting",
    "cron_description": "at 10:00 on 15 Feb",
    "next_execution": "2025-02-15T10:00:00"
  },
  {
    "id": "rem-456",
    "reminder_name": "Presentation",
    "description": "Project presentation",
    "cron_description": "every day at 9:00",
    "next_execution": "2025-02-20T09:00:00"
  }
]
```

- **delete_reminder(reminder_id)** — Deletes an existing reminder by its unique identifier. The tool validates that the provided ID is a UUID and attempts removal. If the ID is invalid or does not correspond to any reminder, an error is returned. In practice, this tool is often used together with `get_reminders()` to retrieve the identifier of the reminder the user intends to delete.

Arguments:

- `reminder_id` — UUID of the reminder to remove.

Output schema: a success message on success, or an error message.

Example output (success):

```
{
  "message": "Reminder deleted successfully."
}
```

Example output (error):

```
{
  "errors": "The reminder does not exist."
}
```

C.1.3 Weather Agent Tools

- **get_daily_forecast(city, country)** — Retrieves the *daily weather forecast* for the coming days (up to one week ahead) in a given city and country, providing a structured summary for each day with normalized values.

Arguments:

- `city` — name of the target city (e.g., “Lisbon”).

- `country` – name of the country (e.g., “Portugal”).

Output schema: A structured object containing city, country and a list of forecast items. Each item includes:

- `date` – date in format YYYY-MM-DD
- `weekday` – weekday (e.g., “Monday”)
- `max_temperature`, `min_temperature` – integers in °C
- `weather` – textual weather description

Example output:

```
{
  "city": "Lisbon",
  "country": "Portugal",
  "daily_forecast": [
    {
      "date": "2025-03-15",
      "weekday": "Saturday",
      "max_temperature": 19,
      "min_temperature": 11,
      "weather": "Clear sky"
    }
  ]
}
```

- `get_hourly_forecast(city, country, target_date)` – Retrieves the *hourly forecast* for a specific day in a given location. The tool validates the date format (YYYY-MM-DD) and filters forecast entries to the requested day only.

Arguments:

- `city` – name of the target city.
- `country` – name of the country.
- `target_date` – target date in format YYYY-MM-DD.

Output schema: A structured object with:

- `city`, `country`
- `date` – the requested date
- `weekday` – weekday
- `forecast` – list of hourly forecast entries, each with:

- * hour — time in HH:MM
- * temperature — integer in °C
- * weather — textual description

Example output:

```
{
  "city": "Lisbon",
  "country": "Portugal",
  "date": "2025-03-15",
  "weekday": "Saturday",
  "forecast": [
    { "hour": "10:00", "temperature": 15, "weather": "Scattered clouds" },
    { "hour": "13:00", "temperature": 18, "weather": "Clear sky" }
  ]
}
```

C.1.4 Memory Agent Tools

- `manage_memory(action, content=None, id=None)` — Handles memory operations by creating, updating, deleting or skipping entries.

Arguments:

- action — one of "create", "update", "delete", or "nop".
- content — factual information to be stored (required for "create" and "update").
- id — unique identifier of a memory entry (required for "update" and "delete").

Output schema: The output is a dictionary that always contains two fields. The first field, `status`, indicates whether the operation was successful or not, taking the value "success" in the case of success and "error" in the case of failure. The second field, `message`, is a string that provides a natural language description of the outcome of the operation, intended to be easily interpretable by the agent.

Example output (success):

```
{
  "status": "success",
  "message": "The fact ['Is allergic to peanuts'] was stored successfully."
}
```

Example output (error):

```
{
  "status": "error",
  "message": "The fact could not be deleted because it does not exist."
}
```


C.2 STT Test Set Phrases

The Table 13 presents a curated subset of 15 ground-truth transcriptions used to evaluate the Speech-to-Text (STT) models in Subsection 4.5.4. Chosen to reflect the variety of the 50-clip test set, these phrases span essential interaction types.

Table 13: Representative subset of tested phrases for STT model evaluation

#	Transcription
1.	Apaga a luz da sala.
2.	Como vai estar o tempo amanhã?
3.	Clarisse, que horas são?
4.	Qual é o brilho da luz da sala?
5.	Clarisse, liga a televisão da sala.
6.	Clarisse, define um alarme para as 7 da manhã.
7.	Clarisse, adiciona leite à lista de compras.
8.	Clarisse, vai chover amanhã?
9.	Clarisse, apaga as luzes da sala e fecha os estores, por favor.
10.	Clarisse, qual é a previsão do tempo para o fim de semana no Porto?
11.	Podes diminuir a luz do quarto para 20%?
12.	Que dispositivos estão ligados na cozinha neste momento?
13.	Amanhã vou precisar de uma sombrinha?
14.	Se puderes, traz-me um café cheio e um pastel de nata bem quentinho, daqueles que acabaram de sair do forno, como só se faz cá.
15.	Ontem o jogo foi renhido até ao fim, mas aquele golo aos noventa minutos deixou toda a gente de boca aberta. Só mesmo em Portugal é que se vive o futebol assim!

C.3 Load Testing Configuration

The following command was used to perform the LLM load and concurrency benchmark described in Subsection 5.2.3:

```
vllm bench serve \
  --model neuralmagic-ent/Llama-3.3-70B-Instruct-quantized.w8a8 \
  --backend openai-chat \
  --base-url http://192.168.155.56:8081 \
  --dataset-name share-gpt \
  --random_input_len 1024 \
  --random_output_len 128 \
  --num-prompts 1000 \
  --ramp-up-strategy linear \
  --ramp-up-start-rps 2 \
  --ramp-up-end-rps 8 \
  --percentile-metrics ttft,tpot,itl \
  --metric-percentiles "50,90,95,99" \
  --save_result \
  --save_detailed \
  --result_dir bench_results \
```